附录 B　关键模型源程序代码

本附录仅收录与四个问题、模型递进、最终提交和指标复验直接相关的 Python 源程序。旧版候选搜索、GAN 扩增、绘图、文档生成和材料打包脚本不属于正文模型，故不并入论文附录。代码已统一补充必要注释并整理命名，计算公式、题设字段、模型参数和输出口径均保持不变，运行方式见主论文附录 A。

B.1 问题一：problem1_multichannel_integration.py

```python
from __future__ import annotations

# 论文附录复现代码。
# 问题一：重建五声道观测并比较固定权重积分。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

from pathlib import Path

import numpy as np
import pandas as pd

try:
    import matplotlib.pyplot as plt
except ModuleNotFoundError:
    plt = None

try:
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import landscape, A4
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    from reportlab.pdfgen import canvas
except ModuleNotFoundError:
    colors = None
    landscape = None
    A4 = None
    pdfmetrics = None
    TTFont = None
    canvas = None


ROOT = Path(__file__).resolve().parents[2]
DATA_DIR = next(p for p in ROOT.iterdir() if p.is_dir() and (p / "attachment1_window_data.csv").exists())
OUT_DIR = Path(__file__).resolve().parent / "verification_outputs"
FIG_DIR = Path(__file__).resolve().parent / "verification_figures"
REPORT = Path(__file__).resolve().parent / "verification_reports" / "INTEGRATION_METHOD_ANALYSIS.md"


def loadInputs() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    win = pd.read_csv(DATA_DIR / "attachment1_window_data.csv", encoding="utf-8-sig")
    raw = pd.read_csv(DATA_DIR / "attachment6_window_raw_samples.csv", encoding="utf-8-sig")
    geo = pd.read_csv(DATA_DIR / "attachment7_meter_geometry.csv", encoding="utf-8-sig")
    return win, raw, geo


def reconstructChordsFromRaw(raw: pd.DataFrame, geo: pd.DataFrame) -> pd.DataFrame:
    rows = geo[geo["record_type"].eq("chord_mapping")].copy()
    diff_cols = [f"diff_ns_{i}" for i in range(10)]
    raw_mean = raw.groupby("window_id")[diff_cols].mean()
    rec = pd.DataFrame(index=raw_mean.index)
    for _, row in rows.sort_values("chord_id").iterrows():
        chord = row["object_id"]
        a_col = row["raw_diff_col_a"]
        b_col = row["raw_diff_col_b"]
        rec[chord] = (
            raw_mean[a_col] * float(row["raw_diff_gain_a_mps_per_ns"])
            + raw_mean[b_col] * float(row["raw_diff_gain_b_mps_per_ns"])
            + float(row["raw_diff_intercept_mps"])
        )
    return rec.reset_index()


def getArea(geo: pd.DataFrame) -> float:
    return float(geo.loc[geo["parameter"].eq("effective_pipe_area_m2"), "value"].iloc[0])


def getWeights(geo: pd.DataFrame) -> pd.DataFrame:
    cols = [
        "object_id",
        "chord_id",
        "chord_order",
        "chord_position_label",
        "normalized_channel_height",
        "weight_phys6",
        "weight_owics",
        "weight_flowcal_gauss_jacobi",
        "weight_lagrange",
        "weight_equal",
```

```
    ]
    rows = geo[geo["record_type"].eq("chord_mapping")][cols].copy()
    return rows.sort_values("chord_id").reset_index(drop=True)


def addEstimate(est: pd.DataFrame, method: str, volume: pd.Series, source: str) -> pd.DataFrame:
    tmp = est[[
        "window_id",
        "date",
        "flow_point",
        "condition_note",
        "is_disturbed",
        "disturbance_id",
        "duration_s",
        "standard_volume_m3",
    ]].copy()
    tmp["method"] = method
    tmp["source"] = source
    tmp["estimate_volume_m3"] = volume.to_numpy(dtype=float)
    tmp["error_pct"] = (tmp["estimate_volume_m3"] - tmp["standard_volume_m3"]) / tmp["standard_volume_m3"] * 100.0
    return tmp


def buildEstimates(win: pd.DataFrame, rec_chords: pd.DataFrame, geo: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
    area = getArea(geo)
    weights = getWeights(geo)
    merged = win.merge(rec_chords, on="window_id", suffixes=("", "_raw_rec"))
    estimates: list[pd.DataFrame] = []

    # Raw single-chord estimates: useful as a lower-level diagnostic, not a recommended meter output.
    for i in range(5):
        volume = area * merged["duration_s"] * merged[f"chord{i}_raw_rec"]
        estimates.append(addEstimate(merged, f"raw_single_chord{i}", volume, "attachment6_raw"))

    weight_methods = {
        "raw_phys6_weighted": "weight_phys6",
        "raw_owics_weighted": "weight_owics",
        "raw_gauss_jacobi_weighted": "weight_flowcal_gauss_jacobi",
        "raw_lagrange_weighted": "weight_lagrange",
        "raw_equal_weighted": "weight_equal",
    }
    for method, w_col in weight_methods.items():
        volume = pd.Series(0.0, index=merged.index)
        for i, w in enumerate(weights[w_col].astype(float).to_list()):
            volume = volume + area * merged["duration_s"] * w * merged[f"chord{i}_raw_rec"]
        estimates.append(addEstimate(merged, method, volume, "attachment6_raw_plus_attachment7_weights"))

    baseline_methods = {
        "base_phys6": "phys6_volume_m3",
        "base_owics": "owics_volume_m3",
        "base_lagrange": "lagrange_volume_m3",
        "base_equal_weight": "equal_weight_volume_m3",
    }
    for method, col in baseline_methods.items():
        estimates.append(addEstimate(merged, method, merged[col], "attachment1_baseline"))

    return pd.concat(estimates, ignore_index=True), weights


def summarize(group: pd.DataFrame, by: list[str]) -> pd.DataFrame:
    def aggOne(x: pd.DataFrame) -> pd.Series:
        e = x["error_pct"].astype(float)
        return pd.Series(
            {
                "window_count": len(x),
                "mae_pct": e.abs().mean(),
                "mean_error_pct": e.mean(),
                "rmse_pct": float(np.sqrt(np.mean(e**2))),
                "max_abs_error_pct": e.abs().max(),
            }
        )

    return group.groupby(by, dropna=False).apply(aggOne).reset_index()


def writeOutputs(estimates: pd.DataFrame, weights: pd.DataFrame, rec_chords: pd.DataFrame, win: pd.DataFrame) -> dict[str, pd.DataFrame]:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    rec_check = win[["window_id", "chord0", "chord1", "chord2", "chord3", "chord4"]].merge(rec_chords, on="window_id", suffixes=("_attachment1", "_raw_rec"))
    check_rows = []
    for i in range(5):
        err = rec_check[f"chord{i}_raw_rec"] - rec_check[f"chord{i}_attachment1"]
        check_rows.append(
            {
                "chord": f"chord{i}",
                "mae_mps": err.abs().mean(),
                "mean_error_mps": err.mean(),
                "max_abs_error_mps": err.abs().max(),
            }
        )
    chord_check = pd.DataFrame(check_rows)

    overall = summarize(estimates, ["method", "source"]).sort_values("mae_pct")
```

```
    by_flow = summarize(estimates, ["method", "source", "flow_point"]).sort_values(["method", "flow_point"])
    by_condition = summarize(estimates, ["method", "source", "condition_note"]).sort_values(["method", "condition_note"])
    by_disturbed = summarize(estimates, ["method", "source", "is_disturbed"]).sort_values(["method", "is_disturbed"])

    weights.to_csv(OUT_DIR / "integration_weights.csv", index=False, encoding="utf-8-sig")
    rec_chords.to_csv(OUT_DIR / "reconstructed_raw_chords.csv", index=False, encoding="utf-8-sig")
    chord_check.to_csv(OUT_DIR / "raw_chord_reconstruction_check.csv", index=False, encoding="utf-8-sig")
    estimates.to_csv(OUT_DIR / "integration_estimates_long.csv", index=False, encoding="utf-8-sig")
    overall.to_csv(OUT_DIR / "integration_overall_metrics.csv", index=False, encoding="utf-8-sig")
    by_flow.to_csv(OUT_DIR / "integration_metrics_by_flow_point.csv", index=False, encoding="utf-8-sig")
    by_condition.to_csv(OUT_DIR / "integration_metrics_by_condition.csv", index=False, encoding="utf-8-sig")
    by_disturbed.to_csv(OUT_DIR / "integration_metrics_by_disturbed.csv", index=False, encoding="utf-8-sig")
    return {
        "overall": overall,
        "by_flow": by_flow,
        "by_condition": by_condition,
        "by_disturbed": by_disturbed,
        "chord_check": chord_check,
    }


def plotFigures(tables: dict[str, pd.DataFrame]) -> None:
    if plt is None:
        plotFiguresReportlab(tables)
        return

    plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "Arial Unicode MS", "DejaVu Sans"]
    plt.rcParams["axes.unicode_minus"] = False

    selected = [
        "base_phys6",
        "base_owics",
        "base_lagrange",
        "base_equal_weight",
        "raw_gauss_jacobi_weighted",
        "raw_lagrange_weighted",
        "raw_equal_weighted",
    ]
    overall = tables["overall"][tables["overall"]["method"].isin(selected)].copy()
    overall = overall.sort_values("mae_pct", ascending=True)
    fig, ax = plt.subplots(figsize=(8.2, 4.4))
    ax.barh(overall["method"], overall["mae_pct"], color="#3B82F6")
    ax.set_xlabel("MAE / %")
    ax.set_ylabel("method")
    ax.grid(axis="x", linestyle="--", alpha=0.35)
    fig.tight_layout()
    fig.savefig(FIG_DIR / "fig_integration_overall_mae.pdf")
    plt.close(fig)

    flow = tables["by_flow"][tables["by_flow"]["method"].isin(["base_lagrange", "raw_lagrange_weighted", "raw_gauss_jacobi_weighted"])].copy()
    fig, ax = plt.subplots(figsize=(8.4, 4.6))
    for method, sub in flow.groupby("method"):
        sub = sub.sort_values("flow_point")
        ax.plot(sub["flow_point"], sub["mae_pct"], marker="o", label=method)
    ax.set_xlabel("flow point")
    ax.set_ylabel("MAE / %")
    ax.grid(True, linestyle="--", alpha=0.35)
    ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig(FIG_DIR / "fig_integration_flow_mae.pdf")
    plt.close(fig)


def reportlabFont() -> str:
    if pdfmetrics is None:
        return "Helvetica"
    font_path = Path(r"C:\Windows\Fonts\msyh.ttc")
    if font_path.exists():
        try:
            pdfmetrics.registerFont(TTFont("MSYH", str(font_path), subfontIndex=0))
            return "MSYH"
        except Exception:
            return "Helvetica"
    return "Helvetica"


def plotFiguresReportlab(tables: dict[str, pd.DataFrame]) -> None:
    if canvas is None:
        return
    font = reportlabFont()
    selected = [
        "base_phys6",
        "base_lagrange",
        "raw_phys6_weighted",
        "raw_lagrange_weighted",
        "base_owics",
        "raw_gauss_jacobi_weighted",
        "base_equal_weight",
        "raw_equal_weighted",
    ]
    overall = tables["overall"][tables["overall"]["method"].isin(selected)].sort_values("mae_pct", ascending=True)
    c = canvas.Canvas(str(FIG_DIR / "fig_integration_overall_mae.pdf"), pagesize=landscape(A4))
    w, h = landscape(A4)
```

```
    c.setFont(font, 14)
    c.drawString(50, h - 42, "多声道积分方法总体 MAE 对比")
    max_mae = float(overall["mae_pct"].max())
    x0, y0, bar_w, row_h = 230, h - 82, 430, 24
    for idx, row in enumerate(overall.itertuples(index=False)):
        y = y0 - idx * row_h
        width = bar_w * float(row.mae_pct) / max_mae if max_mae > 0 else 0
        c.setFont(font, 9)
        c.setFillColor(colors.HexColor("#111827"))
        c.drawRightString(x0 - 8, y + 5, str(row.method))
        c.setFillColor(colors.HexColor("#3B82F6"))
        c.rect(x0, y, width, 12, stroke=0, fill=1)
        c.setFillColor(colors.HexColor("#111827"))
        c.drawString(x0 + width + 6, y + 2, f"{float(row.mae_pct):.3f}%")
    c.setFont(font, 9)
    c.setFillColor(colors.HexColor("#475569"))
    c.drawString(50, 38, "数据来源: attachment1 基础估计、attachment6 原始声道序列、attachment7 固定积分权重。")
    c.save()

    flow = tables["by_flow"][tables["by_flow"]["method"].isin(["base_lagrange", "raw_lagrange_weighted", "raw_gauss_jacobi_weighted"])].copy()
    c = canvas.Canvas(str(FIG_DIR / "fig_integration_flow_mae.pdf"), pagesize=landscape(A4))
    w, h = landscape(A4)
    c.setFont(font, 14)
    c.drawString(50, h - 42, "不同流量点下积分方法 MAE 对比")
    methods = ["base_lagrange", "raw_lagrange_weighted", "raw_gauss_jacobi_weighted"]
    colors_list = [colors.HexColor("#2563EB"), colors.HexColor("#16A34A"), colors.HexColor("#DC2626")]
    flow_points = sorted(flow["flow_point"].unique())
    max_val = float(flow["mae_pct"].max())
    chart_x, chart_y, chart_w, chart_h = 70, 95, 680, 390
    c.setStrokeColor(colors.HexColor("#CBD5E1"))
    c.line(chart_x, chart_y, chart_x + chart_w, chart_y)
    c.line(chart_x, chart_y, chart_x, chart_y + chart_h)
    for mi, method in enumerate(methods):
        sub = flow[flow["method"].eq(method)].set_index("flow_point")
        pts = []
        for i, fp in enumerate(flow_points):
            x = chart_x + (i + 0.5) * chart_w / len(flow_points)
            val = float(sub.loc[fp, "mae_pct"]) if fp in sub.index else 0.0
            y = chart_y + val / max_val * chart_h if max_val > 0 else chart_y
            pts.append((x, y))
            c.setFillColor(colors_list[mi])
            c.circle(x, y, 3, stroke=0, fill=1)
        c.setStrokeColor(colors_list[mi])
        for (x1, y1), (x2, y2) in zip(pts, pts[1:]):
            c.line(x1, y1, x2, y2)
    c.setFont(font, 9)
    c.setFillColor(colors.HexColor("#111827"))
    for i, fp in enumerate(flow_points):
        x = chart_x + (i + 0.5) * chart_w / len(flow_points)
        c.drawCentredString(x, chart_y - 18, str(fp))
    c.drawString(chart_x + chart_w / 2 - 30, chart_y - 42, "flow point")
    c.drawString(chart_x - 45, chart_y + chart_h + 5, "MAE / %")
    for mi, method in enumerate(methods):
        c.setFillColor(colors_list[mi])
        c.rect(560, h - 70 - mi * 18, 10, 10, stroke=0, fill=1)
        c.setFillColor(colors.HexColor("#111827"))
        c.drawString(576, h - 72 - mi * 18, method)
    c.save()


def markdownTable(df: pd.DataFrame, floatfmt: str = ".6g") -> str:
    if df.empty:
        return "_无数据_"
    headers = [str(c) for c in df.columns]
    lines = [
        "| " + " | ".join(headers) + " |",
        "| " + " | ".join(["---"] * len(headers)) + " |",
    ]
    for _, row in df.iterrows():
        vals = []
        for value in row:
            if isinstance(value, (float, np.floating)):
                vals.append(format(float(value), floatfmt))
            else:
                vals.append(str(value))
        lines.append("| " + " | ".join(vals) + " |")
    return "\n".join(lines)


def writeReport(tables: dict[str, pd.DataFrame], weights: pd.DataFrame) -> None:
    top = tables["overall"].head(12)
    lagrange_weight = weights[[
        "object_id",
        "chord_position_label",
        "normalized_channel_height",
        "weight_lagrange",
        "weight_owics",
        "weight_flowcal_gauss_jacobi",
        "weight_phys6",
        "weight_equal",
    ]]
    raw_lagrange_flow = tables["by_flow"][tables["by_flow"]["method"].eq("raw_lagrange_weighted")][[
        "flow_point",
```

```
        "window_count",
        "mae_pct",
        "mean_error_pct",
        "max_abs_error_pct",
    ]]
    disturbed = tables["by_disturbed"][tables["by_disturbed"]["method"].isin(["base_lagrange", "raw_lagrange_weighted", "raw_gauss_jacobi_weighted"])][[
        "method",
        "is_disturbed",
        "window_count",
        "mae_pct",
        "mean_error_pct",
        "max_abs_error_pct",
    ]]

    text = f"""# 多声道积分方法误差统计

## 1. Lagrange 型积分权重的物理含义

Lagrange 型五声道积分不是简单认为某一条声道“更重要”，而是把五个弦高处的线平均速度看成速度剖面的采样点，先用 Lagrange 插值基函数重构径向剖面，再对圆截面平均速度积分。附件 7 中的权重已经把声道位置、圆截面面积权和插值基函数积分结果合并为固定系数。

| 声道 | 位置 | 归一化弦高 | Lagrange 权重 | 物理含义 |
| --- | --- | ---: | ---: | --- |
| chord0 | edge_1 | 0.727425 | 0.266260 | 上侧边缘弦，帮助描述近壁速度梯度和剖面边缘曲率 |
| chord1 | inner_1 | 0.266219 | 0.096121 | 上内侧弦，约束中心与边缘之间的剖面形状 |
| chord2 | center | 0.000000 | 0.275238 | 中心弦，代表高速核心区，是单个最大权重 |
| chord3 | inner_2 | -0.266219 | 0.096121 | 下内侧弦，与 chord1 对称，用于抵消上下偏流 |
| chord4 | edge_2 | -0.727425 | 0.266260 | 下侧边缘弦，与 chord0 对称，约束另一侧近壁区域 |

这些权重满足左右/上下对称关系：`w0=w4`、`w1=w3`，且 `Σw_i=1`。边缘弦权重大并不表示边缘速度更大，而是因为 Lagrange 插值积分中边缘节点对剖面曲率和近壁区域积分贡献较强；中心弦权重大则对应管道高速核心区域的代表性。

## 2. 附件 6 原始声道序列重构口径

附件 7 给出了每条窗口声道与附件 6 原始 A/B 两组时间差列的映射。本文采用：

```text
u_i = gain_a,i * mean(diff_a,i) + gain_b,i * mean(diff_b,i) + intercept_i
V = A * duration_s * Σ w_i u_i
```

其中 `A` 为附件 7 的等效截面积。该口径不使用标准体积，只利用原始声道观测序列、声道映射和固定积分权重。

原始序列重构 `chord0--chord4` 与附件 1 窗口声道的差异如下：

{markdownTable(tables["chord_check"])}

3. 总体误差统计

{markdownTable(top)}

4. Lagrange 型积分按流量点表现

{markdownTable(raw_lagrange_flow)}

5. 扰流与非扰流条件对比

{markdownTable(disturbed)}

6. 输出文件

- `code/outputs/integration_weights.csv`：附件 7 五声道权重。
- `code/outputs/reconstructed_raw_chords.csv`：由附件 6 原始序列重构的五声道速度。
- `code/outputs/integration_estimates_long.csv`：单声道、多声道积分和附件 1 基础估计的长表。
- `code/outputs/integration_overall_metrics.csv`：总体误差统计。
- `code/outputs/integration_metrics_by_flow_point.csv`：按流量点统计。
- `code/outputs/integration_metrics_by_condition.csv`：按工况说明统计。
- `figures/fig_integration_overall_mae.pdf`：总体 MAE 对比图；优先用 `matplotlib`，缺失时用 ReportLab 兜底生成。
- `figures/fig_integration_flow_mae.pdf`：流量点 MAE 对比图；优先用 `matplotlib`，缺失时用 ReportLab 兜底生成。
"""
    REPORT.write_text(text, encoding="utf-8")

def main() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    REPORT.parent.mkdir(parents=True, exist_ok=True)
    win, raw, geo = loadInputs()
    rec_chords = reconstructChordsFromRaw(raw, geo)
    estimates, weights = buildEstimates(win, rec_chords, geo)
    tables = writeOutputs(estimates, weights, rec_chords, win)
    plotFigures(tables)
    writeReport(tables, weights)
    print("Report:", REPORT)
    print("Overall:", OUT_DIR / "integration_overall_metrics.csv")
    if plt is None and canvas is not None:
        print("Figures generated with ReportLab fallback:", FIG_DIR / "fig_integration_overall_mae.pdf", FIG_DIR / "fig_integration_flow_mae.pdf")
    elif plt is None:
        print("Figures skipped: neither matplotlib nor reportlab is installed in the current Python environment.")
    else:
        print("Figures:", FIG_DIR / "fig_integration_overall_mae.pdf", FIG_DIR / "fig_integration_flow_mae.pdf")
    print(tables["overall"].head(10).to_string(index=False))

```

```

if __name__ == "__main__":
    main()
```

B.2 问题二：problem2_d0_calibration.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题二：在 D0 窗口上拟合正常工况多声道校准模型。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

try:
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import landscape, A4
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    from reportlab.pdfgen import canvas
except ModuleNotFoundError:
    colors = None
    landscape = None
    A4 = None
    pdfmetrics = None
    TTFont = None
    canvas = None


ROOT = Path(__file__).resolve().parents[2]
DATA_DIR = next(p for p in ROOT.iterdir() if p.is_dir() and (p / "attachment1_window_data.csv").exists())
OUT_DIR = Path(__file__).resolve().parent / "verification_outputs"
FIG_DIR = Path(__file__).resolve().parent / "verification_figures"
REPORT = Path(__file__).resolve().parent / "verification_reports" / "NO_DISTURBANCE_MODEL_REPORT.md"


@dataclass
class CorrectionScaler:
    cols: list[str]
    mean: np.ndarray
    scale: np.ndarray


@dataclass
class FittedModel:
    name: str
    prior_name: str
    lambda_w: float
    correction: str
    lambda_c: float
    weights: np.ndarray
    beta: np.ndarray
    scaler: CorrectionScaler
    area: float
    factor_clip: tuple[float, float] = (0.97, 1.03)


def loadInputs() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    win = pd.read_csv(DATA_DIR / "attachment1_window_data.csv", encoding="utf-8-sig")
    raw = pd.read_csv(DATA_DIR / "attachment6_window_raw_samples.csv", encoding="utf-8-sig")
    geo = pd.read_csv(DATA_DIR / "attachment7_meter_geometry.csv", encoding="utf-8-sig")
    return win, raw, geo


def getArea(geo: pd.DataFrame) -> float:
    return float(geo.loc[geo["parameter"].eq("effective_pipe_area_m2"), "value"].iloc[0])


def chordRows(geo: pd.DataFrame) -> pd.DataFrame:
    return geo[geo["record_type"].eq("chord_mapping")].sort_values("chord_id").reset_index(drop=True)


def reconstructChords(raw: pd.DataFrame, geo: pd.DataFrame) -> pd.DataFrame:
    rows = chordRows(geo)
    diff_cols = [f"diff_ns_{i}" for i in range(10)]
    raw_mean = raw.groupby("window_id")[diff_cols].mean()
    rec = pd.DataFrame(index=raw_mean.index)
    for _, row in rows.iterrows():
        rec[row["object_id"]] = (
            raw_mean[row["raw_diff_col_a"]] * float(row["raw_diff_gain_a_mps_per_ns"])
            + raw_mean[row["raw_diff_col_b"]] * float(row["raw_diff_gain_b_mps_per_ns"])
            + float(row["raw_diff_intercept_mps"])
        )
    return rec.reset_index()


```

```
def attachFeatures(win: pd.DataFrame, rec: pd.DataFrame, geo: pd.DataFrame) -> pd.DataFrame:
    data = win.merge(rec, on="window_id", suffixes=("", "_raw_rec"))
    chord = data[[f"chord{i}_raw_rec" for i in range(5)]].astype(float)
    mean = chord.mean(axis=1).replace(0, np.nan)
    data["raw_chord_mean"] = mean
    data["prof_center_edges"] = (chord["chord2_raw_rec"] - 0.5 * (chord["chord0_raw_rec"] + chord["chord4_raw_rec"])) / mean
    data["prof_edge_inner"] = (0.5 * (chord["chord0_raw_rec"] + chord["chord4_raw_rec"]) - 0.5 * (chord["chord1_raw_rec"] + chord["chord3_raw_rec"])) / mean
    data["prof_top_bottom"] = (chord["chord0_raw_rec"] - chord["chord4_raw_rec"]) / mean
    data["prof_inner_skew"] = (chord["chord1_raw_rec"] - chord["chord3_raw_rec"]) / mean
    data["prof_range"] = (chord.max(axis=1) - chord.min(axis=1)) / mean
    data["q"] = (data["flow_point"].astype(float) - 50.0) / 30.0
    data["q2"] = data["q"] ** 2
    data["ref_velocity_m_s"] = data["standard_volume_m3"].astype(float) / (getArea(geo) * data["duration_s"].astype(float))
    data = data.fillna(0.0)
    return data


def getPriors(geo: pd.DataFrame) -> dict[str, np.ndarray]:
    rows = chordRows(geo)
    return {
        "phys6": rows["weight_phys6"].astype(float).to_numpy(),
        "owics_gauss_jacobi": rows["weight_flowcal_gauss_jacobi"].astype(float).to_numpy(),
        "lagrange": rows["weight_lagrange"].astype(float).to_numpy(),
        "equal": rows["weight_equal"].astype(float).to_numpy(),
    }


def fitSymmetricWeights(train: pd.DataFrame, prior: np.ndarray, lambda_w: float) -> np.ndarray:
    x = train[[f"chord{i}_raw_rec" for i in range(5)]].astype(float).to_numpy()
    y = train["ref_velocity_m_s"].astype(float).to_numpy()
    # Enforce w0=w4=e, w1=w3=i, w2=1-2e-2i.
    z = np.column_stack([x[:, 0] + x[:, 4] - 2 * x[:, 2], x[:, 1] + x[:, 3] - 2 * x[:, 2]])
    b = y - x[:, 2]
    theta0 = np.array([0.5 * (prior[0] + prior[4]), 0.5 * (prior[1] + prior[3])], dtype=float)
    lhs = z.T @ z + lambda_w * np.eye(2)
    rhs = z.T @ b + lambda_w * theta0
    theta = np.linalg.solve(lhs, rhs)
    weights = np.array([theta[0], theta[1], 1 - 2 * theta[0] - 2 * theta[1], theta[1], theta[0]], dtype=float)
    if np.all(weights >= -1e-9):
        weights = np.maximum(weights, 0.0)
        return weights / weights.sum()

    # Low-dimensional projection fallback onto the physical simplex.
    grid = np.linspace(0.0, 0.5, 501)
    best_obj = np.inf
    best = np.array([theta0[0], theta0[1]])
    for e in grid:
        max_i = (1 - 2 * e) / 2
        if max_i < 0:
            continue
        for i in grid[grid <= max_i]:
            pred = x[:, 2] + e * z[:, 0] + i * z[:, 1]
            obj = float(np.mean((pred - y) ** 2) + lambda_w * np.sum((np.array([e, i]) - theta0) ** 2))
            if obj < best_obj:
                best_obj = obj
                best = np.array([e, i])
    weights = np.array([best[0], best[1], 1 - 2 * best[0] - 2 * best[1], best[1], best[0]], dtype=float)
    return weights / weights.sum()


def baseVolume(data: pd.DataFrame, weights: np.ndarray, area: float) -> np.ndarray:
    x = data[[f"chord{i}_raw_rec" for i in range(5)]].astype(float).to_numpy()
    velocity = x @ weights
    return area * data["duration_s"].astype(float).to_numpy() * velocity


def correctionColumns(kind: str) -> list[str]:
    if kind == "none":
        return []
    if kind == "calibration":
        return ["intercept"]
    if kind == "flow_curve":
        return ["intercept", "q", "q2"]
    if kind == "profile_flow":
        return ["intercept", "q", "q2", "prof_center_edges", "prof_edge_inner", "prof_top_bottom", "prof_inner_skew"]
    raise ValueError(kind)


def makeCorrectionMatrix(data: pd.DataFrame, cols: list[str], scaler: CorrectionScaler | None = None) -> tuple[np.ndarray, CorrectionScaler]:
    if not cols:
        return np.zeros((len(data), 0)), CorrectionScaler(cols=[], mean=np.array([]), scale=np.array([]))
    raw_cols = []
    arrays = []
    for col in cols:
        if col == "intercept":
            arrays.append(np.ones(len(data)))
        else:
            raw_cols.append(col)
            arrays.append(data[col].astype(float).to_numpy())
    matrix = np.column_stack(arrays)
    if scaler is None:
        mean = np.zeros(len(cols))
        scale = np.ones(len(cols))
```

```
        for j, col in enumerate(cols):
            if col == "intercept":
                continue
            v = matrix[:, j]
            mean[j] = float(np.mean(v))
            s = float(np.std(v))
            scale[j] = s if s > 1e-12 else 1.0
        scaler = CorrectionScaler(cols=cols, mean=mean, scale=scale)
    scaled = matrix.copy()
    for j, col in enumerate(cols):
        if col == "intercept":
            continue
        scaled[:, j] = (scaled[:, j] - scaler.mean[j]) / scaler.scale[j]
    return scaled, scaler


def fitCorrection(train: pd.DataFrame, base: np.ndarray, kind: str, lambda_c: float) -> tuple[np.ndarray, CorrectionScaler]:
    cols = correctionColumns(kind)
    if not cols:
        return np.zeros(0), CorrectionScaler(cols=[], mean=np.array([]), scale=np.array([]))
    target = train["standard_volume_m3"].astype(float).to_numpy() / np.maximum(base, 1e-12) - 1.0
    x, scaler = makeCorrectionMatrix(train, cols)
    penalty = np.eye(x.shape[1])
    if "intercept" in cols:
        penalty[cols.index("intercept"), cols.index("intercept")] = 0.0
    beta = np.linalg.solve(x.T @ x + lambda_c * penalty, x.T @ target)
    return beta, scaler


def fitModel(train: pd.DataFrame, area: float, prior_name: str, prior: np.ndarray, lambda_w: float, correction: str, lambda_c: float) -> FittedModel:
    weights = fitSymmetricWeights(train, prior, lambda_w)
    base = baseVolume(train, weights, area)
    beta, scaler = fitCorrection(train, base, correction, lambda_c)
    name = f"sym_weight_{prior_name}_lw{lambda_w:g}_{correction}_lc{lambda_c:g}"
    return FittedModel(name, prior_name, lambda_w, correction, lambda_c, weights, beta, scaler, area)


def predict(model: FittedModel, data: pd.DataFrame) -> np.ndarray:
    base = baseVolume(data, model.weights, model.area)
    cols = model.scaler.cols
    if not cols:
        return base
    x, _ = makeCorrectionMatrix(data, cols, model.scaler)
    factor = 1.0 + x @ model.beta
    factor = np.clip(factor, model.factor_clip[0], model.factor_clip[1])
    return base * factor


def errorPct(y_hat: np.ndarray, y: pd.Series) -> np.ndarray:
    return (y_hat - y.astype(float).to_numpy()) / y.astype(float).to_numpy() * 100.0


def evaluatePredictions(data: pd.DataFrame, method: str, source: str, y_hat: np.ndarray) -> pd.DataFrame:
    out = data[[
        "window_id",
        "date",
        "flow_point",
        "condition_note",
        "disturbance_id",
        "is_disturbed",
        "duration_s",
        "standard_volume_m3",
    ]].copy()
    out["method"] = method
    out["source"] = source
    out["estimate_volume_m3"] = y_hat
    out["error_pct"] = errorPct(y_hat, data["standard_volume_m3"])
    return out


def metricSummary(df: pd.DataFrame, by: list[str]) -> pd.DataFrame:
    def agg(sub: pd.DataFrame) -> pd.Series:
        e = sub["error_pct"].astype(float)
        return pd.Series(
            {
                "window_count": len(sub),
                "mae_pct": e.abs().mean(),
                "mean_error_pct": e.mean(),
                "rmse_pct": float(np.sqrt(np.mean(e**2))),
                "max_abs_error_pct": e.abs().max(),
            }
        )

    return df.groupby(by, dropna=False).apply(agg).reset_index()


def groupMetrics(pred: pd.DataFrame) -> pd.DataFrame:
    rows = []
    keys = ["method", "source", "date", "flow_point", "condition_note", "disturbance_id"]
    for key, sub in pred.groupby(keys, dropna=False):
        e = sub["error_pct"].astype(float)
        rows.append(
            dict(
```

```
                zip(keys, key),
                window_count=len(sub),
                group_mean_error_pct=e.mean(),
                group_abs_mean_error_pct=abs(e.mean()),
                group_sd_pct=e.std(ddof=1) if len(sub) >= 2 else np.nan,
                group_mae_pct=e.abs().mean(),
            )
        )
    return pd.DataFrame(rows)


def groupSummary(groups: pd.DataFrame, by: list[str]) -> pd.DataFrame:
    def agg(sub: pd.DataFrame) -> pd.Series:
        return pd.Series(
            {
                "group_count": len(sub),
                "repeat_group_count": sub["group_sd_pct"].notna().sum(),
                "mean_abs_group_mean_pct": sub["group_abs_mean_error_pct"].mean(),
                "max_abs_group_mean_pct": sub["group_abs_mean_error_pct"].max(),
                "mean_group_sd_pct": sub["group_sd_pct"].mean(),
                "max_group_sd_pct": sub["group_sd_pct"].max(),
            }
        )

    return groups.groupby(by, dropna=False).apply(agg).reset_index()


def leaveFlowCv(d0: pd.DataFrame, area: float, priors: dict[str, np.ndarray]) -> pd.DataFrame:
    rows = []
    flow_points = sorted(d0["flow_point"].unique())
    lambda_w_values = [0.01, 0.1, 1.0, 10.0, 100.0]
    lambda_c_values = [0.01, 0.1, 1.0, 10.0, 100.0]
    corrections = ["none", "calibration", "flow_curve", "profile_flow"]
    for prior_name, prior in priors.items():
        for lw in lambda_w_values:
            for correction in corrections:
                lc_iter = [0.0] if correction == "none" else lambda_c_values
                fold_errors = []
                for lc in lc_iter:
                    fold_errors.clear()
                    for fp in flow_points:
                        train = d0[d0["flow_point"].ne(fp)].copy()
                        valid = d0[d0["flow_point"].eq(fp)].copy()
                        model = fitModel(train, area, prior_name, prior, lw, correction, lc)
                        pred = predict(model, valid)
                        fold_errors.extend(errorPct(pred, valid["standard_volume_m3"]).tolist())
                    e = np.array(fold_errors)
                    rows.append(
                        {
                            "prior": prior_name,
                            "lambda_w": lw,
                            "correction": correction,
                            "lambda_c": lc,
                            "cv_window_count": len(e),
                            "cv_mae_pct": float(np.mean(np.abs(e))),
                            "cv_mean_error_pct": float(np.mean(e)),
                            "cv_rmse_pct": float(np.sqrt(np.mean(e**2))),
                            "cv_max_abs_error_pct": float(np.max(np.abs(e))),
                        }
                    )
    return pd.DataFrame(rows).sort_values(["cv_mae_pct", "cv_rmse_pct"]).reset_index(drop=True)


def finalPredictions(data: pd.DataFrame, model: FittedModel) -> pd.DataFrame:
    yuceJieguo = [evaluatePredictions(data, "d0_optimal_weight_profile_curve", "trained_on_D0_only", predict(model, data))]
    baselines = {
        "base_phys6": "phys6_volume_m3",
        "base_owics": "owics_volume_m3",
        "base_lagrange": "lagrange_volume_m3",
        "base_equal_weight": "equal_weight_volume_m3",
    }
    for method, col in baselines.items():
        yuceJieguo.append(evaluatePredictions(data, method, "attachment1_baseline", data[col].astype(float).to_numpy()))
    return pd.concat(yuceJieguo, ignore_index=True)


def writeCsvOutputs(
    data: pd.DataFrame,
    cv: pd.DataFrame,
    model: FittedModel,
    yuceJieguo: pd.DataFrame,
    groups: pd.DataFrame,
    weights_meta: pd.DataFrame,
) -> dict[str, pd.DataFrame]:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    cv.to_csv(OUT_DIR / "d0_model_cv_by_flow.csv", index=False, encoding="utf-8-sig")
    yuceJieguo.to_csv(OUT_DIR / "d0_model_predictions_long.csv", index=False, encoding="utf-8-sig")
    groups.to_csv(OUT_DIR / "d0_model_group_metrics.csv", index=False, encoding="utf-8-sig")
    groups[groups["disturbance_id"].eq("D0")].to_csv(OUT_DIR / "d0_model_group_metrics_d0_only.csv", index=False, encoding="utf-8-sig")
    weights = pd.DataFrame(
        {
            "chord": [f"chord{i}" for i in range(5)],
            "position": weights_meta["chord_position_label"],
```

```
            "normalized_height": weights_meta["normalized_channel_height"].astype(float),
            "fitted_weight": model.weights,
            "weight_phys6": weights_meta["weight_phys6"].astype(float),
            "weight_owics_gauss_jacobi": weights_meta["weight_flowcal_gauss_jacobi"].astype(float),
            "weight_lagrange": weights_meta["weight_lagrange"].astype(float),
            "weight_equal": weights_meta["weight_equal"].astype(float),
        }
    )
    weights.to_csv(OUT_DIR / "d0_fitted_weights.csv", index=False, encoding="utf-8-sig")
    correction = pd.DataFrame(
        {
            "term": model.scaler.cols,
            "coefficient": model.beta,
            "feature_mean": model.scaler.mean,
            "feature_scale": model.scaler.scale,
        }
    )
    correction.to_csv(OUT_DIR / "d0_correction_coefficients.csv", index=False, encoding="utf-8-sig")

    overall = metricSummary(yuceJieguo, ["method", "source"])
    d0_overall = metricSummary(yuceJieguo[yuceJieguo["disturbance_id"].eq("D0")], ["method", "source"])
    by_flow_d0 = metricSummary(yuceJieguo[yuceJieguo["disturbance_id"].eq("D0")], ["method", "source", "flow_point"])
    by_date = metricSummary(yuceJieguo, ["method", "source", "date"])
    by_condition = metricSummary(yuceJieguo, ["method", "source", "condition_note", "disturbance_id"])
    g_summary = groupSummary(groups, ["method", "source"])
    g_by_date = groupSummary(groups, ["method", "source", "date"])

    overall.to_csv(OUT_DIR / "d0_model_overall_all_dates.csv", index=False, encoding="utf-8-sig")
    d0_overall.to_csv(OUT_DIR / "d0_model_overall_d0_only.csv", index=False, encoding="utf-8-sig")
    by_flow_d0.to_csv(OUT_DIR / "d0_model_by_flow_d0_only.csv", index=False, encoding="utf-8-sig")
    by_date.to_csv(OUT_DIR / "d0_model_by_date_holdout.csv", index=False, encoding="utf-8-sig")
    by_condition.to_csv(OUT_DIR / "d0_model_by_condition.csv", index=False, encoding="utf-8-sig")
    g_summary.to_csv(OUT_DIR / "d0_model_group_summary.csv", index=False, encoding="utf-8-sig")
    g_by_date.to_csv(OUT_DIR / "d0_model_group_summary_by_date.csv", index=False, encoding="utf-8-sig")
    return {
        "weights": weights,
        "correction": correction,
        "overall": overall,
        "d0_overall": d0_overall,
        "by_flow_d0": by_flow_d0,
        "by_date": by_date,
        "by_condition": by_condition,
        "group_summary": g_summary,
        "group_by_date": g_by_date,
    }


def fontName() -> str:
    if pdfmetrics is None:
        return "Helvetica"
    font_path = Path(r"C:\Windows\Fonts\msyh.ttc")
    if font_path.exists():
        try:
            pdfmetrics.registerFont(TTFont("MSYH", str(font_path), subfontIndex=0))
            return "MSYH"
        except Exception:
            return "Helvetica"
    return "Helvetica"


def plotPdf(tables: dict[str, pd.DataFrame]) -> None:
    if canvas is None:
        return
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    font = fontName()

    flow = tables["by_flow_d0"].copy()
    selected = ["d0_optimal_weight_profile_curve", "base_phys6", "base_lagrange", "base_owics", "base_equal_weight"]
    flow = flow[flow["method"].isin(selected)]
    c = canvas.Canvas(str(FIG_DIR / "fig_d0_model_flow_improvement.pdf"), pagesize=landscape(A4))
    w, h = landscape(A4)
    c.setFont(font, 12)
    c.drawString(45, h - 42, "D0 无扰流样本：不同流量点 MAE 对比")
    flow_points = sorted(flow["flow_point"].unique())
    methods = selected
    palette = [colors.HexColor(x) for x in ["#2563EB", "#16A34A", "#F97316", "#DC2626", "#64748B"]]
    max_val = float(flow["mae_pct"].max())
    chart_x, chart_y, chart_w, chart_h = 68, 95, 680, 390
    c.setStrokeColor(colors.HexColor("#CBD5E1"))
    c.line(chart_x, chart_y, chart_x + chart_w, chart_y)
    c.line(chart_x, chart_y, chart_x, chart_y + chart_h)
    for mi, method in enumerate(methods):
        sub = flow[flow["method"].eq(method)].set_index("flow_point")
        pts = []
        for i, fp in enumerate(flow_points):
            if fp not in sub.index:
                continue
            x = chart_x + (i + 0.5) * chart_w / len(flow_points)
            val = float(sub.loc[fp, "mae_pct"])
            y = chart_y + val / max_val * chart_h if max_val > 0 else chart_y
            pts.append((x, y))
            c.setFillColor(palette[mi])
            c.circle(x, y, 3, stroke=0, fill=1)
```

```
        c.setStrokeColor(palette[mi])
        for (x1, y1), (x2, y2) in zip(pts, pts[1:]):
            c.line(x1, y1, x2, y2)
        c.setFillColor(palette[mi])
        c.rect(540, h - 72 - mi * 17, 9, 9, stroke=0, fill=1)
        c.setFillColor(colors.HexColor("#111827"))
        c.setFont(font, 8)
        c.drawString(555, h - 74 - mi * 17, method)
    c.setFont(font, 9)
    for i, fp in enumerate(flow_points):
        x = chart_x + (i + 0.5) * chart_w / len(flow_points)
        c.drawCentredString(x, chart_y - 18, str(fp))
    c.drawString(chart_x + 300, chart_y - 42, "flow point")
    c.drawString(chart_x - 45, chart_y + chart_h + 5, "MAE / %")
    c.save()

    date = tables["by_date"]
    date = date[date["method"].eq("d0_optimal_weight_profile_curve")].sort_values("date")
    c = canvas.Canvas(str(FIG_DIR / "fig_d0_model_date_holdout.pdf"), pagesize=landscape(A4))
    w, h = landscape(A4)
    c.setFont(font, 12)
    c.drawString(45, h - 42, "D0 标定模型：按日期外部留出 MAE")
    max_val = float(date["mae_pct"].max())
    x0, y0, bar_w, row_h = 160, h - 83, 500, 31
    for idx, row in enumerate(date.itertuples(index=False)):
        y = y0 - idx * row_h
        width = bar_w * float(row.mae_pct) / max_val if max_val > 0 else 0
        c.setFont(font, 9)
        c.setFillColor(colors.HexColor("#111827"))
        c.drawRightString(x0 - 8, y + 4, str(row.date))
        c.setFillColor(colors.HexColor("#3B82F6") if str(row.date) == "20260607" else colors.HexColor("#94A3B8"))
        c.rect(x0, y, width, 12, stroke=0, fill=1)
        c.setFillColor(colors.HexColor("#111827"))
        c.drawString(x0 + width + 6, y + 1, f"{float(row.mae_pct):.3f}%")
    c.save()


def markdownTable(df: pd.DataFrame, floatfmt: str = ".6g") -> str:
    if df.empty:
        return "_无数据_"
    headers = [str(c) for c in df.columns]
    lines = ["| " + " | ".join(headers) + " |", "| " + " | ".join(["---"] * len(headers)) + " |"]
    for _, row in df.iterrows():
        vals = []
        for val in row:
            if isinstance(val, (float, np.floating)):
                vals.append(format(float(val), floatfmt))
            else:
                vals.append(str(val))
        lines.append("| " + " | ".join(vals) + " |")
    return "\n".join(lines)


def writeReport(
    data: pd.DataFrame,
    d0: pd.DataFrame,
    cv: pd.DataFrame,
    model: FittedModel,
    tables: dict[str, pd.DataFrame],
    groups: pd.DataFrame,
) -> None:
    best_basic = tables["by_flow_d0"][tables["by_flow_d0"]["method"].isin(["base_phys6", "base_owics", "base_lagrange", "base_equal_weight"])]
    final_flow = tables["by_flow_d0"][tables["by_flow_d0"]["method"].eq("d0_optimal_weight_profile_curve")][["flow_point", "mae_pct"]].rename(columns={"mae_pct": "model_mae_pct"})
    basic_min = best_basic.groupby("flow_point")["mae_pct"].min().reset_index().rename(columns={"mae_pct": "best_basic_mae_pct"})
    improve = final_flow.merge(basic_min, on="flow_point")
    improve["improved_vs_best_basic"] = improve["model_mae_pct"] < improve["best_basic_mae_pct"]
    d0_dates = sorted(d0["date"].astype(str).unique())
    date_note = (
        "无扰流 D0 仅覆盖 20260607 一天，无法在无扰流样本内部进行按日期留出；本文改用按流量点留出进行 D0 内部防过拟合检验，"
        "并将 20260608--20260615 各扰流日期作为外部日期留出集，检验 D0 标定模型跨日期、跨工况外推稳定性。"
    )
    d0_group_detail = groups[
        groups["method"].eq("d0_optimal_weight_profile_curve") & groups["disturbance_id"].eq("D0")
    ][["flow_point", "window_count", "group_mean_error_pct", "group_sd_pct", "group_mae_pct"]].sort_values("flow_point")
    text = f"""# 无扰流 D0 多声道加权积分模型

## 1. 数据口径

- 数据日期范围：`{data['date'].min()}` 至 `{data['date'].max()}`。
- 无扰流训练数据：`condition_note = no_disturbance_reference` 且 `disturbance_id = D0`。
- 无扰流窗口数：`{len(d0)}`；无扰流日期：`{', '.join(d0_dates)}`。
- 重要限制：{date_note}

## 2. 模型输入变量及物理意义

模型只使用运行时可获得的物理量，不使用日期、窗口序号、标准体积或人工扰流编号作为预测输入。

| 输入 | 来源 | 物理意义 |
| --- | --- | --- |
| `diff_ns_0--diff_ns_9` | 附件 6 | 10 路 A/B 方向传播时间差序列，反映各声路上的轴向速度投影 |
| `raw_diff_gain_a/b`、`raw_diff_intercept` | 附件 7 | 将 A/B 原始时间差映射为五条窗口等效声道速度 |
| `effective_pipe_area_m2` | 附件 7 | 管道等效截面积，用于 `V=A T v` |
| `chord0--chord4` 重构值 | 附件 6+7 | 五个弦高处的线平均速度采样 |
```

```
| `prof_center_edges` | 派生 | 中心弦相对边缘弦的偏差，刻画速度剖面鼓起程度 |
| `prof_edge_inner` | 派生 | 边缘弦与内侧弦差异，刻画近壁区速度梯度 |
| `prof_top_bottom`、`prof_inner_skew` | 派生 | 上下不对称与内侧偏斜，刻画非轴对称剖面 |
| `q,q2` | 流量点派生 | 流量点修正曲线，用于描述不同流量水平下的系统偏差 |

## 3. 参数估计与过拟合控制

模型形式为：

```text
V_hat = A T · sum_i w_i u_i · C(q, profile)
```

其中 `w_i` 为五声道积分权重，`C(q,profile)=1+beta^T x` 为流量点与剖面修正系数。权重约束为：

```text
w0=w4, w1=w3, sum_i w_i=1, w_i>=0
```

这对应圆管几何和上下对称布置，防止 10 个 D0 小样本把权重拟合成非物理形态。权重以 Phys6、OWICS/Gauss-Jacobi、Lagrange、等权作为候选先验，并用岭惩罚向先验收缩。修正系数采用岭回归，并将最终校正因子限制在 `[0.97, 1.03]`，对应 Physically Constrained GAN 文献中“物理约束校准/修正系数”的思想：允许校准，但不允许脱离物理积分基准任意漂移。

模型选择采用 D0 内部“按流量点留出”交叉验证，避免流量点修正曲线只记住训练流量点。全局交叉验证最优的最简模型是 calibration-only；但为满足本文采用“剖面特征修正 + 流量点修正曲线”的结构要求，最终从 `profile_flow` 候选族中选择交叉验证误差最小的模型。该处理牺牲极少 CV 精度，换取与 geometric flow profile correction factor 思路一致的可解释结构。

最优候选：

```text
{model.name}
```

拟合权重：

{markdownTable(tables["weights"][["chord", "position", "normalized_height", "fitted_weight", "weight_lagrange", "weight_phys6"]])}

修正系数：

{markdownTable(tables["correction"])}

全部候选交叉验证前 8 名：

{markdownTable(cv.head(8))}

4. 相对误差、组均值误差和组内重复性 SD

D0 无扰流窗口总体误差

{markdownTable(tables["d0_overall"].sort_values("mae_pct"))}

D0 无扰流分流量点组均值误差与组内 SD

{markdownTable(d0_group_detail)}

全部日期外部检验总体误差

{markdownTable(tables["overall"].sort_values("mae_pct").head(12))}

组均值误差和组内重复性

分组口径为 `date + flow_point + condition_note + disturbance_id`。`group_sd_pct` 只对同组窗口数不少于 2 的组计算。

{markdownTable(tables["group_summary"].sort_values("mean_abs_group_mean_pct").head(10))}

按日期外部留出表现

{markdownTable(tables["by_date"][tables["by_date"]["method"].eq("d0_optimal_weight_profile_curve")].sort_values("date"))}

5. 与基础估计方法的流量点对比

以下表格仅针对 D0 无扰流窗口。`improved_vs_best_basic=True` 表示模型在该流量点 MAE 低于 Phys6、OWICS、Lagrange、等权中的最优者。

{markdownTable(improve)}

完整按流量点统计：

{markdownTable(tables["by_flow_d0"].sort_values(["flow_point", "method"]))}

6. 结论

1. D0 无扰流数据只有 10 个窗口，且只来自 20260607，因此模型必须采用强物理约束：对称权重、权重和为 1、非负、岭收缩和小幅修正系数。
2. 基于 D0 的最终模型本质是“OWICS/Gauss-Jacobi 物理积分先验 + 几何剖面修正 + 流量点修正曲线 + 小幅校准系数”，符合 geometric flow profile correction factor 的建模思想。
3. 若只看 D0 内部，模型相对基础估计方法有校准收益；但外推到 D1--D8 扰流日期时，误差会显著上升，说明无扰流标定不能替代扰流补偿模型。
4. 由于无扰流没有跨日期重复数据，论文中应明确说明：按日期留出只能作为外部扰流日期稳定性检验，不能解释为无扰流同分布验证。

7. 输出文件

- `code/no_disturbance_multichannel_model.py`：可复现实验脚本。
- `code/outputs/d0_fitted_weights.csv`：拟合权重。
- `code/outputs/d0_correction_coefficients.csv`：校准与剖面修正系数。
- `code/outputs/d0_model_cv_by_flow.csv`：按流量点留出交叉验证。
- `code/outputs/d0_model_predictions_long.csv`：模型与基础方法预测长表。
- `code/outputs/d0_model_group_metrics_d0_only.csv`：D0 无扰流组均值误差和组内 SD。
- `code/outputs/d0_model_by_flow_d0_only.csv`：D0 按流量点对比。
```

```
- `code/outputs/d0_model_by_date_holdout.csv`: 按日期外部留出结果。
- `figures/fig_d0_model_flow_improvement.pdf`: D0 流量点 MAE 对比图。
- `figures/fig_d0_model_date_holdout.pdf`: 按日期外部留出 MAE 图。
"""
    REPORT.write_text(text, encoding="utf-8")


def main() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    REPORT.parent.mkdir(parents=True, exist_ok=True)
    win, raw, geo = loadInputs()
    rec = reconstructChords(raw, geo)
    data = attachFeatures(win, rec, geo)
    d0 = data[data["condition_note"].eq("no_disturbance_reference") & data["disturbance_id"].eq("D0")].copy()
    area = getArea(geo)
    priors = getPriors(geo)
    cv = leaveFlowCv(d0, area, priors)
    profile_cv = cv[cv["correction"].eq("profile_flow")].copy()
    best = profile_cv.iloc[0] if not profile_cv.empty else cv.iloc[0]
    model = fitModel(
        d0,
        area,
        str(best["prior"]),
        priors[str(best["prior"])],
        float(best["lambda_w"]),
        str(best["correction"]),
        float(best["lambda_c"]),
    )
    yuceJieguo = finalPredictions(data, model)
    groups = groupMetrics(yuceJieguo)
    tables = writeCsvOutputs(data, cv, model, yuceJieguo, groups, chordRows(geo))
    plotPdf(tables)
    writeReport(data, d0, cv, model, tables, groups)
    print("Report:", REPORT)
    print("Model:", model.name)
    print("D0 windows:", len(d0), "D0 dates:", ",".join(sorted(d0["date"].astype(str).unique())))
    print("Weights:", ", ".join(f"{w:.6f}" for w in model.weights))
    print(tables["d0_overall"].sort_values("mae_pct").to_string(index=False))
    print("Date holdout:")
    print(tables["by_date"][tables["by_date"]["method"].eq("d0_optimal_weight_profile_curve")].sort_values("date").to_string(index=False))


if __name__ == "__main__":
    main()
```

B.3 问题三：problem3_disturbance_features.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题三：计算扰流特征重要性、PCA 与聚类结果。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

import math
from collections import Counter, defaultdict
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

from model_evolution_ridge_ann import OUT_DIR, loadData
from physical_feature_library import enhancedFeatureFrame
from compact_rbf_krr import columnsForFeatureSet


ROOT = Path(__file__).resolve().parents[2]
FIG_DIR = Path(__file__).resolve().parent / "verification_figures"
REPORT = Path(__file__).resolve().parent / "verification_reports" / "DISTURBANCE_MECHANISM_ANALYSIS.md"
FAILURE_MODE_FILE = ROOT / "code" / "outputs" / "krr_leave_one_group_failure_modes.csv"

RANDOM_SEED = 20260711
CLUSTER_K_RANGE = range(2, 6)


def registerPlotFont() -> str:
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.cidfonts import UnicodeCIDFont
    from reportlab.pdfbase.ttfonts import TTFont

    for font_path in [Path(r"C:\Windows\Fonts\msyh.ttc"), Path(r"C:\Windows\Fonts\simsun.ttc")]:
        if not font_path.exists():
            continue
        try:
            pdfmetrics.registerFont(TTFont("MechanismCN", str(font_path), subfontIndex=0))
            return "MechanismCN"
        except Exception:
            pass
    pdfmetrics.registerFont(UnicodeCIDFont("STSong-Light"))
```

```
    return "STSong-Light"


def ensureDirs() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    REPORT.parent.mkdir(parents=True, exist_ok=True)


@dataclass
class ScaleParams:
    columns: list[str]
    mean: np.ndarray
    scale: np.ndarray

    @classmethod
    def fit(cls, frame: pd.DataFrame, columns: list[str]) -> "ScaleParams":
        arr = frame[columns].replace([np.inf, -np.inf], np.nan).fillna(0.0).to_numpy(dtype=float)
        mean = arr.mean(axis=0)
        scale = arr.std(axis=0)
        scale[scale < 1e-12] = 1.0
        return cls(columns, mean, scale)

    def transform(self, frame: pd.DataFrame) -> np.ndarray:
        arr = frame[self.columns].replace([np.inf, -np.inf], np.nan).fillna(0.0).to_numpy(dtype=float)
        return (arr - self.mean) / self.scale


def compactFeatureFrame(data: pd.DataFrame) -> tuple[pd.DataFrame, list[str]]:
    features = enhancedFeatureFrame(data)
    cols = columnsForFeatureSet(features, "compact_physical")
    return features, cols


def rankAverage(values: np.ndarray) -> np.ndarray:
    order = np.argsort(values, kind="mergesort")
    ranks = np.empty(len(values), dtype=float)
    i = 0
    while i < len(values):
        j = i + 1
        while j < len(values) and values[order[j]] == values[order[i]]:
            j += 1
        avg = 0.5 * (i + 1 + j)
        ranks[order[i:j]] = avg
        i = j
    return ranks


def aucBinary(feature: np.ndarray, y: np.ndarray) -> float:
    y = y.astype(int)
    n_pos = int(y.sum())
    n_neg = int(len(y) - n_pos)
    if n_pos == 0 or n_neg == 0:
        return float("nan")
    ranks = rankAverage(feature)
    rank_pos = float(ranks[y == 1].sum())
    auc = (rank_pos - n_pos * (n_pos + 1) / 2.0) / (n_pos * n_neg)
    return float(auc)


def logLoss(y: np.ndarray, p: np.ndarray) -> float:
    p = np.clip(p, 1e-8, 1.0 - 1e-8)
    return float(-(y * np.log(p) + (1.0 - y) * np.log(1.0 - p)).mean())


def fitLogisticL2(x: np.ndarray, y: np.ndarray, l2: float = 0.05, epochs: int = 8000, lr: float = 0.03) -> tuple[np.ndarray, float, np.ndarray]:
    rng = np.random.default_rng(RANDOM_SEED)
    beta = rng.normal(0.0, 0.02, size=x.shape[1])
    intercept = float(math.log((y.mean() + 1e-6) / (1.0 - y.mean() + 1e-6)))
    for _ in range(epochs):
        z = x @ beta + intercept
        p = 1.0 / (1.0 + np.exp(-np.clip(z, -40, 40)))
        err = p - y
        grad_beta = x.T @ err / len(y) + 2.0 * l2 * beta
        grad_intercept = float(err.mean())
        beta -= lr * grad_beta
        intercept -= lr * grad_intercept
    p = 1.0 / (1.0 + np.exp(-np.clip(x @ beta + intercept, -40, 40)))
    return beta, intercept, p


def permutationImportanceLogistic(x: np.ndarray, y: np.ndarray, beta: np.ndarray, intercept: float, repeats: int = 30) -> np.ndarray:
    rng = np.random.default_rng(RANDOM_SEED + 19)
    base_p = 1.0 / (1.0 + np.exp(-np.clip(x @ beta + intercept, -40, 40)))
    base_loss = logLoss(y, base_p)
    importances = np.zeros(x.shape[1], dtype=float)
    for j in range(x.shape[1]):
        losses = []
        for _ in range(repeats):
            xp = x.copy()
            xp[:, j] = rng.permutation(xp[:, j])
            p = 1.0 / (1.0 + np.exp(-np.clip(xp @ beta + intercept, -40, 40)))
            losses.append(logLoss(y, p) - base_loss)
```

```
        importances[j] = float(np.mean(losses))
    return importances


def featureImportance(data: pd.DataFrame, features: pd.DataFrame, cols: list[str]) -> pd.DataFrame:
    y = (~data["disturbance_id"].eq("D0")).astype(int).to_numpy()
    scaler = ScaleParams.fit(features, cols)
    x = scaler.transform(features)
    beta, intercept, _ = fitLogisticL2(x, y)
    perm = permutationImportanceLogistic(x, y, beta, intercept)

    rows = []
    for j, col in enumerate(cols):
        values = features[col].replace([np.inf, -np.inf], np.nan).fillna(0.0).to_numpy(dtype=float)
        d0 = values[y == 0]
        disturbed = values[y == 1]
        n0, n1 = len(d0), len(disturbed)
        mean0 = float(d0.mean())
        mean1 = float(disturbed.mean())
        sd0 = float(d0.std(ddof=1)) if n0 > 1 else 0.0
        sd1 = float(disturbed.std(ddof=1)) if n1 > 1 else 0.0
        pooled = math.sqrt(((n0 - 1) * sd0**2 + (n1 - 1) * sd1**2) / max(n0 + n1 - 2, 1))
        effect = abs(mean1 - mean0) / max(pooled, 1e-12)
        auc = aucBinary(values, y)
        separation_auc = max(auc, 1.0 - auc) if pd.notna(auc) else float("nan")
        rows.append(
            {
                "feature": col,
                "mean_D0": mean0,
                "mean_disturbed": mean1,
                "effect_size_abs": effect,
                "auc_disturbed_gt_D0": auc,
                "separation_auc": separation_auc,
                "logistic_coef": float(beta[j]),
                "logistic_abs_coef": abs(float(beta[j])),
                "permutation_logloss_increase": float(perm[j]),
            }
        )
    out = pd.DataFrame(rows)
    for metric in ["effect_size_abs", "separation_auc", "logistic_abs_coef", "permutation_logloss_increase"]:
        out[f"rank_{metric}"] = out[metric].rank(ascending=False, method="average")
    out["composite_rank"] = out[
        [
            "rank_effect_size_abs",
            "rank_separation_auc",
            "rank_logistic_abs_coef",
            "rank_permutation_logloss_increase",
        ]
    ].mean(axis=1)
    return out.sort_values(["composite_rank", "effect_size_abs"], ascending=[True, False]).reset_index(drop=True)


def pcaSvd(x: np.ndarray, n_components: int = 5) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    u, s, vt = np.linalg.svd(x, full_matrices=False)
    coords = u[:, :n_components] * s[:n_components]
    eig = (s**2) / max(len(x) - 1, 1)
    explained = eig / max(eig.sum(), 1e-12)
    return coords, explained[:n_components], vt[:n_components]


def pairwiseDistances(x: np.ndarray) -> np.ndarray:
    x2 = np.sum(x * x, axis=1)[:, None]
    d2 = np.maximum(x2 + x2.T - 2.0 * (x @ x.T), 0.0)
    return np.sqrt(d2)


def kmeansLabels(x: np.ndarray, k: int, n_init: int = 20, max_iter: int = 200) -> tuple[np.ndarray, float]:
    rng = np.random.default_rng(RANDOM_SEED + k)
    best_labels = None
    best_inertia = float("inf")
    n = len(x)
    for _ in range(n_init):
        centroids = x[rng.choice(n, size=k, replace=False)].copy()
        labels = np.zeros(n, dtype=int)
        for _ in range(max_iter):
            dist = np.sum((x[:, None, :] - centroids[None, :, :]) ** 2, axis=2)
            new_labels = dist.argmin(axis=1)
            if np.array_equal(new_labels, labels):
                break
            labels = new_labels
            for c in range(k):
                if np.any(labels == c):
                    centroids[c] = x[labels == c].mean(axis=0)
                else:
                    centroids[c] = x[rng.integers(0, n)]
        inertia = float(np.sum((x - centroids[labels]) ** 2))
        if inertia < best_inertia:
            best_inertia = inertia
            best_labels = labels.copy()
    return best_labels if best_labels is not None else np.zeros(n, dtype=int), best_inertia


def wardLabels(x: np.ndarray, k: int) -> np.ndarray:
```

```
    clusters = {i: [i] for i in range(len(x))}
    means = {i: x[i].copy() for i in range(len(x))}
    sizes = {i: 1 for i in range(len(x))}
    next_id = len(x)
    while len(clusters) > k:
        keys = list(clusters)
        best_pair = None
        best_delta = float("inf")
        for a_pos in range(len(keys)):
            a = keys[a_pos]
            for b in keys[a_pos + 1 :]:
                delta = sizes[a] * sizes[b] / (sizes[a] + sizes[b]) * float(np.sum((means[a] - means[b]) ** 2))
                if delta < best_delta:
                    best_delta = delta
                    best_pair = (a, b)
        if best_pair is None:
            break
        a, b = best_pair
        merged = clusters[a] + clusters[b]
        clusters[next_id] = merged
        sizes[next_id] = sizes[a] + sizes[b]
        means[next_id] = (sizes[a] * means[a] + sizes[b] * means[b]) / sizes[next_id]
        for key in [a, b]:
            del clusters[key]
            del means[key]
            del sizes[key]
        next_id += 1
    labels = np.zeros(len(x), dtype=int)
    for c, members in enumerate(clusters.values()):
        labels[members] = c
    return labels


def gmmDiagLabels(x: np.ndarray, k: int, n_init: int = 10, max_iter: int = 150) -> tuple[np.ndarray, float]:
    rng = np.random.default_rng(RANDOM_SEED + 100 + k)
    n, p = x.shape
    best_labels = None
    best_ll = -float("inf")
    for _ in range(n_init):
        labels, _ = kmeansLabels(x, k, n_init=1, max_iter=50)
        means = np.vstack([x[labels == c].mean(axis=0) if np.any(labels == c) else x[rng.integers(0, n)] for c in range(k)])
        vars_ = np.vstack(
            [x[labels == c].var(axis=0) + 1e-4 if np.any(labels == c) else np.ones(p) for c in range(k)]
        )
        weights = np.array([max(np.mean(labels == c), 1.0 / n) for c in range(k)])
        weights = weights / weights.sum()
        prev_ll = -float("inf")
        resp = np.zeros((n, k))
        for _ in range(max_iter):
            log_prob = np.zeros((n, k))
            for c in range(k):
                log_det = np.sum(np.log(vars_[c]))
                quad = np.sum((x - means[c]) ** 2 / vars_[c], axis=1)
                log_prob[:, c] = math.log(weights[c] + 1e-12) - 0.5 * (p * math.log(2.0 * math.pi) + log_det + quad)
            max_log = log_prob.max(axis=1, keepdims=True)
            prob = np.exp(log_prob - max_log)
            denom = np.maximum(prob.sum(axis=1, keepdims=True), 1e-12)
            resp = prob / denom
            ll = float(np.sum(max_log[:, 0] + np.log(denom[:, 0])))
            nk = np.maximum(resp.sum(axis=0), 1e-8)
            weights = nk / n
            means = (resp.T @ x) / nk[:, None]
            for c in range(k):
                diff = x - means[c]
                vars_[c] = (resp[:, c][:, None] * diff * diff).sum(axis=0) / nk[c] + 1e-4
            if abs(ll - prev_ll) < 1e-5:
                break
            prev_ll = ll
        labels = resp.argmax(axis=1)
        if prev_ll > best_ll and len(np.unique(labels)) == k:
            best_ll = prev_ll
            best_labels = labels.copy()
    return best_labels if best_labels is not None else np.zeros(n, dtype=int), best_ll


def clusterMetrics(x: np.ndarray, labels: np.ndarray) -> dict:
    unique = np.unique(labels)
    k = len(unique)
    n = len(x)
    if k <= 1 or k >= n:
        return {"silhouette": float("nan"), "calinski_harabasz": float("nan"), "davies_bouldin": float("nan")}

    dist = pairwiseDistances(x)
    sil_values = []
    for i in range(n):
        same = labels == labels[i]
        if same.sum() <= 1:
            a = 0.0
        else:
            a = float(dist[i, same].sum() / (same.sum() - 1))
        b = float("inf")
        for c in unique:
            if c == labels[i]:
```

```
                continue
            other = labels == c
            b = min(b, float(dist[i, other].mean()))
        sil_values.append((b - a) / max(a, b, 1e-12))
    silhouette = float(np.mean(sil_values))

    overall = x.mean(axis=0)
    ss_between = 0.0
    ss_within = 0.0
    centroids = []
    scatters = []
    for c in unique:
        xc = x[labels == c]
        centroid = xc.mean(axis=0)
        centroids.append(centroid)
        ss_between += len(xc) * float(np.sum((centroid - overall) ** 2))
        within = float(np.sum((xc - centroid) ** 2))
        ss_within += within
        scatters.append(float(np.sqrt(within / max(len(xc), 1))))
    ch = (ss_between / (k - 1)) / max(ss_within / (n - k), 1e-12)

    db_values = []
    for i in range(k):
        worst = -float("inf")
        for j in range(k):
            if i == j:
                continue
            d = float(np.linalg.norm(centroids[i] - centroids[j]))
            worst = max(worst, (scatters[i] + scatters[j]) / max(d, 1e-12))
        db_values.append(worst)
    db = float(np.mean(db_values))
    return {"silhouette": silhouette, "calinski_harabasz": float(ch), "davies_bouldin": db}


def relabelBySize(labels: np.ndarray) -> np.ndarray:
    counts = Counter(labels.tolist())
    order = [old for old, _ in sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))]
    mapping = {old: new for new, old in enumerate(order)}
    return np.array([mapping[x] for x in labels], dtype=int)


def runClustering(x: np.ndarray) -> tuple[pd.DataFrame, dict[str, np.ndarray]]:
    rows = []
    labels_by_model: dict[str, np.ndarray] = {}
    for k in CLUSTER_K_RANGE:
        for method in ["kmeans", "ward", "gmm_diag"]:
            if method == "kmeans":
                labels, inertia = kmeansLabels(x, k)
                extra = {"inertia": inertia, "log_likelihood": float("nan")}
            elif method == "ward":
                labels = wardLabels(x, k)
                extra = {"inertia": float(np.sum((x - np.vstack([x[labels == c].mean(axis=0) for c in np.unique(labels)])[labels]) ** 2)), "log_likelihood": float("nan")}
            else:
                labels, ll = gmmDiagLabels(x, k)
                extra = {"inertia": float("nan"), "log_likelihood": ll}
            labels = relabelBySize(labels)
            metrics = clusterMetrics(x, labels)
            model_id = f"{method}_k{k}"
            labels_by_model[model_id] = labels
            rows.append({"cluster_model": model_id, "method": method, "k": k, **metrics, **extra})
    quality = pd.DataFrame(rows)
    quality["rank_silhouette"] = quality["silhouette"].rank(ascending=False, method="average")
    quality["rank_db"] = quality["davies_bouldin"].rank(ascending=True, method="average")
    quality["rank_ch"] = quality["calinski_harabasz"].rank(ascending=False, method="average")
    quality["overall_rank"] = quality[["rank_silhouette", "rank_db", "rank_ch"]].mean(axis=1)
    quality = quality.sort_values(["overall_rank", "rank_silhouette"]).reset_index(drop=True)
    return quality, labels_by_model


def majority(values: pd.Series) -> str:
    counts = Counter(str(v) for v in values.dropna().tolist())
    if not counts:
        return ""
    return counts.most_common(1)[0][0]


def clusterCrossTables(
    disturbed: pd.DataFrame,
    labels: np.ndarray,
    failure_modes: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    windows = disturbed[["window_id", "date", "flow_point", "disturbance_id"]].copy()
    windows["cluster"] = labels

    cross_dist = pd.crosstab(windows["cluster"], windows["disturbance_id"]).reset_index()
    cross_dist.columns = [str(c) for c in cross_dist.columns]

    group_cluster = (
        windows.groupby(["date", "flow_point", "disturbance_id"], sort=True)["cluster"]
        .agg(lambda s: int(Counter(s.tolist()).most_common(1)[0][0]))
        .reset_index()
    )
    merged = group_cluster.merge(
```

```
        failure_modes,
        on=["date", "flow_point", "disturbance_id"],
        how="left",
        validate="one_to_one",
    )
    merged["mechanism_note"] = merged["mechanism_note"].fillna("unlabeled_small_group")
    merged["threshold_failure_type"] = merged["threshold_failure_type"].fillna("unlabeled_small_group")
    cross_mech = pd.crosstab(merged["cluster"], merged["mechanism_note"]).reset_index()
    cross_mech.columns = [str(c) for c in cross_mech.columns]
    cross_fail = pd.crosstab(merged["cluster"], merged["threshold_failure_type"]).reset_index()
    cross_fail.columns = [str(c) for c in cross_fail.columns]
    summary = (
        merged.groupby("cluster", sort=True)
        .agg(
            group_count=("disturbance_id", "size"),
            mean_abs_error_pct=("abs_mean_error_pct", "mean"),
            mean_sd_pct=("repeatability_sd_pct", "mean"),
            mean_ood=("ood_nearest_euclidean_mean", "mean"),
            dominant_mechanism=("mechanism_note", majority),
        )
        .reset_index()
    )
    return cross_dist, cross_mech, cross_fail, summary


def drawFeatureImportancePdf(df: pd.DataFrame) -> None:
    try:
        from reportlab.lib import colors
        from reportlab.lib.pagesizes import landscape, A4
        from reportlab.pdfgen import canvas
    except ModuleNotFoundError:
        return
    path = FIG_DIR / "fig_disturbance_feature_importance.pdf"
    c = canvas.Canvas(str(path), pagesize=landscape(A4))
    page_w, page_h = landscape(A4)
    font = registerPlotFont()
    label_map = {
        "profile_swirl": "旋流代理",
        "profile_ab_abs": "A/B 方向不对称",
        "ab_abs_mean": "A/B 绝对差均值",
        "lagrange_volume_m3_minus_phys6_frac": "Lagrange 与 Phys6 相对差",
        "chord_edge_minus_inner_frac": "边缘与内侧声道相对差",
        "profile_edge_inner": "边缘与内侧剖面差",
        "owics_volume_m3_minus_phys6_frac": "OWICS 与 Phys6 相对差",
        "zero_rate_med": "零点速率中位数",
    }
    top = df.head(8).iloc[::-1]
    x0, y0 = 225, 62
    w, h = page_w - 305, page_h - 125
    vals = top["effect_size_abs"].to_numpy(dtype=float)
    vmax = max(1e-6, float(vals.max()) * 1.15)
    bar_h = h / len(top) * 0.72
    c.setFont(font, 16)
    for i, (_, row) in enumerate(top.iterrows()):
        y = y0 + i * h / len(top)
        bw = w * float(row["effect_size_abs"]) / vmax
        c.setFillColor(colors.HexColor("#1F5AA6"))
        c.rect(x0, y, bw, bar_h, fill=1, stroke=0)
        c.setFillColor(colors.black)
        c.drawRightString(x0 - 8, y + 3, label_map.get(str(row["feature"]), str(row["feature"])))
        c.drawString(x0 + bw + 6, y + 3, f"{row['effect_size_abs']:.2f}")
    c.setFont(font, 17)
    c.drawCentredString(x0 + w / 2, 28, "标准化效应量绝对值")
    c.save()


def drawPcaPdf(coords: pd.DataFrame, explained: pd.DataFrame, cluster_col: str = "best_cluster") -> None:
    try:
        from reportlab.lib import colors
        from reportlab.lib.pagesizes import landscape, A4
        from reportlab.pdfgen import canvas
    except ModuleNotFoundError:
        return
    path = FIG_DIR / "fig_disturbance_pca_clusters.pdf"
    c = canvas.Canvas(str(path), pagesize=landscape(A4))
    page_w, page_h = landscape(A4)
    font = registerPlotFont()
    x0, y0, w, h = 70, 60, page_w - 150, page_h - 120
    x = coords["PC1"].to_numpy(dtype=float)
    y = coords["PC2"].to_numpy(dtype=float)
    xmin, xmax = float(x.min()), float(x.max())
    ymin, ymax = float(y.min()), float(y.max())
    pad_x = max((xmax - xmin) * 0.08, 1e-6)
    pad_y = max((ymax - ymin) * 0.08, 1e-6)
    xmin, xmax = xmin - pad_x, xmax + pad_x
    ymin, ymax = ymin - pad_y, ymax + pad_y
    palette = [
        colors.HexColor("#4C78A8"),
        colors.HexColor("#F58518"),
        colors.HexColor("#54A24B"),
        colors.HexColor("#B279A2"),
        colors.HexColor("#E45756"),
        colors.HexColor("#72B7B2"),
```

```
    ]
    c.setStrokeColor(colors.black)
    c.setLineWidth(1.3)
    c.line(x0, y0, x0 + w, y0)
    c.line(x0, y0, x0, y0 + h)
    c.setFont(font, 16)
    for tick in range(5):
        tx = xmin + (xmax - xmin) * tick / 4
        ty = ymin + (ymax - ymin) * tick / 4
        px = x0 + w * tick / 4
        py = y0 + h * tick / 4
        c.setStrokeColor(colors.HexColor("#D0D0D0"))
        c.setLineWidth(0.6)
        c.line(px, y0, px, y0 + h)
        c.line(x0, py, x0 + w, py)
        c.setFillColor(colors.black)
        c.drawCentredString(px, y0 - 17, f"{tx:.1f}")
        c.drawRightString(x0 - 8, py - 3, f"{ty:.1f}")
    for _, row in coords.iterrows():
        px = x0 + w * (float(row["PC1"]) - xmin) / (xmax - xmin)
        py = y0 + h * (float(row["PC2"]) - ymin) / (ymax - ymin)
        cluster = int(row[cluster_col])
        c.setFillColor(palette[cluster % len(palette)])
        c.circle(px, py, 4.0, fill=1, stroke=0)
    pc1 = float(explained.loc[explained["component"].eq("PC1"), "explained_variance_ratio"].iloc[0])
    pc2 = float(explained.loc[explained["component"].eq("PC2"), "explained_variance_ratio"].iloc[0])
    c.setFillColor(colors.black)
    c.setFont(font, 16)
    c.drawCentredString(x0 + w / 2, 28, f"PC1 ({pc1:.1%})")
    c.saveState()
    c.translate(26, y0 + h / 2)
    c.rotate(90)
    c.drawCentredString(0, 0, f"PC2 ({pc2:.1%})")
    c.restoreState()
    legend_x, legend_y = x0 + w - 165, y0 + h - 18
    for cluster, label in [(0, "簇 0: 其余扰流为主"), (1, "簇 1: D3 为主")]:
        c.setFillColor(palette[cluster])
        c.circle(legend_x, legend_y - cluster * 20, 4.5, fill=1, stroke=0)
        c.setFillColor(colors.black)
        c.setFont(font, 16)
        c.drawString(legend_x + 11, legend_y - 4 - cluster * 20, label)
    c.save()


def markdownTable(df: pd.DataFrame, max_rows: int = 20) -> str:
    if df.empty:
        return "_无数据。_"
    use = df.head(max_rows).copy()
    for col in use.columns:
        if pd.api.types.is_float_dtype(use[col]):
            use[col] = use[col].map(lambda x: "" if pd.isna(x) else f"{x:.6g}")
    use = use.astype(str)
    headers = list(use.columns)
    rows = use.values.tolist()
    widths = []
    for i, header in enumerate(headers):
        widths.append(max(len(header), *(len(row[i]) for row in rows)))
    header_line = "| " + " | ".join(header.ljust(widths[i]) for i, header in enumerate(headers)) + " |"
    sep_line = "| " + " | ".join("-" * widths[i] for i in range(len(headers))) + " |"
    body = ["| " + " | ".join(row[i].ljust(widths[i]) for i in range(len(headers))) + " |" for row in rows]
    return "\n".join([header_line, sep_line] + body)


def writeReport(
    feature_rank: pd.DataFrame,
    explained: pd.DataFrame,
    quality: pd.DataFrame,
    cross_dist: pd.DataFrame,
    cross_mech: pd.DataFrame,
    cluster_summary: pd.DataFrame,
    best_model_id: str,
) -> None:
    feature_cols = [
        "feature",
        "effect_size_abs",
        "separation_auc",
        "logistic_abs_coef",
        "permutation_logloss_increase",
        "composite_rank",
    ]
    quality_cols = ["cluster_model", "method", "k", "silhouette", "calinski_harabasz", "davies_bouldin", "overall_rank"]
    best_quality = quality.loc[quality["cluster_model"].eq(best_model_id)].iloc[0]
    best_silhouette = float(best_quality["silhouette"])
    decision = (
        "当前最佳聚类为 K=2, Silhouette 约为 "
        f"`{best_silhouette:.3f}`。聚类与扰流编号交叉表显示，第二簇主要对应 D3，而 D1、D2、D4、D5、D6、D7、D8 "
        "仍集中在同一大簇中；聚类与失效机制交叉表也显示，系统漂移型和重复性失稳型没有被清晰分离。因此当前证据不足以支持把硬聚类或软门控专家升级为最终主模型。"
        "更稳妥的结论是：D1-D8 在 compact physical 特征空间中存在局部结构，但总体仍呈连续重叠分布，统一 RBF-KRR 连续相似性补偿仍是主线；聚类只作为解释性分支和后续定向增强候选。"
    )
    text = f"""# 问题3：扰流特征重要性、PCA 聚类与失效机制对齐

## 1. 分析目标

```

```
当前最终主模型已冻结为 `krr_phys6_volume_m3_compact_physical_g0.1_lam0.1_none`。本报告不训练新的最终模型，而是服务于问题3：解释 D1-D8 扰流在在线特征空间中的结构，并判断扰流聚类是否足够清晰，是否有资格作为后续软门控增强分支。

分析对象仅包括 D1-D8 扰流窗口；`20260607-60-D0` 属于正常锚点缺失型 OOD，不进入扰流聚类解释。

## 2. 扰流特征重要性

特征重要性从三类证据综合得到：D0 与扰流窗口的标准化效应量、单特征 AUC、L2 Logistic 标准化系数和 permutation logloss increase。

{markdownTable(feature_rank[feature_cols], 15)}

这些特征的理论含义与论文库一致：剖面类特征刻画速度剖面的中心-边缘和上下差异，A/B 类特征刻画声路方向不对称，动态/零点类特征刻画测量稳定性。

## 3. PCA 结果

PCA 仅在 D1-D8 的 compact physical 特征上进行。

{markdownTable(explained, 5)}

## 4. 聚类质量比较

比较 KMeans、Ward 和对角 GMM，聚类数 `K=2--5`，评价指标包括 Silhouette、Calinski-Harabasz 和 Davies-Bouldin。当前综合排序第一的聚类模型为：

```text
{best_model_id}
```

{markdownTable(quality[quality_cols], 12)}

5. 聚类与扰流编号交叉表

{markdownTable(cross_dist, 20)}

6. 聚类与失效机制交叉表

{markdownTable(cross_mech, 20)}

7. 聚类组统计

{markdownTable(cluster_summary, 20)}

8. 结论

{decision}

9. 输出文件

- `code/disturbance_mechanism_analysis.py`
- `code/outputs/disturbance_feature_importance.csv`
- `code/outputs/disturbance_pca_coordinates.csv`
- `code/outputs/disturbance_pca_explained_variance.csv`
- `code/outputs/disturbance_cluster_quality.csv`
- `code/outputs/disturbance_cluster_labels.csv`
- `code/outputs/disturbance_cluster_cross_disturbance.csv`
- `code/outputs/disturbance_cluster_cross_mechanism.csv`
- `code/outputs/disturbance_cluster_cross_failure_type.csv`
- `code/outputs/disturbance_cluster_summary.csv`
- `figures/fig_disturbance_feature_importance.pdf`
- `figures/fig_disturbance_pca_clusters.pdf`
"""
    REPORT.write_text(text, encoding="utf-8")

def main() -> None:
    ensureDirs()
    data = loadData()
    features, cols = compactFeatureFrame(data)
    feature_rank = featureImportance(data, features, cols)
    feature_rank.to_csv(OUT_DIR / "disturbance_feature_importance.csv", index=False, encoding="utf-8-sig")

    disturbed = data.loc[~data["disturbance_id"].eq("D0")].reset_index(drop=True)
    disturbed_features = enhancedFeatureFrame(disturbed)
    disturbed_cols = columnsForFeatureSet(disturbed_features, "compact_physical")
    scaler = ScaleParams.fit(disturbed_features, disturbed_cols)
    x = scaler.transform(disturbed_features)
    coords, explained_ratio, loadings = pcaSvd(x, n_components=5)

    explained = pd.DataFrame(
        {
            "component": [f"PC{i+1}" for i in range(len(explained_ratio))],
            "explained_variance_ratio": explained_ratio,
            "cumulative_ratio": np.cumsum(explained_ratio),
        }
)
    pca_coords = disturbed[["window_id", "date", "flow_point", "disturbance_id"]].copy()
    for i in range(coords.shape[1]):
        pca_coords[f"PC{i+1}"] = coords[:, i]
    loading_rows = []
    for i in range(loadings.shape[0]):
        for j, col in enumerate(disturbed_cols):
            loading_rows.append({"component": f"PC{i+1}", "feature": col, "loading": float(loadings[i, j])})
    loadings_df = pd.DataFrame(loading_rows)

```

```
    quality, labels_by_model = runClustering(x)
    best_model_id = str(quality.iloc[0]["cluster_model"])
    best_labels = labels_by_model[best_model_id]
    pca_coords["best_cluster"] = best_labels
    labels_out = disturbed[["window_id", "date", "flow_point", "disturbance_id"]].copy()
    for model_id, labels in labels_by_model.items():
        labels_out[model_id] = labels
    labels_out["best_cluster_model"] = best_model_id
    labels_out["best_cluster"] = best_labels

    if FAILURE_MODE_FILE.exists():
        failure_modes = pd.read_csv(FAILURE_MODE_FILE, encoding="utf-8-sig")
        failure_modes["date"] = failure_modes["date"].astype(str)
        disturbed["date"] = disturbed["date"].astype(str)
    else:
        failure_modes = pd.DataFrame(columns=["date", "flow_point", "disturbance_id", "mechanism_note", "threshold_failure_type"])

    cross_dist, cross_mech, cross_fail, cluster_summary = clusterCrossTables(disturbed, best_labels, failure_modes)

    explained.to_csv(OUT_DIR / "disturbance_pca_explained_variance.csv", index=False, encoding="utf-8-sig")
    pca_coords.to_csv(OUT_DIR / "disturbance_pca_coordinates.csv", index=False, encoding="utf-8-sig")
    loadings_df.to_csv(OUT_DIR / "disturbance_pca_loadings.csv", index=False, encoding="utf-8-sig")
    quality.to_csv(OUT_DIR / "disturbance_cluster_quality.csv", index=False, encoding="utf-8-sig")
    labels_out.to_csv(OUT_DIR / "disturbance_cluster_labels.csv", index=False, encoding="utf-8-sig")
    cross_dist.to_csv(OUT_DIR / "disturbance_cluster_cross_disturbance.csv", index=False, encoding="utf-8-sig")
    cross_mech.to_csv(OUT_DIR / "disturbance_cluster_cross_mechanism.csv", index=False, encoding="utf-8-sig")
    cross_fail.to_csv(OUT_DIR / "disturbance_cluster_cross_failure_type.csv", index=False, encoding="utf-8-sig")
    cluster_summary.to_csv(OUT_DIR / "disturbance_cluster_summary.csv", index=False, encoding="utf-8-sig")

    drawFeatureImportancePdf(feature_rank)
    drawPcaPdf(pca_coords, explained)
    writeReport(feature_rank, explained, quality, cross_dist, cross_mech, cluster_summary, best_model_id)

    print("Top feature importance:")
    print(feature_rank[["feature", "effect_size_abs", "separation_auc", "logistic_abs_coef", "permutation_logloss_increase"]].head(10).to_string(index=False))
    print("Cluster quality:")
    print(quality[["cluster_model", "silhouette", "calinski_harabasz", "davies_bouldin", "overall_rank"]].head(8).to_string(index=False))
    print("Best cluster model:", best_model_id)
    print("Report:", REPORT)

if __name__ == "__main__":
    main()
```

B.4 全量拟合对照：fullfit_expression_baseline.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题四对照：估计高维全量拟合的样本内表达上限。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

import json
import math
import shutil
import subprocess
import sys
import warnings
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd
from pandas.errors import PerformanceWarning
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4, landscape
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.pdfgen import canvas


ROOT = Path(__file__).resolve().parents[2]
DATA_DIR = ROOT / "A题 (1)"
OUTPUT_DIR = Path(__file__).resolve().parent / "verification_outputs"
FIG_DIR = Path(__file__).resolve().parent / "verification_figures"
REPORT_DIR = Path(__file__).resolve().parent / "verification_reports"

WINDOW_DATA = DATA_DIR / "attachment1_window_data.csv"
BASELINE_SUMMARY = DATA_DIR / "attachment4_baseline_summary.csv"
EVALUATE_SCRIPT = DATA_DIR / "evaluate_submission.py"

RANDOM_SEED = 20260709
warnings.filterwarnings("ignore", category=PerformanceWarning)


@dataclass
class FittedRidge:
    name: str
    columns: list[str]
    median: pd.Series
```

```
    mean: np.ndarray
    scale: np.ndarray
    coef: np.ndarray
    lam: float
    target_mode: str
    base_col: str


def ensureDirs() -> None:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    REPORT_DIR.mkdir(parents=True, exist_ok=True)


def relativeErrorPct(moxingTiji: pd.Series, biaozhunTiji: pd.Series) -> pd.Series:
    return (moxingTiji.astype(float) - biaozhunTiji.astype(float)) / biaozhunTiji.astype(float) * 100.0


def groupTable(df: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for (date, flow_point), group in df.groupby(["date", "flow_point"], sort=True):
        if len(group) < 3:
            continue
        values = group["model_error_pct"].astype(float)
        mean_err = float(values.mean())
        sd = float(values.std(ddof=1))
        rows.append(
            {
                "date": str(date),
                "flow_point": int(flow_point),
                "n": int(len(group)),
                "mean_error_pct": mean_err,
                "repeatability_sd_pct": sd,
                "range_pct": float(values.max() - values.min()),
                "pass_group": bool(abs(mean_err) <= 0.2 and sd <= 0.040),
            }
        )
    return pd.DataFrame(rows)


def linearityUL(df: pd.DataFrame) -> float:
    use = df[(df["condition_note"].eq("no_disturbance_reference")) & (df["flow_point"].between(40, 100))]
    by_flow = use.groupby("flow_point")["model_error_pct"].mean()
    if len(by_flow) <= 1:
        return float("nan")
    return math.sqrt(float((by_flow**2).sum()) / (len(by_flow) - 1))


def disturbanceUD(df: pd.DataFrame) -> float:
    use = df[df["flow_point"].between(40, 100)].copy()
    base = (
        use[use["condition_note"].eq("no_disturbance_reference")]
        .groupby("flow_point")["model_error_pct"]
        .agg(["mean", "count", "std"])
        .rename(columns={"mean": "baseline_mean", "count": "baseline_n", "std": "baseline_sd"})
        .reset_index()
    )
    disturb = (
        use[use["condition_note"].eq("disturbed_test")]
        .groupby(["disturbance_id", "flow_point"])["model_error_pct"]
        .agg(["mean", "count", "std"])
        .rename(columns={"mean": "disturb_mean", "count": "disturb_n", "std": "disturb_sd"})
        .reset_index()
    )
    detail = disturb.merge(base, on="flow_point", how="left")
    detail["disturb_sd"] = detail["disturb_sd"].fillna(0.0)
    detail["abs_drift"] = (detail["baseline_mean"] - detail["disturb_mean"]).abs()
    available = detail[(detail["baseline_n"] >= 1) & (detail["disturb_n"] >= 1)].copy()
    if available.empty:
        return float("nan")
    u_d_c = float(available["abs_drift"].max()) / math.sqrt(3.0)
    u_d_r = float(available["disturb_sd"].max())
    return math.sqrt(u_d_c * u_d_c + u_d_r * u_d_r)


def evaluatePredictions(data: pd.DataFrame, moxingTiji: np.ndarray) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    df = data.copy()
    df["model_volume_m3"] = moxingTiji
    df["model_error_pct"] = relativeErrorPct(df["model_volume_m3"], df["standard_volume_m3"])
    groups = groupTable(df)
    u_l = linearityUL(df)
    u_r = float(groups["repeatability_sd_pct"].max()) if not groups.empty else float("nan")
    u_d = disturbanceUD(df)
    summary = pd.DataFrame(
        [
            {
                "window_count": int(len(df)),
                "mae_pct": float(df["model_error_pct"].abs().mean()),
                "mean_error_pct": float(df["model_error_pct"].mean()),
                "max_abs_error_pct": float(df["model_error_pct"].abs().max()),
                "group_pass": f"{int(groups['pass_group'].sum())}/{len(groups)}",
                "group_pass_count": int(groups["pass_group"].sum()),
                "group_count": int(len(groups)),
```

```
                "u_nor_L_pct": u_l,
                "u_nor_r_pct": u_r,
                "u_nor_d_pct": u_d,
                "pass_u_nor_L": bool(pd.notna(u_l) and u_l < 0.036),
                "pass_u_nor_r": bool(pd.notna(u_r) and u_r < 0.040),
                "pass_u_nor_d": bool(pd.notna(u_d) and u_d < 0.115),
                "pass_all_3": bool(pd.notna(u_l) and u_l < 0.036 and pd.notna(u_r) and u_r < 0.040 and pd.notna(u_d) and u_d < 0.115),
            }
        ]
    )
    windows = df[["window_id", "date", "flow_point", "condition_note", "disturbance_id", "standard_volume_m3", "model_volume_m3", "model_error_pct"]]
    return summary, groups, windows


def loadData() -> pd.DataFrame:
    data = pd.read_csv(WINDOW_DATA, encoding="utf-8-sig")
    return data.sort_values("window_id").reset_index(drop=True)


def buildFeatures(data: pd.DataFrame, mode: str) -> pd.DataFrame:
    allowed_prefixes = ("chord", "ab", "profile_", "dyn_", "zero_")
    base_cols = [
        "duration_s",
        "flow_point",
        "phys6_volume_m3",
        "owics_volume_m3",
        "lagrange_volume_m3",
        "equal_weight_volume_m3",
    ]
    online_cols = [c for c in data.columns if c.startswith(allowed_prefixes)]
    feats = data[base_cols + online_cols].copy()

    eps = 1e-12
    q = (data["flow_point"].astype(float) - 50.0) / 30.0
    feats["q"] = q
    feats["q2"] = q**2
    feats["q3"] = q**3
    feats["log_duration"] = np.log(np.maximum(data["duration_s"].astype(float), eps))
    feats["phys6_rate_proxy"] = data["phys6_volume_m3"].astype(float) / np.maximum(data["duration_s"].astype(float), eps)

    for col in ["owics_volume_m3", "lagrange_volume_m3", "equal_weight_volume_m3"]:
        feats[f"{col}_minus_phys6_frac"] = (data[col].astype(float) - data["phys6_volume_m3"].astype(float)) / np.maximum(data["phys6_volume_m3"].astype(float).abs(), eps)

    chord_cols = [f"chord{i}" for i in range(5)]
    ab_cols = [f"ab{i}" for i in range(5)]
    if all(c in data.columns for c in chord_cols):
        chord = data[chord_cols].astype(float)
        feats["chord_mean"] = chord.mean(axis=1)
        feats["chord_std"] = chord.std(axis=1)
        feats["chord_range"] = chord.max(axis=1) - chord.min(axis=1)
        feats["chord_center_minus_edges"] = chord["chord2"] - 0.5 * (chord["chord0"] + chord["chord4"])
        feats["chord_inner_skew_calc"] = chord["chord1"] - chord["chord3"]
        for col in chord_cols:
            feats[f"{col}_rel_mean"] = (chord[col] - feats["chord_mean"]) / np.maximum(feats["chord_mean"].abs(), eps)

    if all(c in data.columns for c in ab_cols):
        ab = data[ab_cols].astype(float)
        feats["ab_mean"] = ab.mean(axis=1)
        feats["ab_abs_mean"] = ab.abs().mean(axis=1)
        feats["ab_std"] = ab.std(axis=1)
        feats["ab_range"] = ab.max(axis=1) - ab.min(axis=1)

    interaction_roots = [
        "profile_top_bottom",
        "profile_center_all",
        "profile_edge_inner",
        "profile_inner_skew",
        "profile_ab_abs",
        "profile_swirl",
        "dyn_plateau_cv",
        "dyn_active_eq_s",
        "zero_rate_med",
        "zero_rate_mad",
        "phys6_rate_proxy",
        "lagrange_volume_m3_minus_phys6_frac",
        "owics_volume_m3_minus_phys6_frac",
    ]
    for col in interaction_roots:
        if col in feats.columns:
            feats[f"{col}_x_q"] = feats[col].astype(float) * q
            feats[f"{col}_x_q2"] = feats[col].astype(float) * q**2

    if mode == "compact":
        keep = [
            "duration_s",
            "flow_point",
            "phys6_volume_m3",
            "q",
            "q2",
            "q3",
            "phys6_rate_proxy",
            "lagrange_volume_m3_minus_phys6_frac",
            "owics_volume_m3_minus_phys6_frac",
```

```
            "profile_top_bottom",
            "profile_center_all",
            "profile_edge_inner",
            "profile_inner_skew",
            "profile_ab_abs",
            "profile_swirl",
            "dyn_plateau_cv",
            "dyn_active_eq_s",
            "zero_rate_med",
            "zero_rate_mad",
        ]
        return feats[[c for c in keep if c in feats.columns]].copy()

    if mode == "rich":
        return feats.copy()

    if mode == "poly":
        rich = feats.copy()
        roots = [c for c in rich.columns if c not in {"phys6_volume_m3", "owics_volume_m3", "lagrange_volume_m3", "equal_weight_volume_m3"}]
        numeric = rich[roots].astype(float)
        for col in roots:
            rich[f"{col}_sq"] = numeric[col] ** 2
        selected = [c for c in roots if c.startswith(("profile_", "dyn_", "zero_", "chord", "ab"))][:30]
        for i, a in enumerate(selected):
            for b in selected[i + 1 : min(i + 9, len(selected))]:
                rich[f"{a}_x_{b}"] = numeric[a] * numeric[b]
        return rich

    raise ValueError(f"Unknown feature mode: {mode}")


def targetFromMode(data: pd.DataFrame, mode: str, base_col: str) -> np.ndarray:
    base = np.maximum(data[base_col].astype(float).to_numpy(), 1e-12)
    std = data["standard_volume_m3"].astype(float).to_numpy()
    if mode == "relative_correction":
        return std / base - 1.0
    if mode == "log_ratio":
        return np.log(std / base)
    if mode == "direct_volume":
        return std
    raise ValueError(f"Unknown target mode: {mode}")


def volumeFromPrediction(data: pd.DataFrame, pred: np.ndarray, mode: str, base_col: str) -> np.ndarray:
    base = data[base_col].astype(float).to_numpy()
    if mode == "relative_correction":
        return base * (1.0 + pred)
    if mode == "log_ratio":
        return base * np.exp(pred)
    if mode == "direct_volume":
        return pred
    raise ValueError(f"Unknown target mode: {mode}")


def fitRidge(features: pd.DataFrame, y: np.ndarray, lam: float, name: str, target_mode: str, base_col: str) -> FittedRidge:
    clean = features.replace([np.inf, -np.inf], np.nan)
    median = clean.median(numeric_only=True).fillna(0.0)
    filled = clean.fillna(median)
    x = filled.to_numpy(dtype=float)
    mean = x.mean(axis=0)
    scale = x.std(axis=0)
    scale[scale < 1e-10] = 1.0
    z = (x - mean) / scale
    design = np.column_stack([np.ones(len(z)), z])
    ident = np.eye(design.shape[1])
    ident[0, 0] = 0.0
    coef = np.linalg.solve(design.T @ design + lam * ident, design.T @ y)
    return FittedRidge(name, list(features.columns), median, mean, scale, coef, lam, target_mode, base_col)


def predictRidge(model: FittedRidge, features: pd.DataFrame) -> np.ndarray:
    clean = features[model.columns].replace([np.inf, -np.inf], np.nan).fillna(model.median)
    x = clean.to_numpy(dtype=float)
    z = (x - model.mean) / model.scale
    design = np.column_stack([np.ones(len(z)), z])
    return design @ model.coef


def ridgeExperiment(data: pd.DataFrame, feature_mode: str, target_mode: str, base_col: str, lam: float) -> tuple[FittedRidge, np.ndarray]:
    features = buildFeatures(data, feature_mode)
    y = targetFromMode(data, target_mode, base_col)
    model = fitRidge(features, y, lam, f"{feature_mode}_{target_mode}_{base_col}_lam{lam:g}", target_mode, base_col)
    pred_target = predictRidge(model, features)
    volume = volumeFromPrediction(data, pred_target, target_mode, base_col)
    return model, volume


def scoreSummary(summary: pd.Series) -> float:
    penalty = 0.0
    penalty += max(0.0, float(summary["u_nor_L_pct"]) - 0.036) * 20.0
    penalty += max(0.0, float(summary["u_nor_r_pct"]) - 0.040) * 20.0
    penalty += max(0.0, float(summary["u_nor_d_pct"]) - 0.115) * 20.0
    penalty += (int(summary["group_count"]) - int(summary["group_pass_count"])) * 0.5
```

```
    return float(summary["mae_pct"]) + penalty


def runModelSearch(data: pd.DataFrame) -> tuple[pd.DataFrame, FittedRidge, np.ndarray]:
    rows = []
    best_model: FittedRidge | None = None
    best_volume: np.ndarray | None = None
    best_score = float("inf")

    baseline_cols = ["phys6_volume_m3", "lagrange_volume_m3", "owics_volume_m3", "equal_weight_volume_m3"]
    for col in baseline_cols:
        summary, _, _ = evaluatePredictions(data, data[col].astype(float).to_numpy())
        row = summary.iloc[0].to_dict()
        row.update({"model_name": col, "feature_count": 0, "lambda": None, "target_mode": "baseline", "base_col": col})
        row["selection_score"] = scoreSummary(summary.iloc[0])
        rows.append(row)

    feature_modes = ["compact", "rich", "poly"]
    target_modes = ["relative_correction", "log_ratio", "direct_volume"]
    base_cols = ["phys6_volume_m3", "lagrange_volume_m3"]
    lambdas = [1e-9, 1e-7, 1e-5, 1e-3, 1e-2, 1e-1, 1.0, 10.0]

    for feature_mode in feature_modes:
        feature_count = buildFeatures(data, feature_mode).shape[1]
        for target_mode in target_modes:
            for base_col in base_cols:
                if target_mode == "direct_volume" and base_col != "phys6_volume_m3":
                    continue
                for lam in lambdas:
                    try:
                        model, volume = ridgeExperiment(data, feature_mode, target_mode, base_col, lam)
                        summary, _, _ = evaluatePredictions(data, volume)
                    except np.linalg.LinAlgError:
                        continue
                    row = summary.iloc[0].to_dict()
                    row.update(
                        {
                            "model_name": model.name,
                            "feature_count": feature_count,
                            "lambda": lam,
                            "target_mode": target_mode,
                            "base_col": base_col,
                        }
                    )
                    row["selection_score"] = scoreSummary(summary.iloc[0])
                    rows.append(row)
                    if row["selection_score"] < best_score:
                        best_score = row["selection_score"]
                        best_model = model
                        best_volume = volume

    if best_model is None or best_volume is None:
        raise RuntimeError("No ridge model fitted")

    metrics = pd.DataFrame(rows).sort_values(["pass_all_3", "selection_score"], ascending=[False, True])
    return metrics, best_model, best_volume


def leaveOneDateCv(data: pd.DataFrame, model_template: FittedRidge, feature_mode: str) -> pd.DataFrame:
    yuceJieguo = []
    features_all = buildFeatures(data, feature_mode)
    for date in sorted(data["date"].unique()):
        train_idx = data["date"] != date
        test_idx = data["date"] == date
        y = targetFromMode(data.loc[train_idx], model_template.target_mode, model_template.base_col)
        model = fitRidge(
            features_all.loc[train_idx],
            y,
            model_template.lam,
            f"cv_{date}",
            model_template.target_mode,
            model_template.base_col,
        )
        pred_target = predictRidge(model, features_all.loc[test_idx])
        volume = volumeFromPrediction(data.loc[test_idx], pred_target, model.target_mode, model.base_col)
        fold = data.loc[test_idx, ["window_id", "date", "flow_point", "standard_volume_m3"]].copy()
        fold["cv_model_volume_m3"] = volume
        fold["cv_error_pct"] = relativeErrorPct(fold["cv_model_volume_m3"], fold["standard_volume_m3"])
        yuceJieguo.append(fold)
    return pd.concat(yuceJieguo, ignore_index=True).sort_values("window_id")


def exportSubmission(data: pd.DataFrame, volume: np.ndarray, path: Path) -> None:
    sub = pd.DataFrame({"window_id": data["window_id"], "model_volume_m3": volume})
    sub.to_csv(path, index=False, encoding="utf-8-sig", float_format="%.12g")


def saveModelArtifacts(model: FittedRidge) -> None:
    coef_df = pd.DataFrame({"term": ["intercept"] + model.columns, "coefficient": model.coef})
    coef_df.to_csv(OUTPUT_DIR / "model_coefficients.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    meta = {
        "artifact_role": "full-data interpolation submission; not a generalization result",
        "name": model.name,
```

```
        "lambda": model.lam,
        "target_mode": model.target_mode,
        "base_col": model.base_col,
        "feature_count": len(model.columns),
        "forbidden_online_inputs_excluded": [
            "standard_volume_m3",
            "standard_rate_m3h",
            "date",
            "start_time",
            "window_id",
            "condition_note",
            "is_disturbed",
            "disturbance_id",
            "disturbance_angle_deg",
        ],
    }
    (OUTPUT_DIR / "model_meta.json").write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")


def runOfficialEvaluator(submission_path: Path) -> None:
    official_dir = OUTPUT_DIR / "official_evaluation"
    if official_dir.exists():
        shutil.rmtree(official_dir)
    cmd = [sys.executable, str(EVALUATE_SCRIPT), str(submission_path), "--output-dir", str(official_dir)]
    subprocess.run(cmd, cwd=str(DATA_DIR), check=True)


def registerFonts() -> str:
    font_candidates = [
        Path(r"C:\Windows\Fonts\simsun.ttc"),
        Path(r"C:\Windows\Fonts\simhei.ttf"),
        Path(r"C:\Windows\Fonts\msyh.ttc"),
    ]
    for path in font_candidates:
        if path.exists():
            try:
                pdfmetrics.registerFont(TTFont("CNFont", str(path)))
                return "CNFont"
            except Exception:
                continue
    return "Helvetica"


def drawBarChart(path: Path, title: str, labels: list[str], values: list[float], y_label: str, font: str) -> None:
    width, height = landscape(A4)
    c = canvas.Canvas(str(path), pagesize=landscape(A4))
    margin = 60
    plot_w = width - 2 * margin
    plot_h = height - 2 * margin - 30
    c.setFont(font, 14)
    c.drawCentredString(width / 2, height - 35, title)
    max_v = max(values) if values else 1.0
    max_v = max_v * 1.15 if max_v > 0 else 1.0
    bar_gap = 16
    bar_w = (plot_w - bar_gap * (len(labels) + 1)) / max(1, len(labels))
    x0 = margin + bar_gap
    y0 = margin
    c.setStrokeColor(colors.black)
    c.line(margin, y0, margin, y0 + plot_h)
    c.line(margin, y0, margin + plot_w, y0)
    c.setFont(font, 10)
    c.drawString(12, y0 + plot_h / 2, y_label)
    for i, (label, value) in enumerate(zip(labels, values)):
        x = x0 + i * (bar_w + bar_gap)
        h = value / max_v * plot_h
        c.setFillColor(colors.HexColor("#4C78A8"))
        c.rect(x, y0, bar_w, h, stroke=0, fill=1)
        c.setFillColor(colors.black)
        c.drawCentredString(x + bar_w / 2, y0 - 18, label)
        c.drawCentredString(x + bar_w / 2, y0 + h + 8, f"{value:.4f}")
    c.save()


def drawErrorByFlow(path: Path, data: pd.DataFrame, font: str) -> None:
    width, height = landscape(A4)
    c = canvas.Canvas(str(path), pagesize=landscape(A4))
    margin = 60
    plot_w = width - 2 * margin
    plot_h = height - 2 * margin - 30
    c.setFont(font, 14)
    c.drawCentredString(width / 2, height - 35, "各流量点模型误差分布")
    stats = data.groupby("flow_point")["model_error_pct"].agg(["mean", "std", "min", "max"]).reset_index()
    stats.to_csv(FIG_DIR / "fig_error_by_flow_data.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    y_abs = max(float(stats["max"].abs().max()), float(stats["min"].abs().max()), 0.01)
    y_abs *= 1.2
    x_vals = stats["flow_point"].astype(float).to_numpy()
    xs = margin + (x_vals - x_vals.min()) / max(1e-9, x_vals.max() - x_vals.min()) * plot_w
    y_mid = margin + plot_h / 2
    c.setStrokeColor(colors.black)
    c.line(margin, y_mid, margin + plot_w, y_mid)
    c.line(margin, margin, margin, margin + plot_h)
    c.setFont(font, 10)
    c.drawString(12, y_mid, "误差(%)")
```

```
    for _, row in stats.iterrows():
        x = margin + (float(row["flow_point"]) - x_vals.min()) / max(1e-9, x_vals.max() - x_vals.min()) * plot_w
        y_mean = y_mid + float(row["mean"]) / y_abs * plot_h / 2
        y_min = y_mid + float(row["min"]) / y_abs * plot_h / 2
        y_max = y_mid + float(row["max"]) / y_abs * plot_h / 2
        c.setStrokeColor(colors.HexColor("#E45756"))
        c.line(x, y_min, x, y_max)
        c.setFillColor(colors.HexColor("#E45756"))
        c.circle(x, y_mean, 4, stroke=0, fill=1)
        c.setFillColor(colors.black)
        c.drawCentredString(x, margin - 18, str(int(row["flow_point"])))
    c.save()


def drawHistogram(path: Path, errors: pd.Series, font: str) -> None:
    width, height = landscape(A4)
    c = canvas.Canvas(str(path), pagesize=landscape(A4))
    margin = 60
    plot_w = width - 2 * margin
    plot_h = height - 2 * margin - 30
    c.setFont(font, 14)
    c.drawCentredString(width / 2, height - 35, "最终模型相对误差分布")
    counts, edges = np.histogram(errors.to_numpy(dtype=float), bins=16)
    hist_df = pd.DataFrame({"left": edges[:-1], "right": edges[1:], "count": counts})
    hist_df.to_csv(FIG_DIR / "fig_error_distribution_data.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    max_count = max(int(counts.max()), 1)
    bar_w = plot_w / len(counts)
    y0 = margin
    c.line(margin, y0, margin + plot_w, y0)
    c.line(margin, y0, margin, y0 + plot_h)
    c.setFont(font, 10)
    c.drawString(12, y0 + plot_h / 2, "窗口数")
    for i, count in enumerate(counts):
        x = margin + i * bar_w
        h = count / max_count * plot_h
        c.setFillColor(colors.HexColor("#72B7B2"))
        c.rect(x + 2, y0, bar_w - 4, h, stroke=0, fill=1)
    c.setFillColor(colors.black)
    c.drawCentredString(width / 2, margin - 22, "相对误差区间(%)")
    c.save()


def makeFigures(data: pd.DataFrame, metrics: pd.DataFrame, final_windows: pd.DataFrame) -> None:
    font = registerFonts()
    baseline = pd.read_csv(BASELINE_SUMMARY, encoding="utf-8-sig")
    labels = baseline["method"].tolist()
    values = baseline["mae_pct"].astype(float).tolist()
    pd.DataFrame({"method": labels, "mae_pct": values}).to_csv(
        FIG_DIR / "fig_model_mae_comparison_data.csv", index=False, encoding="utf-8-sig", float_format="%.12g"
    )
    drawBarChart(FIG_DIR / "fig_model_mae_comparison.pdf", "基础积分方法 MAE 对比", labels, values, "MAE(%)", font)
    drawErrorByFlow(FIG_DIR / "fig_error_by_flow.pdf", final_windows, font)
    drawHistogram(FIG_DIR / "fig_error_distribution.pdf", final_windows["model_error_pct"], font)


def writeResultsReport(
    metrics: pd.DataFrame,
    final_summary: pd.DataFrame,
    final_groups: pd.DataFrame,
    cv_predictions: pd.DataFrame,
    model: FittedRidge,
) -> None:
    best = final_summary.iloc[0]
    cv_mae = float(cv_predictions["cv_error_pct"].abs().mean())
    cv_max = float(cv_predictions["cv_error_pct"].abs().max())
    top = metrics.head(8)[["model_name", "mae_pct", "group_pass", "u_nor_L_pct", "u_nor_r_pct", "u_nor_d_pct", "pass_all_3"]]
    worst_groups = final_groups.sort_values(["pass_group", "repeatability_sd_pct", "mean_error_pct"], ascending=[True, False, False]).head(8)
    mae = float(best["mae_pct"])
    mean_err = float(best["mean_error_pct"])
    max_abs = float(best["max_abs_error_pct"])
    u_l = float(best["u_nor_L_pct"])
    u_r = float(best["u_nor_r_pct"])
    u_d = float(best["u_nor_d_pct"])

    report = f"""# 计算结果

## 运行环境

- Python: `{sys.version.split()[0]}`
- 主要依赖：`numpy`、`pandas`、`reportlab`
- 随机种子：`{RANDOM_SEED}`
- 主程序：`code/model_experiment.py`

## 数据读取与预处理

读取 `attachment1_window_data.csv` 的 159 个窗口样本。运行时输入仅使用持续时间、流量点、基础积分估计量、五声道观测量、A/B 差异、剖面特征、动态特征和零点特征。最终模型未使用 `standard_volume_m3`、`standard_rate_m3h`、`date`、`start_time`、`window_id`、`condition_note`、`is_disturbed`、`disturbance_id`、`disturbance_angle_deg` 作为在线输入。

## 问题一结果

基础积分方法中，Phys6 的整体 MAE 为 0.180653%，Lagrange 为 0.183613%，OWICS 和等权积分误差更大。因此实验以 Phys6 和 Lagrange 为主要物理基准，并训练相对修正项。

## 问题二与问题三模型
```

```
602
603 最终采用 `{model.name}`。模型形式为岭回归残差补偿，目标模式为 `{model.target_mode}`，物理基准列为 `{model.base_col}`，特征数为 `{len(model.columns)}`，正则化参数为 `{model.lam:g}`。该模型属于全量标定型模型：参数在全部已给定窗口标签上估计，再固化用于输出本题提交文件；运行时输入仍只包含在线可获得的流量与声道特征。
604
605 ## 全量插值提交评价（非泛化结果）
606
607 - 窗口数：{int(best['window_count'])}
608 - MAE：{mae:.6e}%
609 - 平均误差：{mean_err:.6e}%
610 - 最大绝对误差：{max_abs:.6e}%
611 - day-flow 分组通过数：{best['group_pass']}
612 - $u_{{\\mathrm{{nor}},L}}$：{u_l:.6e}%
613 - $u_{{\\mathrm{{nor}},r}}$：{u_r:.6e}%
614 - $u_{{\\mathrm{{nor}},d}}$：{u_d:.6e}%
615 - 三项综合指标是否全部达标：{bool(best['pass_all_3'])}
616
617 ## 留一日期验证
618
619 按日期留一验证的 MAE 为 {cv_mae:.6f}%，最大绝对误差为 {cv_max:.6f}%。这说明 159 个窗口的小样本数据难以支撑高维补偿项跨日期外推；若比赛存在未知测试窗口，应优先改用更强正则化或更低维的稳健模型。当前 `submission_fullfit_interpolation.csv` 仅用于记录全量插值基准，不是最终提交模型。
620
621 ## 候选模型摘要
622
623 {markdownTable(top)}
624
625 ## 最差分组检查
626
627 {markdownTable(worst_groups)}
628
629 ## 输出文件
630
631 - `code/outputs/submission_fullfit_interpolation.csv`：题目给定 159 个窗口的全量插值基准文件
632 - `code/outputs/experiment_metrics.csv`：候选模型评价
633 - `code/outputs/model_coefficients.csv`：最终模型系数
634 - `code/outputs/final_evaluation_summary.csv`：最终模型总体指标
635 - `code/outputs/final_evaluation_groups.csv`：day-flow 分组指标
636 - `code/outputs/final_evaluation_windows.csv`：逐窗口误差
637 - `code/outputs/cv_by_date_predictions.csv`：留一日期验证预测
638 - `code/outputs/official_evaluation/`：题目官方评价脚本输出
639
640 ## 图表
641
642 - `figures/fig_model_mae_comparison.pdf`
643 - `figures/fig_error_by_flow.pdf`
644 - `figures/fig_error_distribution.pdf`
645
646 ## 可复现运行方式
647
648 ```powershell
649 python code/model_experiment.py
650 ```
651 """
652     (REPORT_DIR / "RESULTS_REPORT.md").write_text(report, encoding="utf-8")
653
654
655 def markdownTable(df: pd.DataFrame) -> str:
656     if df.empty:
657         return "无"
658     view = df.copy()
659     for col in view.columns:
660         if pd.api.types.is_float_dtype(view[col]):
661             view[col] = view[col].map(lambda x: "" if pd.isna(x) else f"{float(x):.6g}")
662         else:
663             view[col] = view[col].map(lambda x: "" if pd.isna(x) else str(x))
664     header = "| " + " | ".join(view.columns) + " |"
665     sep = "| " + " | ".join(["---"] * len(view.columns)) + " |"
666     rows = ["| " + " | ".join(row) + " |" for row in view.astype(str).to_numpy()]
667     return "\n".join([header, sep] + rows)
668
669
670 def main() -> None:
671     ensureDirs()
672     np.random.seed(RANDOM_SEED)
673     data = loadData()
674
675     metrics, best_model, best_volume = runModelSearch(data)
676     final_summary, final_groups, final_windows = evaluatePredictions(data, best_volume)
677
678     metrics.to_csv(OUTPUT_DIR / "experiment_metrics.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
679     final_summary.to_csv(OUTPUT_DIR / "fullfit_interpolation_evaluation_summary.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
680     final_groups.to_csv(OUTPUT_DIR / "fullfit_interpolation_evaluation_groups.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
681     final_windows.to_csv(OUTPUT_DIR / "fullfit_interpolation_evaluation_windows.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
682
683     # This model is a full-data interpolation benchmark, not the final
684     # constrained DO-RBF postcalibrated dual-kernel KRR.
685     # submission. Keep the filename distinct so rerunning this comparison
686     # cannot overwrite the official-targeted result.
687     submission = OUTPUT_DIR / "submission_fullfit_interpolation.csv"
688     exportSubmission(data, best_volume, submission)
689     saveModelArtifacts(best_model)
690
691     feature_mode = best_model.name.split("_", 1)[0]
692     cv_predictions = leaveOneDateCv(data, best_model, feature_mode)
```

```
    cv_predictions.to_csv(OUTPUT_DIR / "cv_by_date_predictions.csv", index=False, encoding="utf-8-sig", float_format="%.12g")

    makeFigures(data, metrics, final_windows)
    runOfficialEvaluator(submission)
    writeResultsReport(metrics, final_summary, final_groups, cv_predictions, best_model)

    print("Best model:", best_model.name)
    print(final_summary.to_string(index=False))
    print("Submission:", submission)
    print("Report:", REPORT_DIR / "RESULTS_REPORT.md")


if __name__ == "__main__":
    main()
```

B.5 Ridge 与 ANN：model_evolution_ridge_ann.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题四对照：统一实现 Ridge 与 ANN 残差学习路线。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

import json
import math
import sys
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

try:
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4, landscape
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    from reportlab.pdfgen import canvas
except ModuleNotFoundError:
    colors = None
    A4 = None
    landscape = None
    pdfmetrics = None
    TTFont = None
    canvas = None


ROOT = Path(__file__).resolve().parents[2]
DATA_DIR = ROOT / "A题 (1)"
WINDOW_DATA = DATA_DIR / "attachment1_window_data.csv"
OUT_DIR = Path(__file__).resolve().parent / "verification_outputs"
FIG_DIR = Path(__file__).resolve().parent / "verification_figures"
REPORT = Path(__file__).resolve().parent / "verification_reports" / "FINAL_MODEL_THEORY_AND_RANKING.md"
RANDOM_SEED = 20260710
TRAIN_FRACTION = 0.70
SPLIT_MODE = "stratified_repeated_group_internal"
PRESSURE_SPLIT_MODE = "whole_group_holdout_pressure"
MIN_VALIDATION_REPEAT_GROUPS = 6
MLP_EPOCHS = 10000
CORRECTION_CLIP_PCT = 2.0

STANDARD_LIMITS = {
    "group_mean_abs_pct": 0.2,
    "group_sd_pct": 0.040,
    "u_nor_L_pct": 0.036,
    "u_nor_r_pct": 0.040,
    "u_nor_d_pct": 0.115,
}


@dataclass(frozen=True)
class ModelSpec:
    model_id: str
    family: str
    base_col: str
    feature_set: str
    model_type: str
    ridge_lambda: float = 0.0
    hidden_size: int = 0
    l2: float = 0.0
    learning_rate: float = 0.0
    seed_offset: int = 0
    shrinkage: float = 1.0
    epochs: int = 0


@dataclass
class Standardizer:
    columns: list[str]
    median: pd.Series
```

```
    mean: np.ndarray
    scale: np.ndarray

    @classmethod
    def fit(cls, frame: pd.DataFrame) -> "Standardizer":
        clean = frame.replace([np.inf, -np.inf], np.nan).copy()
        median = clean.median(numeric_only=True).fillna(0.0)
        clean = clean.fillna(median)
        arr = clean.to_numpy(dtype=float)
        mean = arr.mean(axis=0)
        scale = arr.std(axis=0)
        scale[scale < 1e-12] = 1.0
        return cls(list(frame.columns), median, mean, scale)

    def transform(self, frame: pd.DataFrame) -> np.ndarray:
        clean = frame[self.columns].replace([np.inf, -np.inf], np.nan).copy()
        clean = clean.fillna(self.median)
        return (clean.to_numpy(dtype=float) - self.mean) / self.scale


@dataclass
class FittedModel:
    spec: ModelSpec
    standardizer: Standardizer | None = None
    beta: np.ndarray | None = None
    w1: np.ndarray | None = None
    b1: np.ndarray | None = None
    w2: np.ndarray | None = None
    b2: float = 0.0

    def correctionPct(self, features: pd.DataFrame) -> np.ndarray:
        if self.spec.model_type == "baseline":
            return np.zeros(len(features))
        if self.standardizer is None:
            raise ValueError("standardizer is required")
        x = self.standardizer.transform(features)
        if self.spec.model_type == "ridge":
            if self.beta is None:
                raise ValueError("ridge beta is required")
            raw = x @ self.beta
        elif self.spec.model_type == "mlp":
            if self.w1 is None or self.b1 is None or self.w2 is None:
                raise ValueError("MLP weights are required")
            hidden = np.tanh(x @ self.w1 + self.b1)
            raw = hidden @ self.w2 + self.b2
        else:
            raise ValueError(self.spec.model_type)
        return self.spec.shrinkage * np.clip(raw, -CORRECTION_CLIP_PCT, CORRECTION_CLIP_PCT)

    def predictVolume(self, data: pd.DataFrame, features: pd.DataFrame) -> np.ndarray:
        base = data[self.spec.base_col].to_numpy(dtype=float)
        return base * (1.0 + self.correctionPct(features) / 100.0)


def ensureDirs() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    REPORT.parent.mkdir(parents=True, exist_ok=True)


def loadData() -> pd.DataFrame:
    return pd.read_csv(WINDOW_DATA, encoding="utf-8-sig").sort_values("window_id").reset_index(drop=True)


GROUP_KEY = ["date", "flow_point", "disturbance_id"]
OFFICIAL_EVAL_GROUP_KEY = ["date", "flow_point"]


def _splitFromIndices(
    data: pd.DataFrame,
    train_idx: set[int],
    validation_idx: set[int],
    split_mode: str,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    split = data[["window_id", "date", "flow_point", "condition_note", "disturbance_id"]].copy()
    split["split"] = ["train" if i in train_idx else "validation" for i in range(len(data))]
    split["split_mode"] = split_mode
    train = data.loc[split["split"].eq("train")].reset_index(drop=True)
    validation = data.loc[split["split"].eq("validation")].reset_index(drop=True)
    return train, validation, split


def makeRandomSplit(data: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    rng = np.random.default_rng(RANDOM_SEED)
    target_validation = int(round(len(data) * (1.0 - TRAIN_FRACTION)))
    validation_idx: set[int] = set()
    train_idx: set[int] = set()

    d0 = data[data["condition_note"].eq("no_disturbance_reference")]
    for _, group in d0.groupby(GROUP_KEY, sort=True):
        validation_idx.update(int(i) for i in group.index.to_numpy())

    whole_group_candidates = []
```

```
    for key, group in data[data["condition_note"].eq("disturbed_test")].groupby(GROUP_KEY, sort=True):
        indices = group.index.to_numpy()
        n = len(indices)
        if n >= 5:
            train_pick = set(int(i) for i in rng.choice(indices, size=2, replace=False))
            train_idx.update(train_pick)
            validation_idx.update(int(i) for i in indices if int(i) not in train_pick)
            continue
        if n in {3, 4}:
            date, flow, disturbance_id = key
            priority = int(40 <= int(flow) <= 100)
            whole_group_candidates.append((priority, str(date), int(flow), str(disturbance_id), indices))
            continue
        train_idx.update(int(i) for i in indices)

    order = rng.permutation(len(whole_group_candidates))
    selected_repeat_groups = 0
    for idx in sorted(order, key=lambda i: (-whole_group_candidates[int(i)][0], rng.random())):
        _, _, _, _, indices = whole_group_candidates[int(idx)]
        if selected_repeat_groups >= MIN_VALIDATION_REPEAT_GROUPS and len(validation_idx) >= target_validation:
            break
        validation_idx.update(int(i) for i in indices)
        selected_repeat_groups += 1

    for _, _, _, _, indices in whole_group_candidates:
        idx_set = set(int(i) for i in indices)
        if not idx_set & validation_idx:
            train_idx.update(idx_set)

    remaining = [i for i in rng.permutation(len(data)).tolist() if i not in validation_idx and i not in train_idx]
    for i in remaining:
        if len(validation_idx) >= target_validation:
            break
        validation_idx.add(int(i))

    train_idx = set(range(len(data))) - validation_idx
    return _splitFromIndices(data, train_idx, validation_idx, SPLIT_MODE)


def makeGroupHoldoutPressureSplit(data: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    rng = np.random.default_rng(RANDOM_SEED + 911)
    target_validation = int(round(len(data) * (1.0 - TRAIN_FRACTION)))
    validation_idx: set[int] = set()

    d0 = data[data["condition_note"].eq("no_disturbance_reference")]
    for _, group in d0.groupby(GROUP_KEY, sort=True):
        validation_idx.update(int(i) for i in group.index.to_numpy())

    candidates = []
    for key, group in data[data["condition_note"].eq("disturbed_test")].groupby(GROUP_KEY, sort=True):
        if len(group) < 3:
            continue
        date, flow, disturbance_id = key
        priority = int(40 <= int(flow) <= 100)
        candidates.append((priority, str(date), int(flow), str(disturbance_id), group.index.to_numpy()))

    order = rng.permutation(len(candidates))
    selected_repeat_groups = 0
    for idx in sorted(order, key=lambda i: (-candidates[int(i)][0], rng.random())):
        _, _, _, _, indices = candidates[int(idx)]
        if selected_repeat_groups >= MIN_VALIDATION_REPEAT_GROUPS and len(validation_idx) >= target_validation:
            break
        validation_idx.update(int(i) for i in indices)
        selected_repeat_groups += 1

    train_idx = set(range(len(data))) - validation_idx
    return _splitFromIndices(data, train_idx, validation_idx, PRESSURE_SPLIT_MODE)


def buildFeatureFrame(data: pd.DataFrame) -> pd.DataFrame:
    eps = 1e-12
    feats = pd.DataFrame(index=data.index)
    allowed = [
        "duration_s",
        "flow_point",
        "phys6_volume_m3",
        "owics_volume_m3",
        "lagrange_volume_m3",
        "equal_weight_volume_m3",
        "profile_top_bottom",
        "profile_center_all",
        "profile_edge_inner",
        "profile_inner_skew",
        "profile_ab_abs",
        "profile_swirl",
        "dyn_first_0p1_s",
        "dyn_tail_0p1_s",
        "dyn_start_over_plateau",
        "dyn_end_over_plateau",
        "dyn_plateau_cv",
        "dyn_active_eq_s",
        "zero_rate_med",
        "zero_rate_mad",
```

```
        "zero_age_s",
    ]
    for col in allowed:
        if col in data.columns:
            feats[col] = data[col].astype(float)

    q = (data["flow_point"].astype(float) - 50.0) / 30.0
    feats["q"] = q
    feats["q2"] = q**2
    feats["q3"] = q**3
    feats["log_duration"] = np.log(np.maximum(data["duration_s"].astype(float), eps))

    base = data["phys6_volume_m3"].astype(float)
    for col in ["owics_volume_m3", "lagrange_volume_m3", "equal_weight_volume_m3"]:
        feats[f"{col}_minus_phys6_frac"] = (data[col].astype(float) - base) / np.maximum(base.abs(), eps)

    chord_cols = [f"chord{i}" for i in range(5)]
    ab_cols = [f"ab{i}" for i in range(5)]
    chord = data[chord_cols].astype(float)
    ab = data[ab_cols].astype(float)
    chord_mean = chord.mean(axis=1).replace(0.0, np.nan)
    feats["chord_mean"] = chord_mean
    feats["chord_std"] = chord.std(axis=1)
    feats["chord_range_frac"] = (chord.max(axis=1) - chord.min(axis=1)) / chord_mean
    feats["chord_center_minus_edges_frac"] = (
        chord["chord2"] - 0.5 * (chord["chord0"] + chord["chord4"])
    ) / chord_mean
    feats["chord_edge_minus_inner_frac"] = (
        0.5 * (chord["chord0"] + chord["chord4"]) - 0.5 * (chord["chord1"] + chord["chord3"])
    ) / chord_mean
    feats["chord_top_bottom_frac"] = (chord["chord0"] - chord["chord4"]) / chord_mean
    feats["chord_inner_skew_frac"] = (chord["chord1"] - chord["chord3"]) / chord_mean
    feats["ab_mean"] = ab.mean(axis=1)
    feats["ab_abs_mean"] = ab.abs().mean(axis=1)
    feats["ab_std"] = ab.std(axis=1)
    feats["ab_range"] = ab.max(axis=1) - ab.min(axis=1)

    for col in [
        "profile_top_bottom",
        "profile_center_all",
        "profile_edge_inner",
        "profile_inner_skew",
        "profile_ab_abs",
        "profile_swirl",
        "dyn_plateau_cv",
        "dyn_active_eq_s",
        "zero_rate_med",
        "zero_rate_mad",
        "lagrange_volume_m3_minus_phys6_frac",
        "owics_volume_m3_minus_phys6_frac",
        "chord_center_minus_edges_frac",
        "chord_top_bottom_frac",
        "ab_abs_mean",
    ]:
        if col in feats.columns:
            feats[f"{col}_x_q"] = feats[col].astype(float) * q

    return feats.replace([np.inf, -np.inf], np.nan).fillna(0.0)


def featureColumns(frame: pd.DataFrame, feature_set: str) -> list[str]:
    profile = [
        "q",
        "q2",
        "profile_top_bottom",
        "profile_center_all",
        "profile_edge_inner",
        "profile_inner_skew",
        "profile_ab_abs",
        "profile_swirl",
        "owics_volume_m3_minus_phys6_frac",
        "lagrange_volume_m3_minus_phys6_frac",
        "equal_weight_volume_m3_minus_phys6_frac",
        "chord_range_frac",
        "chord_center_minus_edges_frac",
        "chord_edge_minus_inner_frac",
        "chord_top_bottom_frac",
        "chord_inner_skew_frac",
        "ab_abs_mean",
        "ab_range",
    ]
    dynamic_zero = [
        "duration_s",
        "log_duration",
        "dyn_start_over_plateau",
        "dyn_end_over_plateau",
        "dyn_plateau_cv",
        "dyn_active_eq_s",
        "zero_rate_med",
        "zero_rate_mad",
        "zero_age_s",
    ]
    interactions = [c for c in frame.columns if c.endswith("_x_q")]
```

```
    if feature_set == "profile_flow":
        cols = profile
    elif feature_set == "profile_dynamic_zero":
        cols = profile + dynamic_zero
    elif feature_set == "runtime_full":
        cols = profile + dynamic_zero + interactions + ["q3", "chord_mean", "chord_std", "ab_mean", "ab_std"]
    else:
        raise ValueError(feature_set)
    return [c for c in cols if c in frame.columns]


def residualTargetPct(data: pd.DataFrame, base_col: str) -> np.ndarray:
    base = np.maximum(data[base_col].to_numpy(dtype=float), 1e-12)
    return (data["standard_volume_m3"].to_numpy(dtype=float) / base - 1.0) * 100.0


def fitRidge(spec: ModelSpec, train: pd.DataFrame, xunlianTezheng: pd.DataFrame) -> FittedModel:
    cols = featureColumns(xunlianTezheng, spec.feature_set)
    std = Standardizer.fit(xunlianTezheng[cols])
    x = std.transform(xunlianTezheng[cols])
    y = residualTargetPct(train, spec.base_col)
    penalty = spec.ridge_lambda * np.eye(x.shape[1])
    beta = np.linalg.solve(x.T @ x + penalty, x.T @ y)
    return FittedModel(spec=spec, standardizer=std, beta=beta)


def fitMlp(spec: ModelSpec, train: pd.DataFrame, xunlianTezheng: pd.DataFrame) -> FittedModel:
    cols = featureColumns(xunlianTezheng, spec.feature_set)
    std = Standardizer.fit(xunlianTezheng[cols])
    x = std.transform(xunlianTezheng[cols])
    y = residualTargetPct(train, spec.base_col)
    rng = np.random.default_rng(RANDOM_SEED + spec.seed_offset + spec.hidden_size * 37)
    n, p = x.shape
    w1 = rng.normal(0.0, math.sqrt(2.0 / max(1, p)), size=(p, spec.hidden_size))
    b1 = np.zeros(spec.hidden_size)
    w2 = rng.normal(0.0, math.sqrt(2.0 / max(1, spec.hidden_size)), size=spec.hidden_size)
    b2 = float(np.mean(y))

    mw1 = np.zeros_like(w1)
    vw1 = np.zeros_like(w1)
    mb1 = np.zeros_like(b1)
    vb1 = np.zeros_like(b1)
    mw2 = np.zeros_like(w2)
    vw2 = np.zeros_like(w2)
    mb2 = 0.0
    vb2 = 0.0
    beta1 = 0.9
    beta2 = 0.999
    eps = 1e-8
    lr = spec.learning_rate
    l2 = spec.l2

    for epoch in range(1, spec.epochs + 1):
        hidden = np.tanh(x @ w1 + b1)
        pred = hidden @ w2 + b2
        err = pred - y
        dpred = (2.0 / n) * err
        grad_w2 = hidden.T @ dpred + 2.0 * l2 * w2
        grad_b2 = float(dpred.sum())
        dh = np.outer(dpred, w2) * (1.0 - hidden**2)
        grad_w1 = x.T @ dh + 2.0 * l2 * w1
        grad_b1 = dh.sum(axis=0)

        mw1 = beta1 * mw1 + (1.0 - beta1) * grad_w1
        vw1 = beta2 * vw1 + (1.0 - beta2) * (grad_w1**2)
        mb1 = beta1 * mb1 + (1.0 - beta1) * grad_b1
        vb1 = beta2 * vb1 + (1.0 - beta2) * (grad_b1**2)
        mw2 = beta1 * mw2 + (1.0 - beta1) * grad_w2
        vw2 = beta2 * vw2 + (1.0 - beta2) * (grad_w2**2)
        mb2 = beta1 * mb2 + (1.0 - beta1) * grad_b2
        vb2 = beta2 * vb2 + (1.0 - beta2) * (grad_b2**2)

        bc1 = 1.0 - beta1**epoch
        bc2 = 1.0 - beta2**epoch
        w1 -= lr * (mw1 / bc1) / (np.sqrt(vw1 / bc2) + eps)
        b1 -= lr * (mb1 / bc1) / (np.sqrt(vb1 / bc2) + eps)
        w2 -= lr * (mw2 / bc1) / (np.sqrt(vw2 / bc2) + eps)
        b2 -= lr * (mb2 / bc1) / (math.sqrt(vb2 / bc2) + eps)

    return FittedModel(spec=spec, standardizer=std, w1=w1, b1=b1, w2=w2, b2=b2)


def fitModel(spec: ModelSpec, train: pd.DataFrame, xunlianTezheng: pd.DataFrame) -> FittedModel:
    if spec.model_type == "baseline":
        return FittedModel(spec=spec)
    if spec.model_type == "ridge":
        return fitRidge(spec, train, xunlianTezheng)
    if spec.model_type == "mlp":
        return fitMlp(spec, train, xunlianTezheng)
    raise ValueError(spec.model_type)


def relativeErrorPct(moxingTiji: np.ndarray, biaozhunTiji: pd.Series | np.ndarray) -> np.ndarray:
```

```
    y = np.asarray(biaozhunTiji, dtype=float)
    return (np.asarray(moxingTiji, dtype=float) - y) / y * 100.0


def groupTable(df: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for (date, flow_point), group in df.groupby(OFFICIAL_EVAL_GROUP_KEY, sort=True):
        if len(group) < 3:
            continue
        values = group["model_error_pct"].astype(float)
        mean_err = float(values.mean())
        sd = float(values.std(ddof=1))
        rows.append(
            {
                "date": str(date),
                "flow_point": int(flow_point),
                "n": int(len(group)),
                "mean_error_pct": mean_err,
                "abs_mean_error_pct": abs(mean_err),
                "repeatability_sd_pct": sd,
                "range_pct": float(values.max() - values.min()),
                "pass_mean": bool(abs(mean_err) <= STANDARD_LIMITS["group_mean_abs_pct"]),
                "pass_sd": bool(sd <= STANDARD_LIMITS["group_sd_pct"]),
                "pass_group": bool(abs(mean_err) <= STANDARD_LIMITS["group_mean_abs_pct"] and sd <= STANDARD_LIMITS["group_sd_pct"]),
            }
        )
    return pd.DataFrame(rows)


def linearityUL(df: pd.DataFrame) -> float:
    use = df[(df["condition_note"].eq("no_disturbance_reference")) & (df["flow_point"].between(40, 100))]
    by_flow = use.groupby("flow_point")["model_error_pct"].mean()
    if len(by_flow) <= 1:
        return float("nan")
    return math.sqrt(float((by_flow**2).sum()) / (len(by_flow) - 1))


def disturbanceUD(df: pd.DataFrame) -> float:
    use = df[df["flow_point"].between(40, 100)].copy()
    base = (
        use[use["condition_note"].eq("no_disturbance_reference")]
        .groupby("flow_point")["model_error_pct"]
        .agg(["mean", "count", "std"])
        .rename(columns={"mean": "baseline_mean", "count": "baseline_n", "std": "baseline_sd"})
        .reset_index()
    )
    disturb = (
        use[use["condition_note"].eq("disturbed_test")]
        .groupby(["disturbance_id", "flow_point"])["model_error_pct"]
        .agg(["mean", "count", "std"])
        .rename(columns={"mean": "disturb_mean", "count": "disturb_n", "std": "disturb_sd"})
        .reset_index()
    )
    detail = disturb.merge(base, on="flow_point", how="left")
    if detail.empty:
        return float("nan")
    detail["disturb_sd"] = detail["disturb_sd"].fillna(0.0)
    detail["abs_drift"] = (detail["baseline_mean"] - detail["disturb_mean"]).abs()
    available = detail[(detail["baseline_n"] >= 1) & (detail["disturb_n"] >= 1)].copy()
    if available.empty:
        return float("nan")
    u_d_c = float(available["abs_drift"].max()) / math.sqrt(3.0)
    u_d_r = float(available["disturb_sd"].max())
    return math.sqrt(u_d_c * u_d_c + u_d_r * u_d_r)


def evaluatePredictions(data: pd.DataFrame, moxingTiji: np.ndarray) -> tuple[pd.Series, pd.DataFrame, pd.DataFrame]:
    df = data.copy()
    df["model_volume_m3"] = moxingTiji
    df["model_error_pct"] = relativeErrorPct(moxingTiji, df["standard_volume_m3"])
    groups = groupTable(df)
    u_l = linearityUL(df)
    u_r = float(groups["repeatability_sd_pct"].max()) if not groups.empty else float("nan")
    u_d = disturbanceUD(df)
    max_mean = float(groups["abs_mean_error_pct"].max()) if not groups.empty else float("nan")
    max_sd = float(groups["repeatability_sd_pct"].max()) if not groups.empty else float("nan")
    pass_group_mean_all = bool(pd.notna(max_mean) and max_mean <= STANDARD_LIMITS["group_mean_abs_pct"])
    pass_group_sd_all = bool(pd.notna(max_sd) and max_sd <= STANDARD_LIMITS["group_sd_pct"])
    row = pd.Series(
        {
            "window_count": int(len(df)),
            "mae_pct": float(df["model_error_pct"].abs().mean()),
            "mean_error_pct": float(df["model_error_pct"].mean()),
            "max_abs_error_pct": float(df["model_error_pct"].abs().max()),
            "group_count": int(len(groups)),
            "group_pass_count": int(groups["pass_group"].sum()) if not groups.empty else 0,
            "group_pass": f"{int(groups['pass_group'].sum())}/{len(groups)}" if not groups.empty else "0/0",
            "max_abs_group_mean_pct": max_mean,
            "max_group_sd_pct": max_sd,
            "u_nor_L_pct": u_l,
            "u_nor_r_pct": u_r,
            "u_nor_d_pct": u_d,
            "pass_group_mean_all": pass_group_mean_all,
```

```
            "pass_group_sd_all": pass_group_sd_all,
            "pass_u_nor_L": bool(pd.notna(u_l) and u_l < STANDARD_LIMITS["u_nor_L_pct"]),
            "pass_u_nor_r": bool(pd.notna(u_r) and u_r < STANDARD_LIMITS["u_nor_r_pct"]),
            "pass_u_nor_d": bool(pd.notna(u_d) and u_d < STANDARD_LIMITS["u_nor_d_pct"]),
        }
    )
    row["pass_all_standards"] = bool(
        row["pass_group_mean_all"]
        and row["pass_group_sd_all"]
        and row["pass_u_nor_L"]
        and row["pass_u_nor_r"]
        and row["pass_u_nor_d"]
    )
    windows = df[
        [
            "window_id",
            "date",
            "flow_point",
            "condition_note",
            "disturbance_id",
            "standard_volume_m3",
            "model_volume_m3",
            "model_error_pct",
        ]
    ].copy()
    return row, groups, windows


def standardsGap(row: pd.Series) -> tuple[int, float]:
    checks = [
        ("max_abs_group_mean_pct", STANDARD_LIMITS["group_mean_abs_pct"], False),
        ("max_group_sd_pct", STANDARD_LIMITS["group_sd_pct"], False),
        ("u_nor_L_pct", STANDARD_LIMITS["u_nor_L_pct"], True),
        ("u_nor_r_pct", STANDARD_LIMITS["u_nor_r_pct"], True),
        ("u_nor_d_pct", STANDARD_LIMITS["u_nor_d_pct"], True),
    ]
    fails = 0
    gap = 0.0
    for col, limit, strict in checks:
        value = row.get(col, np.nan)
        if pd.isna(value):
            fails += 1
            gap += 10.0
            continue
        failed = value >= limit if strict else value > limit
        if failed:
            fails += 1
            gap += (float(value) - limit) / limit
    gap += float(row.get("mae_pct", 0.0))
    return fails, gap


def buildSpecs() -> list[ModelSpec]:
    specs: list[ModelSpec] = []
    for base_col in ["phys6_volume_m3", "lagrange_volume_m3", "owics_volume_m3"]:
        specs.append(
            ModelSpec(
                model_id=f"baseline_{base_col}",
                family="fixed_weight_integral",
                base_col=base_col,
                feature_set="none",
                model_type="baseline",
            )
        )

    for base_col in ["phys6_volume_m3", "lagrange_volume_m3"]:
        for feature_set in ["profile_flow", "profile_dynamic_zero", "runtime_full"]:
            for lam in [0.01, 0.1, 1.0, 10.0, 100.0]:
                specs.append(
                    ModelSpec(
                        model_id=f"ridge_{base_col}_{feature_set}_lam{lam:g}",
                        family=f"ridge_{feature_set}",
                        base_col=base_col,
                        feature_set=feature_set,
                        model_type="ridge",
                        ridge_lambda=lam,
                        shrinkage=1.0,
                    )
                )

    for base_col in ["phys6_volume_m3", "lagrange_volume_m3"]:
        for hidden in [8, 16, 24]:
            for l2 in [1e-4, 1e-3, 1e-2]:
                for lr in [0.003, 0.01]:
                    for seed_offset in [0, 17]:
                        for shrinkage in [0.50, 0.75, 1.00]:
                            specs.append(
                                ModelSpec(
                                    model_id=(
                                        f"ann_{base_col}_runtime_full_h{hidden}_"
                                        f"l2{l2:g}_lr{lr:g}_s{seed_offset}_shrink{shrinkage:g}"
                                    ),
                                    family="ann_runtime_full",
```

```
                                base_col=base_col,
                                feature_set="runtime_full",
                                model_type="mlp",
                                hidden_size=hidden,
                                l2=l2,
                                learning_rate=lr,
                                seed_offset=seed_offset,
                                shrinkage=shrinkage,
                                epochs=MLP_EPOCHS,
                            )
                        )
    return specs


def runSpecs(
    specs: list[ModelSpec],
    train: pd.DataFrame,
    evaluations: dict[str, pd.DataFrame],
) -> tuple[pd.DataFrame, dict[str, FittedModel], dict[tuple[str, str], pd.DataFrame], dict[tuple[str, str], pd.DataFrame]]:
    train_features_all = buildFeatureFrame(train)
    eval_features_all = {name: buildFeatureFrame(frame) for name, frame in evaluations.items()}
    rows = []
    fitted: dict[str, FittedModel] = {}
    groups_by_key: dict[tuple[str, str], pd.DataFrame] = {}
    windows_by_key: dict[tuple[str, str], pd.DataFrame] = {}

    train_cache: dict[tuple, FittedModel] = {}
    for spec in specs:
        train_key = (
            spec.model_type,
            spec.base_col,
            spec.feature_set,
            spec.ridge_lambda,
            spec.hidden_size,
            spec.l2,
            spec.learning_rate,
            spec.seed_offset,
            spec.epochs,
        )
        if spec.model_type == "mlp":
            unshrunk_spec = ModelSpec(
                model_id=spec.model_id.rsplit("_shrink", 1)[0] + "_shrink1",
                family=spec.family,
                base_col=spec.base_col,
                feature_set=spec.feature_set,
                model_type=spec.model_type,
                hidden_size=spec.hidden_size,
                l2=spec.l2,
                learning_rate=spec.learning_rate,
                seed_offset=spec.seed_offset,
                shrinkage=1.0,
                epochs=spec.epochs,
            )
            train_key = (
                spec.model_type,
                spec.base_col,
                spec.feature_set,
                spec.hidden_size,
                spec.l2,
                spec.learning_rate,
                spec.seed_offset,
                spec.epochs,
            )
            if train_key not in train_cache:
                train_cache[train_key] = fitModel(unshrunk_spec, train, train_features_all)
            base_model = train_cache[train_key]
            model = FittedModel(
                spec=spec,
                standardizer=base_model.standardizer,
                w1=base_model.w1,
                b1=base_model.b1,
                w2=base_model.w2,
                b2=base_model.b2,
            )
        else:
            if train_key not in train_cache:
                train_cache[train_key] = fitModel(spec, train, train_features_all)
            model = train_cache[train_key]
            if model.spec.model_id != spec.model_id:
                model = FittedModel(spec=spec, standardizer=model.standardizer, beta=model.beta)
        fitted[spec.model_id] = model

        for operation, frame in evaluations.items():
            features = eval_features_all[operation]
            pred = model.predictVolume(frame, features)
            summary, groups, windows = evaluatePredictions(frame, pred)
            fail_count, gap = standardsGap(summary)
            row = {
                "model_id": spec.model_id,
                "family": spec.family,
                "operation": operation,
                "model_type": spec.model_type,
                "base_col": spec.base_col,
```

```
                "feature_set": spec.feature_set,
                "ridge_lambda": spec.ridge_lambda,
                "hidden_size": spec.hidden_size,
                "l2": spec.l2,
                "learning_rate": spec.learning_rate,
                "epochs": spec.epochs,
                "seed_offset": spec.seed_offset,
                "shrinkage": spec.shrinkage,
                "standard_fail_count": fail_count,
                "standard_gap_score": gap,
            }
            row.update(summary.to_dict())
            rows.append(row)
            groups_by_key[(operation, spec.model_id)] = groups
            windows_by_key[(operation, spec.model_id)] = windows

    metrics = pd.DataFrame(rows)
    return metrics, fitted, groups_by_key, windows_by_key


def buildComprehensiveRanking(metrics: pd.DataFrame) -> pd.DataFrame:
    full = metrics[metrics["operation"].eq("full_fit_all")].set_index("model_id")
    train = metrics[metrics["operation"].eq("random_split_train")].set_index("model_id")
    valid = metrics[metrics["operation"].eq("random_split_validation")].set_index("model_id")
    rows = []
    for model_id, v in valid.iterrows():
        if model_id not in full.index:
            continue
        f = full.loc[model_id]
        t = train.loc[model_id] if model_id in train.index else v
        overfit = max(0.0, float(v["mae_pct"]) - float(t["mae_pct"]))
        rank_score = (
            1000.0 * float(v["standard_fail_count"])
            + 80.0 * float(v["standard_gap_score"])
            + 250.0 * float(f["standard_fail_count"])
            + 20.0 * float(f["standard_gap_score"])
            + 10.0 * overfit
            + float(v["mae_pct"])
        )
        rows.append(
            {
                "model_id": model_id,
                "family": v["family"],
                "model_type": v["model_type"],
                "base_col": v["base_col"],
                "feature_set": v["feature_set"],
                "rank_score": rank_score,
                "validation_fail_count": int(v["standard_fail_count"]),
                "full_fail_count": int(f["standard_fail_count"]),
                "validation_pass_all": bool(v["pass_all_standards"]),
                "full_pass_all": bool(f["pass_all_standards"]),
                "validation_mae_pct": float(v["mae_pct"]),
                "validation_mean_error_pct": float(v["mean_error_pct"]),
                "validation_group_pass": v["group_pass"],
                "validation_max_abs_group_mean_pct": float(v["max_abs_group_mean_pct"]),
                "validation_max_group_sd_pct": float(v["max_group_sd_pct"]),
                "validation_u_nor_L_pct": float(v["u_nor_L_pct"]),
                "validation_u_nor_r_pct": float(v["u_nor_r_pct"]),
                "validation_u_nor_d_pct": float(v["u_nor_d_pct"]),
                "train_mae_pct": float(t["mae_pct"]),
                "full_mae_pct": float(f["mae_pct"]),
                "full_group_pass": f["group_pass"],
                "full_u_nor_L_pct": float(f["u_nor_L_pct"]),
                "full_u_nor_r_pct": float(f["u_nor_r_pct"]),
                "full_u_nor_d_pct": float(f["u_nor_d_pct"]),
                "hidden_size": int(v["hidden_size"]),
                "ridge_lambda": float(v["ridge_lambda"]),
                "l2": float(v["l2"]),
                "learning_rate": float(v["learning_rate"]),
                "epochs": int(v["epochs"]),
                "shrinkage": float(v["shrinkage"]),
            }
        )
    return pd.DataFrame(rows).sort_values(["rank_score", "validation_mae_pct"]).reset_index(drop=True)


def buildPressureRanking(metrics: pd.DataFrame) -> pd.DataFrame:
    full = metrics[metrics["operation"].eq("full_fit_all")].set_index("model_id")
    train = metrics[metrics["operation"].eq("group_holdout_train")].set_index("model_id")
    valid = metrics[metrics["operation"].eq("group_holdout_validation")].set_index("model_id")
    rows = []
    for model_id, v in valid.iterrows():
        if model_id not in full.index:
            continue
        f = full.loc[model_id]
        t = train.loc[model_id] if model_id in train.index else v
        overfit = max(0.0, float(v["mae_pct"]) - float(t["mae_pct"]))
        rank_score = (
            1000.0 * float(v["standard_fail_count"])
            + 80.0 * float(v["standard_gap_score"])
            + 250.0 * float(f["standard_fail_count"])
            + 20.0 * float(f["standard_gap_score"])
            + 10.0 * overfit
```

```
            + float(v["mae_pct"])
        )
        rows.append(
            {
                "model_id": model_id,
                "family": v["family"],
                "model_type": v["model_type"],
                "base_col": v["base_col"],
                "feature_set": v["feature_set"],
                "rank_score": rank_score,
                "pressure_fail_count": int(v["standard_fail_count"]),
                "full_fail_count": int(f["standard_fail_count"]),
                "pressure_pass_all": bool(v["pass_all_standards"]),
                "full_pass_all": bool(f["pass_all_standards"]),
                "pressure_mae_pct": float(v["mae_pct"]),
                "pressure_mean_error_pct": float(v["mean_error_pct"]),
                "pressure_group_pass": v["group_pass"],
                "pressure_max_abs_group_mean_pct": float(v["max_abs_group_mean_pct"]),
                "pressure_max_group_sd_pct": float(v["max_group_sd_pct"]),
                "pressure_u_nor_L_pct": float(v["u_nor_L_pct"]),
                "pressure_u_nor_r_pct": float(v["u_nor_r_pct"]),
                "pressure_u_nor_d_pct": float(v["u_nor_d_pct"]),
                "pressure_train_mae_pct": float(t["mae_pct"]),
                "full_mae_pct": float(f["mae_pct"]),
                "full_group_pass": f["group_pass"],
                "full_u_nor_L_pct": float(f["u_nor_L_pct"]),
                "full_u_nor_r_pct": float(f["u_nor_r_pct"]),
                "full_u_nor_d_pct": float(f["u_nor_d_pct"]),
            }
        )
    if not rows:
        return pd.DataFrame()
    return pd.DataFrame(rows).sort_values(["rank_score", "pressure_mae_pct"]).reset_index(drop=True)


def bestByFamily(ranking: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for family, sub in ranking.groupby("family", sort=False):
        rows.append(sub.sort_values(["rank_score", "validation_mae_pct"]).iloc[0])
    return pd.DataFrame(rows).sort_values(["rank_score", "validation_mae_pct"]).reset_index(drop=True)


def buildFullFitRanking(metrics: pd.DataFrame) -> pd.DataFrame:
    full = metrics[metrics["operation"].eq("full_fit_all")].copy()
    full = full.sort_values(["standard_fail_count", "standard_gap_score", "mae_pct"]).reset_index(drop=True)
    cols = [
        "model_id",
        "family",
        "model_type",
        "base_col",
        "feature_set",
        "standard_fail_count",
        "standard_gap_score",
        "pass_all_standards",
        "mae_pct",
        "mean_error_pct",
        "group_pass",
        "max_abs_group_mean_pct",
        "max_group_sd_pct",
        "u_nor_L_pct",
        "u_nor_r_pct",
        "u_nor_d_pct",
        "hidden_size",
        "ridge_lambda",
        "l2",
        "learning_rate",
        "epochs",
        "shrinkage",
    ]
    return full[cols]


def markdownTable(df: pd.DataFrame, max_rows: int | None = None, floatfmt: str = ".6g") -> str:
    if max_rows is not None:
        df = df.head(max_rows)
    if df.empty:
        return "_无数据_"
    view = df.copy()
    for col in view.columns:
        if pd.api.types.is_float_dtype(view[col]):
            view[col] = view[col].map(lambda x: "" if pd.isna(x) else format(float(x), floatfmt))
        else:
            view[col] = view[col].map(lambda x: "" if pd.isna(x) else str(x))
    header = "| " + " | ".join(view.columns) + " |"
    sep = "| " + " | ".join(["---"] * len(view.columns)) + " |"
    rows = ["| " + " | ".join(row) + " |" for row in view.astype(str).to_numpy()]
    return "\n".join([header, sep] + rows)


def fontName() -> str:
    if pdfmetrics is None:
        return "Helvetica"
    font_path = Path(r"C:\Windows\Fonts\msyh.ttc")
```

```
    if font_path.exists():
        try:
            pdfmetrics.registerFont(TTFont("MSYH", str(font_path), subfontIndex=0))
            return "MSYH"
        except Exception:
            return "Helvetica"
    return "Helvetica"


def drawRankingFigure(path: Path, ranking: pd.DataFrame) -> None:
    if canvas is None:
        return
    font = fontName()
    top = ranking.head(10).copy()
    c = canvas.Canvas(str(path), pagesize=landscape(A4))
    width, height = landscape(A4)
    c.setFont(font, 12)
    c.drawString(42, height - 40, "模型综合排名：随机验证 MAE 与达标差距")
    chart_x, chart_y, chart_w, chart_h = 70, 92, 650, 360
    max_mae = max(float(top["validation_mae_pct"].max()), 1e-9)
    row_h = chart_h / max(1, len(top))
    for i, row in enumerate(top.itertuples(index=False)):
        y = chart_y + chart_h - (i + 1) * row_h + 5
        bar_w = chart_w * float(row.validation_mae_pct) / max_mae
        color = "#22C55E" if bool(row.validation_pass_all) else "#3B82F6"
        c.setFillColor(colors.HexColor(color))
        c.rect(chart_x, y, bar_w, row_h * 0.55, stroke=0, fill=1)
        c.setFillColor(colors.HexColor("#111827"))
        c.setFont(font, 8)
        label = str(row.family)[:28] + f" ({row.base_col.replace('_volume_m3', '')})"
        c.drawRightString(chart_x - 6, y + 2, label)
        c.drawString(chart_x + bar_w + 5, y + 2, f"{float(row.validation_mae_pct):.4f}% / fail {int(row.validation_fail_count)}")
    c.setFont(font, 9)
    c.drawString(chart_x + chart_w / 2 - 40, 42, "validation MAE / %")
    c.save()


def drawStandardGapFigure(path: Path, best: pd.Series, metrics: pd.DataFrame) -> None:
    if canvas is None:
        return
    font = fontName()
    rows = metrics[(metrics["model_id"].eq(best["model_id"])) & (metrics["operation"].isin(["full_fit_all", "random_split_validation"]))]
    labels = [
        ("max_abs_group_mean_pct", STANDARD_LIMITS["group_mean_abs_pct"], "|eg_bar|"),
        ("max_group_sd_pct", STANDARD_LIMITS["group_sd_pct"], "SDg"),
        ("u_nor_L_pct", STANDARD_LIMITS["u_nor_L_pct"], "u_nor,L"),
        ("u_nor_r_pct", STANDARD_LIMITS["u_nor_r_pct"], "u_nor,r"),
        ("u_nor_d_pct", STANDARD_LIMITS["u_nor_d_pct"], "u_nor,d"),
    ]
    c = canvas.Canvas(str(path), pagesize=landscape(A4))
    width, height = landscape(A4)
    c.setFont(font, 12)
    c.drawString(42, height - 40, "最优模型与达标阈值对比")
    x0, y0, bar_w, row_h = 220, height - 95, 420, 42
    palette = {"full_fit_all": "#10B981", "random_split_validation": "#2563EB"}
    for i, (metric, limit, label) in enumerate(labels):
        y = y0 - i * row_h
        c.setFillColor(colors.HexColor("#111827"))
        c.setFont(font, 9)
        c.drawRightString(x0 - 10, y + 6, f"{label} limit={limit:g}%")
        c.setStrokeColor(colors.HexColor("#EF4444"))
        c.line(x0 + bar_w * 0.5, y - 3, x0 + bar_w * 0.5, y + 28)
        for j, r in enumerate(rows.itertuples(index=False)):
            value = float(getattr(r, metric)) if pd.notna(getattr(r, metric)) else 0.0
            width_v = min(bar_w, bar_w * value / max(2.0 * limit, 1e-9))
            yy = y + 15 - j * 14
            c.setFillColor(colors.HexColor(palette[getattr(r, "operation")]))
            c.rect(x0, yy, width_v, 8, stroke=0, fill=1)
            c.setFillColor(colors.HexColor("#111827"))
            c.drawString(x0 + width_v + 5, yy - 1, f"{getattr(r, 'operation')}: {value:.5f}%")
    c.save()


def theorySections() -> str:
    return """## 1. 论文库理论与公式准备

本节把论文库中可直接支撑最终建模的理论压缩为可落地公式。若多个理论重合，优先采用近期期刊和本题数据字段可以直接实现的版本。

### 1.1 多声道积分与圆截面平均速度

多声道 UFM 的基本目标是由各弦高处线平均速度 `u_i` 估计截面平均流速。圆截面平均速度可写成带几何权重的积分：

```text
v_bar = (2/pi) Integral[-1,1] u(eta) sqrt(1-eta^2) d eta
V_hat = A T v_bar
```

其中 `eta=y/R`，`A` 为等效截面积，`T` 为窗口时间。Gauss-Jacobi 和 OWICS 把圆截面几何权重或圆截面最优权重并入离散积分；Lagrange 型积分先用五个弦高节点重构速度剖面，再积分得到固定权重。这对应相诺林关于高精度多声道积分和影响因素的结论：物理积分是主尺度，速度剖面变化会导致固定权重失配。

1.2 剖面修正因子

Mousavi 等关于 geometric flow profile correction factor 的思想可写为：
```

```
```text
v_bar = K(profile) * v_path
```

在本题的五声道场景下，将其扩展为：

```text
V_hat = V_base * C(x),    C(x)=1+f(x)/100
```

其中 `V_base` 取 Phys6、Lagrange 或 OWICS 等物理积分估计，`x` 为中心-边缘、边缘-内侧、上下弦差、A/B 方向差、旋流代理等运行时剖面特征。Gryshanova 2023 和 Sakhavi 2022 的结论共同支持：非轴对称剖面会改变修正因子，模型不能只假设单一理想剖面。

1.3 扰流补偿与在线可观测特征

Papathanasiou 2022、金少杰 2025 和苏杰 2024 均说明上游扰流、弯管、非充分发展流会导致声道方向和剖面形态出现结构化偏移。因此本文不把 `disturbance_id` 当作输入，而是使用在线可观测代理：

```text
x_profile = (profile_top_bottom, profile_center_all, profile_edge_inner,
             profile_inner_skew, profile_ab_abs, profile_swirl)
```

这些变量分别刻画上下不对称、中心高速核、近壁梯度、内侧偏斜、A/B 方向差和旋流/偏流。

1.4 机器学习残差补偿

Zhao 2014 ANN、Li 2023 in-use measurement 和黄恩扬 2024 BP 神经网络补偿都支持用数据驱动方法学习多声道特征到测量误差的映射。为避免黑箱模型脱离物理主尺度，本文只学习相对残差：

```text
r = (V_ref / V_base - 1) * 100
V_hat = V_base * (1 + f_theta(x) / 100)
```

线性补偿采用岭回归：

```text
min_beta sum_j (r_j - beta^T z_j)^2 + lambda ||beta||_2^2
```

ANN 补偿采用一隐层 BP/MLP：

```text
f_theta(x)=a_2^T tanh(W_1 z+b_1)+b_2
```

结合 Cai 2025 物理约束数据增强/校准思想，最终将补偿幅度限制为 `|f_theta(x)| <= 2%`，即修正因子在 `[0.98,1.02]` 内，防止小样本下过拟合出非物理补偿。

1.5 评价公式

窗口相对误差、组均值误差和组内重复性为：

```text
e_j = (V_hat_j - V_ref_j) / V_ref_j * 100%
eg_bar = mean_{j in g}(e_j)
SD_g = sqrt( sum_{j in g}(e_j - eg_bar)^2 / (n_g-1) )
```

综合不确定度采用题面/评价脚本口径：

```text
u_nor,L = sqrt( sum_q eg_q^2 / (m-1) )
u_nor,r = max_g SD_g
u_nor,d = sqrt( (max |eg_D0,q - eg_Dk,q| / sqrt(3))^2 + (max SD_Dk,q)^2 )
```

达标参考为 `|eg_bar| <= 0.2%`、`SD_g <= 0.040%`、`u_nor,L < 0.036%`、`u_nor,r < 0.040%`、`u_nor,d < 0.115%`。

1.6 论文索引到模型组件的对应关系

文献	可吸收理论/公式	本实验落地方式
Mousavi2020FPCF	几何速度剖面修正因子 `v_bar=K(profile)v_path`	把 `K` 写成 `C(x)=1+f(x)/100`
Sakhavi2022GeneralizedProfile	多声道广义速度剖面评价	使用中心-边缘、边缘-内侧、上下偏斜等剖面特征
Gryshanova2023CorrectionFactor	非轴对称剖面下修正因子随测量平面变化	加入 A/B 方向差、上下弦差和旋流代理
Papathanasiou2022FlowDC	基于流场仿真的扰流补偿因子	采用扰流代理特征学习残差补偿，不直接输入扰流编号
JinUpstreamDisturbance	上游扰流误差可由修正系数显著降低	将扰流漂移纳入 `u_nor,d` 排名约束
SuUnderdevelopedFlow	非充分发展流中 A/B 声道组合存在偏移	使用 `profile_ab_abs`、`profile_swirl`、A/B 统计量
XiangMultipathIntegration	速度剖面决定积分误差，AWICS-VP 自适应权重优于固定权重	保留 Phys6/Lagrange/OWICS 作为物理基准，再学习小幅修正
Zhao2014ANN	ANN 可融合多路径声道估计截面平均速度	构造一隐层 MLP/BP 残差模型
Zhao2014CFD	CFD 揭示声道方向、扰流强度和复杂流型对误差的影响	用方向性和剖面特征解释 ML 输入，不把 ML 当纯黑箱
Li2023MLInUse	RF/ANN 可预测在用超声流量计误差	将监督目标设为相对残差 `r=(V_ref/V_base-1)100%`
HuangClampOnOil	Pearson 筛选 + BP 神经网络可补偿外夹式 UFM 误差	使用 BP/MLP 与动态、零点、声道特征联合补偿
Cai2025PhysConstrainedGAN	小样本下应加入物理约束和分布一致性	补偿幅度裁剪到 `[-2%,2%]`，并用随机验证检查泛化
Li2025VMDDenoising	回波噪声会影响瞬时流速和稳定性	保留 `dyn_*`、`zero_*` 特征作为噪声/稳定性代理
Qin2016ProfileIdentification	多声道特征可识别流型且对不确定性较稳健	构造在线剖面代理，避免依赖人工 `disturbance_id`
"""

def writeReport(
    data: pd.DataFrame,
    split: pd.DataFrame,
    pressureSplit: pd.DataFrame,
```

```
    metrics: pd.DataFrame,
    ranking: pd.DataFrame,
    pressure_ranking: pd.DataFrame,
    family_ranking: pd.DataFrame,
    full_fit_ranking: pd.DataFrame,
    best_groups: pd.DataFrame,
    best_full_groups: pd.DataFrame,
) -> None:
    best = ranking.iloc[0]
    best_full = full_fit_ranking.iloc[0]
    split_counts = split["split"].value_counts().rename_axis("split").reset_index(name="count")
    pressure_split_counts = pressureSplit["split"].value_counts().rename_axis("split").reset_index(name="count")
    group_sort_cols = ["pass_group", "repeatability_sd_pct", "abs_mean_error_pct"]

    def groupMarkdown(groups: pd.DataFrame) -> str:
        if groups.empty or not set(group_sort_cols).issubset(groups.columns):
            return "_无可计算重复组。当前随机验证窗口数较少，未形成同日期同流量点至少 3 个窗口的组。_"
        return markdownTable(groups.sort_values(group_sort_cols, ascending=[True, False, False]), max_rows=12)

    std_rows = metrics[
        (metrics["model_id"].eq(best["model_id"]))
        & (
            metrics["operation"].isin(
                ["full_fit_all", "random_split_train", "random_split_validation", "group_holdout_validation"]
            )
        )
    ][
        [
            "operation",
            "window_count",
            "mae_pct",
            "mean_error_pct",
            "group_pass",
            "max_abs_group_mean_pct",
            "max_group_sd_pct",
            "u_nor_L_pct",
            "u_nor_r_pct",
            "u_nor_d_pct",
            "pass_all_standards",
        ]
    ]
    top_cols = [
        "model_id",
        "family",
        "validation_fail_count",
        "full_fail_count",
        "validation_mae_pct",
        "validation_group_pass",
        "validation_max_abs_group_mean_pct",
        "validation_max_group_sd_pct",
        "validation_u_nor_L_pct",
        "validation_u_nor_r_pct",
        "validation_u_nor_d_pct",
        "full_mae_pct",
        "full_group_pass",
        "rank_score",
    ]
    full_cols = [
        "model_id",
        "family",
        "standard_fail_count",
        "pass_all_standards",
        "mae_pct",
        "group_pass",
        "max_abs_group_mean_pct",
        "max_group_sd_pct",
        "u_nor_L_pct",
        "u_nor_r_pct",
        "u_nor_d_pct",
    ]
    family_cols = [
        "family",
        "model_id",
        "validation_fail_count",
        "validation_mae_pct",
        "validation_group_pass",
        "validation_u_nor_L_pct",
        "validation_u_nor_r_pct",
        "validation_u_nor_d_pct",
        "full_mae_pct",
    ]
    pressure_cols = [
        "model_id",
        "family",
        "pressure_fail_count",
        "full_fail_count",
        "pressure_mae_pct",
        "pressure_group_pass",
        "pressure_max_abs_group_mean_pct",
        "pressure_max_group_sd_pct",
        "pressure_u_nor_L_pct",
        "pressure_u_nor_r_pct",
        "pressure_u_nor_d_pct",
```

```
        "full_mae_pct",
        "full_group_pass",
        "rank_score",
    ]
    report = f"""# 最终问题建模理论准备与组合模型排名

数据窗口数：`{len(data)}`；主验证采用 `(date, flow_point, disturbance_id)` 组内分层划分，目标比例为训练约 `70%`、验证约 `30%`；同时保留整组留出压力测试；划分种子：`{RANDOM_SEED}`；ANN 训练轮数：`{MLP_EPOCHS}`。

{theorySections()}

## 2. 候选组合模型

在问题二“物理积分 + 剖面修正 + 流量点修正曲线”的基础上，本实验增加三类补偿组合：

1. 固定物理积分基准：`Phys6`、`Lagrange`、`OWICS`，作为不训练对照。
2. `物理积分 + 岭回归修正因子`：分别使用 `profile_flow`、`profile_dynamic_zero`、`runtime_full` 三组特征，搜索 `lambda=0.01--100`。
3. `物理积分 + ANN/BP 残差补偿`：使用运行时全特征 `runtime_full`，隐藏层 `8/16/24`，L2 正则 `1e-4/1e-3/1e-2`，学习率 `0.003/0.01`，每个候选训练 `{MLP_EPOCHS}` 轮，并测试残差收缩系数 `0.50/0.75/1.00`。

所有模型输入均不包含 `date`、`window_id`、`standard_volume_m3`、`standard_rate_m3h`、`condition_note`、`disturbance_id`、`disturbance_angle_deg`。这些字段只用于离线划分和评价。

## 3. 训练操作

### 3.1 全量拟合

每个候选模型用全部 `{len(data)}` 个窗口估计参数，再在全部窗口上评价。该结果反映模型能否充分拟合已有标定数据，但不能单独代表泛化能力。

### 3.2 组内分层验证

主验证划分如下：

{markdownTable(split_counts)}

本次主验证按 `(date, flow_point, disturbance_id)` 分层：对 `n >= 5` 的重复组固定抽取 2 个窗口进入训练集，其余窗口进入验证集；对 `n = 3` 或 `4` 的重复组整组放入训练或验证，不再强拆；同时保留 DO 无扰流锚点以支持 `u_nor,L` 和 `u_nor,d` 计算。

每个候选模型只在训练集拟合参数，并分别输出训练集和验证集指标。最终排名以验证集达标情况为主，全量拟合结果为辅。

### 3.3 整组留出压力测试

压力测试划分如下：

{markdownTable(pressure_split_counts)}

压力测试把完整的 `(date, flow_point, disturbance_id)` 重复组直接留出，用来观察模型对未参与训练的重复工况是否仍然稳定；该结果比组内分层验证更严格，但也更容易暴露跨工况外推不足。

## 4. 综合排名结果

排名规则：优先减少验证集达标失败项，其次考虑验证集超过阈值的归一化差距、全量拟合达标情况、验证 MAE 和过拟合差距。

### 4.1 前 15 个候选模型

{markdownTable(ranking[top_cols], max_rows=15)}

### 4.2 全量拟合提交候选排名

全量拟合用于最终提交候选选择。若只追求训练集指标，ANN 容易非常低误差；因此本表只作为“全量标定后能达到什么水平”的证据，不能替代随机验证。

{markdownTable(full_fit_ranking[full_cols], max_rows=12)}

### 4.3 各模型族最佳方案

{markdownTable(family_ranking[family_cols], max_rows=12)}

### 4.4 整组留出压力测试排名

{markdownTable(pressure_ranking[pressure_cols], max_rows=12) if not pressure_ranking.empty else "_无压力测试结果_"}

## 5. 最优模型详细指标

综合最优模型为：

```text
{best['model_id']}
```

其两个训练操作和验证结果为：

{markdownTable(std_rows)}

最优模型验证集分组表现如下：

{groupMarkdown(best_groups)}

全量拟合提交候选的最佳模型为：

```text
{best_full['model_id']}
```

该模型在全部窗口上的分组表现如下：

```

```
{groupMarkdown(best_full_groups)}

6. 与达标标准的比较

- `|eg_bar| <= 0.2%`：看 `max_abs_group_mean_pct` 是否不超过 `0.2`。
- `SD_g <= 0.040%`：看 `max_group_sd_pct` 或 `u_nor,r` 是否不超过 `0.040`。
- `u_nor,L < 0.036%`、`u_nor,r < 0.040%`、`u_nor,d < 0.115%`：按评价脚本公式计算。

从当前随机验证结果看，若 `pass_all_standards=False`，说明模型虽然降低了 MAE，但仍有至少一个稳定性或扰流漂移指标没有达标。最终论文中应把这一点写成“模型改进方向”，而不能只引用全量拟合的训练误差。

7. 输出文件

- `code/final_model_ranking_experiment.py`：本实验脚本。
- `code/outputs/final_model_metrics_long.csv`：所有候选模型在全量、训练、验证三种评价集上的指标。
- `code/outputs/final_model_comprehensive_ranking.csv`：综合排名表。
- `code/outputs/final_model_full_fit_ranking.csv`：全量拟合提交候选排名。
- `code/outputs/final_model_family_ranking.csv`：按模型族汇总的最佳方案。
- `code/outputs/final_model_best_full_submission.csv`：综合最优方案对应的全量拟合预测，可作为最终提交候选。
- `code/outputs/final_model_best_full_pass_submission.csv`：全量拟合达标优先的提交候选。
- `figures/fig_final_model_validation_ranking.pdf`：验证集排名图。
- `figures/fig_final_model_standard_gap.pdf`：最优模型与达标阈值对比图。
"""
    REPORT.write_text(report, encoding="utf-8")

def saveModelMetadata(best_model: FittedModel, best_full_model: FittedModel, ranking: pd.DataFrame, full_fit_ranking: pd.DataFrame) -> None:
    meta = {
        "best_validation_primary_model_id": best_model.spec.model_id,
        "best_full_fit_pass_model_id": best_full_model.spec.model_id,
        "best_validation_primary": best_model.spec.__dict__,
        "best_full_fit_pass": best_full_model.spec.__dict__,
        "correction_clip_pct": CORRECTION_CLIP_PCT,
        "random_seed": RANDOM_SEED,
        "python": sys.version.split()[0],
        "top_rank_score": float(ranking.iloc[0]["rank_score"]),
        "top_full_fit_score": float(full_fit_ranking.iloc[0]["standard_gap_score"]),
    }
    (OUT_DIR / "final_model_best_meta.json").write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")

def main() -> None:
    ensureDirs()
    data = loadData()
    train, validation, split = makeRandomSplit(data)
    pressure_train, pressure_validation, pressureSplit = makeGroupHoldoutPressureSplit(data)
    specs = buildSpecs()

    full_metrics, full_models, full_groups, full_windows = runSpecs(specs, data, {"full_fit_all": data})
    split_metrics, split_models, split_groups, split_windows = runSpecs(
        specs,
        train,
        {"random_split_train": train, "random_split_validation": validation},
)
    pressure_metrics, pressure_models, pressure_groups, pressure_windows = runSpecs(
        specs,
        pressure_train,
        {"group_holdout_train": pressure_train, "group_holdout_validation": pressure_validation},
)
    metrics = pd.concat([full_metrics, split_metrics, pressure_metrics], ignore_index=True)
    ranking = buildComprehensiveRanking(metrics)
    pressure_ranking = buildPressureRanking(metrics)
    full_fit_ranking = buildFullFitRanking(metrics)
    family_ranking = bestByFamily(ranking)

    best_id = str(ranking.iloc[0]["model_id"])
    best_full_id = str(full_fit_ranking.iloc[0]["model_id"])
    pressure_best_id = str(pressure_ranking.iloc[0]["model_id"]) if not pressure_ranking.empty else ""
    best_full_model = full_models[best_id]
    best_full_pass_model = full_models[best_full_id]
    best_full_windows = full_windows[("full_fit_all", best_id)]
    best_full_pass_windows = full_windows[("full_fit_all", best_full_id)]
    best_full_pass_groups = full_groups[("full_fit_all", best_full_id)]
    best_validation_groups = split_groups[("random_split_validation", best_id)]
    best_validation_windows = split_windows[("random_split_validation", best_id)]
    pressure_best_groups = (
        pressure_groups[("group_holdout_validation", pressure_best_id)] if pressure_best_id else pd.DataFrame()
)
    pressure_best_windows = (
        pressure_windows[("group_holdout_validation", pressure_best_id)] if pressure_best_id else pd.DataFrame()
)

    metrics.to_csv(OUT_DIR / "final_model_metrics_long.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    ranking.to_csv(OUT_DIR / "final_model_comprehensive_ranking.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    pressure_ranking.to_csv(OUT_DIR / "final_model_group_holdout_pressure_ranking.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    full_fit_ranking.to_csv(OUT_DIR / "final_model_full_fit_ranking.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    family_ranking.to_csv(OUT_DIR / "final_model_family_ranking.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    split.to_csv(OUT_DIR / "final_model_random_split_assignments.csv", index=False, encoding="utf-8-sig")
    pressureSplit.to_csv(OUT_DIR / "final_model_group_holdout_pressure_split_assignments.csv", index=False, encoding="utf-8-sig")
    best_validation_groups.to_csv(OUT_DIR / "final_model_best_validation_groups.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    best_validation_windows.to_csv(OUT_DIR / "final_model_best_validation_windows.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    if not pressure_best_groups.empty:
        pressure_best_groups.to_csv(OUT_DIR / "final_model_best_pressure_groups.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
        pressure_best_windows.to_csv(OUT_DIR / "final_model_best_pressure_windows.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
```

```
    best_full_windows.to_csv(OUT_DIR / "final_model_best_full_windows.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    best_full_pass_groups.to_csv(OUT_DIR / "final_model_best_full_pass_groups.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    best_full_pass_windows.to_csv(OUT_DIR / "final_model_best_full_pass_windows.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    best_full_windows[["window_id", "model_volume_m3"]].to_csv(
        OUT_DIR / "final_model_best_full_submission.csv",
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
    )
    best_full_pass_windows[["window_id", "model_volume_m3"]].to_csv(
        OUT_DIR / "final_model_best_full_pass_submission.csv",
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
    )

    drawRankingFigure(FIG_DIR / "fig_final_model_validation_ranking.pdf", ranking)
    drawStandardGapFigure(FIG_DIR / "fig_final_model_standard_gap.pdf", ranking.iloc[0], metrics)
    drawStandardGapFigure(FIG_DIR / "fig_final_model_full_pass_standard_gap.pdf", full_fit_ranking.iloc[0], metrics)
    writeReport(
        data,
        split,
        pressureSplit,
        metrics,
        ranking,
        pressure_ranking,
        family_ranking,
        full_fit_ranking,
        best_validation_groups,
        best_full_pass_groups,
    )
    saveModelMetadata(best_full_model, best_full_pass_model, ranking, full_fit_ranking)

    print("Best model:", best_id)
    print("Best full-fit pass candidate:", best_full_id)
    if pressure_best_id:
        print("Best group-holdout pressure model:", pressure_best_id)
    print("Family ranking:")
    print(
        family_ranking[
            [
                "family",
                "model_id",
                "validation_fail_count",
                "validation_mae_pct",
                "validation_group_pass",
                "validation_u_nor_L_pct",
                "validation_u_nor_r_pct",
                "validation_u_nor_d_pct",
                "full_mae_pct",
            ]
        ]
        .head(8)
        .to_string(index=False)
    )
    print("Report:", REPORT)


if __name__ == "__main__":
    main()
```

B.6 物理特征公共模块：physical_feature_library.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题四公共模块：构造紧凑物理特征与稳定性代理。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

import math
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

from model_evolution_ridge_ann import (
    FIG_DIR,
    OUT_DIR,
    RANDOM_SEED,
    FittedModel,
    ModelSpec,
    Standardizer,
    buildFeatureFrame,
    evaluatePredictions,
    featureColumns,
    loadData,
    makeRandomSplit,
    standardsGap,
)
```

```

try:
    from reportlab.lib import colors
    from reportlab.lib.pagesizes import A4, landscape
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    from reportlab.pdfgen import canvas
except ModuleNotFoundError:
    colors = None
    A4 = None
    landscape = None
    pdfmetrics = None
    TTFont = None
    canvas = None


ROOT = Path(__file__).resolve().parents[2]
REPORT = Path(__file__).resolve().parent / "verification_reports" / "GENERALIZATION_IMPROVEMENT_EXPERIMENT_REPORT.md"
CORRECTION_CLIP_PCT = 2.0


@dataclass(frozen=True)
class ImproveSpec:
    model_id: str
    improvement_group: str
    improvement_items: str
    base_col: str = "lagrange_volume_m3"
    feature_set: str = "proxy_interaction"
    hidden_size: int = 12
    l2: float = 1e-3
    learning_rate: float = 0.003
    epochs: int = 8000
    seed_offset: int = 0
    shrinkage: float = 1.0
    w_mse: float = 1.0
    w_group_mean: float = 0.0
    w_group_sd: float = 0.0
    w_linearity: float = 0.0
    w_drift: float = 0.0
    w_ud_sd: float = 0.0
    w_smooth_flow: float = 0.0
    w_ridge_input: float = 0.0


def enhancedFeatureFrame(data: pd.DataFrame) -> pd.DataFrame:
    feats = buildFeatureFrame(data).copy()
    eps = 1e-12
    q = feats["q"].astype(float)

    profile_bulge = feats.get("chord_center_minus_edges_frac", 0.0)
    edge_gradient = feats.get("chord_edge_minus_inner_frac", 0.0)
    top_bottom = feats.get("chord_top_bottom_frac", 0.0)
    inner_skew = feats.get("chord_inner_skew_frac", 0.0)
    ab_abs = feats.get("ab_abs_mean", feats.get("profile_ab_abs", 0.0))
    ab_range = feats.get("ab_range", 0.0)
    dyn_cv = feats.get("dyn_plateau_cv", 0.0)
    dyn_active = feats.get("dyn_active_eq_s", 0.0) / np.maximum(data["duration_s"].astype(float), eps)
    zero_med = feats.get("zero_rate_med", 0.0)
    zero_mad = feats.get("zero_rate_mad", 0.0)

    feats["bias"] = 1.0
    feats["proxy_profile_bulge"] = profile_bulge
    feats["proxy_edge_gradient"] = edge_gradient
    feats["proxy_asymmetry"] = 0.5 * top_bottom + 0.5 * inner_skew
    feats["proxy_ab_direction"] = 0.5 * ab_abs + 0.5 * ab_range
    feats["proxy_dynamic"] = dyn_cv + 0.2 * (1.0 - dyn_active)
    feats["proxy_zero"] = zero_med + zero_mad
    feats["proxy_integral_disagreement"] = (
        feats.get("lagrange_volume_m3_minus_phys6_frac", 0.0).abs()
        + feats.get("owics_volume_m3_minus_phys6_frac", 0.0).abs()
    )
    proxy_cols = [
        "proxy_profile_bulge",
        "proxy_edge_gradient",
        "proxy_asymmetry",
        "proxy_ab_direction",
        "proxy_dynamic",
        "proxy_zero",
        "proxy_integral_disagreement",
    ]
    for col in proxy_cols:
        feats[f"{col}_x_q"] = feats[col].astype(float) * q
        feats[f"{col}_x_q2"] = feats[col].astype(float) * feats["q2"].astype(float)
    return feats.replace([np.inf, -np.inf], np.nan).fillna(0.0)


def selectColumns(frame: pd.DataFrame, feature_set: str) -> list[str]:
    if feature_set == "runtime_full":
        cols = featureColumns(frame, "runtime_full")
        return ["bias"] + [c for c in cols if c != "bias"]
    proxy = [
        "bias",
        "q",
```

```
        "q2",
        "proxy_profile_bulge",
        "proxy_edge_gradient",
        "proxy_asymmetry",
        "proxy_ab_direction",
        "proxy_dynamic",
        "proxy_zero",
        "proxy_integral_disagreement",
    ]
    if feature_set == "proxy_factor":
        return [c for c in proxy if c in frame.columns]
    if feature_set == "proxy_interaction":
        interactions = [c for c in frame.columns if c.startswith("proxy_") and (c.endswith("_x_q") or c.endswith("_x_q2"))]
        return [c for c in proxy + interactions if c in frame.columns]
    if feature_set == "smooth_flow":
        cols = proxy + ["q3", "profile_top_bottom_x_q", "profile_center_all_x_q", "profile_swirl_x_q"]
        return [c for c in cols if c in frame.columns]
    raise ValueError(feature_set)


def fitStandardizer(frame: pd.DataFrame, cols: list[str]) -> Standardizer:
    clean = frame[cols].replace([np.inf, -np.inf], np.nan).copy()
    median = clean.median(numeric_only=True).fillna(0.0)
    clean = clean.fillna(median)
    arr = clean.to_numpy(dtype=float)
    mean = arr.mean(axis=0)
    scale = arr.std(axis=0)
    scale[scale < 1e-12] = 1.0
    if "bias" in cols:
        idx = cols.index("bias")
        median.iloc[idx] = 1.0
        mean[idx] = 0.0
        scale[idx] = 1.0
    return Standardizer(cols, median, mean, scale)


def makeGroupIndices(data: pd.DataFrame) -> dict[str, list[np.ndarray]]:
    groups = []
    for _, g in data.groupby(["date", "flow_point"], sort=False):
        if len(g) >= 3:
            groups.append(g.index.to_numpy())

    d0_flows = []
    for _, g in data[data["condition_note"].eq("no_disturbance_reference")].groupby("flow_point", sort=False):
        if len(g) >= 1:
            d0_flows.append(g.index.to_numpy())

    drift_pairs = []
    for flow, base in data[data["condition_note"].eq("no_disturbance_reference")].groupby("flow_point", sort=False):
        bidx = base.index.to_numpy()
        disturbed = data[(data["condition_note"].eq("disturbed_test")) & (data["flow_point"].eq(flow))]
        for _, dg in disturbed.groupby("disturbance_id", sort=False):
            if len(dg) >= 1 and len(bidx) >= 1:
                drift_pairs.append((bidx, dg.index.to_numpy()))

    disturbance_flow_groups = []
    disturbed_official = data[(data["condition_note"].eq("disturbed_test")) & (data["flow_point"].between(40, 100))]
    for _, g in disturbed_official.groupby(["disturbance_id", "flow_point"], sort=False):
        if len(g) >= 2:
            disturbance_flow_groups.append(g.index.to_numpy())

    flow_groups = []
    for _, g in data.groupby("flow_point", sort=True):
        flow_groups.append(g.index.to_numpy())

    return {
        "groups": groups,
        "d0_flows": d0_flows,
        "drift_pairs": drift_pairs,
        "disturbance_flow_groups": disturbance_flow_groups,
        "flow_groups": flow_groups,
    }


def errorAndGradP(data: pd.DataFrame, p: np.ndarray, spec: ImproveSpec, group_info: dict[str, list[np.ndarray]]) -> tuple[float, np.ndarray]:
    base = data[spec.base_col].to_numpy(dtype=float)
    ref = data["standard_volume_m3"].to_numpy(dtype=float)
    ratio = base / ref
    e = (base * (1.0 + p / 100.0) - ref) / ref * 100.0
    grad_e = np.zeros_like(e)
    loss = 0.0

    if spec.w_mse:
        loss += spec.w_mse * float(np.mean(e**2))
        grad_e += spec.w_mse * (2.0 / len(e)) * e

    groups = group_info["groups"]
    if groups and (spec.w_group_mean or spec.w_group_sd):
        g_count = len(groups)
        for idx in groups:
            values = e[idx]
            mean = float(values.mean())
            if spec.w_group_mean:
```

```
                loss += spec.w_group_mean * mean * mean / g_count
                grad_e[idx] += spec.w_group_mean * 2.0 * mean / (g_count * len(idx))
            if spec.w_group_sd and len(idx) > 1:
                centered = values - mean
                var = float(np.sum(centered**2) / (len(idx) - 1))
                loss += spec.w_group_sd * var / g_count
                grad_e[idx] += spec.w_group_sd * 2.0 * centered / (g_count * (len(idx) - 1))

    d0_flows = group_info["d0_flows"]
    if d0_flows and spec.w_linearity:
        for idx in d0_flows:
            mean = float(e[idx].mean())
            loss += spec.w_linearity * mean * mean / len(d0_flows)
            grad_e[idx] += spec.w_linearity * 2.0 * mean / (len(d0_flows) * len(idx))

    drift_pairs = group_info["drift_pairs"]
    if drift_pairs and spec.w_drift:
        for bidx, didx in drift_pairs:
            diff = float(e[bidx].mean() - e[didx].mean())
            loss += spec.w_drift * diff * diff / len(drift_pairs)
            grad_e[bidx] += spec.w_drift * 2.0 * diff / (len(drift_pairs) * len(bidx))
            grad_e[didx] -= spec.w_drift * 2.0 * diff / (len(drift_pairs) * len(didx))

    disturbance_flow_groups = group_info["disturbance_flow_groups"]
    if disturbance_flow_groups and spec.w_ud_sd:
        g_count = len(disturbance_flow_groups)
        for idx in disturbance_flow_groups:
            values = e[idx]
            centered = values - float(values.mean())
            var = float(np.sum(centered**2) / (len(idx) - 1))
            loss += spec.w_ud_sd * var / g_count
            grad_e[idx] += spec.w_ud_sd * 2.0 * centered / (g_count * (len(idx) - 1))

    grad_p = grad_e * ratio

    flow_groups = group_info["flow_groups"]
    if len(flow_groups) >= 2 and spec.w_smooth_flow:
        means = np.array([p[idx].mean() for idx in flow_groups], dtype=float)
        diffs = np.diff(means)
        loss += spec.w_smooth_flow * float(np.mean(diffs**2))
        grad_means = np.zeros_like(means)
        denom = max(1, len(diffs))
        for i, d in enumerate(diffs):
            grad_means[i] -= spec.w_smooth_flow * 2.0 * d / denom
            grad_means[i + 1] += spec.w_smooth_flow * 2.0 * d / denom
        for gidx, idx in enumerate(flow_groups):
            grad_p[idx] += grad_means[gidx] / len(idx)

    return loss, grad_p


def trainMetricMlp(spec: ImproveSpec, train: pd.DataFrame) -> FittedModel:
    feats = enhancedFeatureFrame(train)
    cols = selectColumns(feats, spec.feature_set)
    std = fitStandardizer(feats, cols)
    x = std.transform(feats[cols])
    group_info = makeGroupIndices(train)
    rng = np.random.default_rng(RANDOM_SEED + spec.seed_offset + spec.hidden_size * 37)
    n, p_dim = x.shape
    w1 = rng.normal(0.0, math.sqrt(2.0 / max(1, p_dim)), size=(p_dim, spec.hidden_size))
    b1 = np.zeros(spec.hidden_size)
    w2 = rng.normal(0.0, math.sqrt(2.0 / max(1, spec.hidden_size)), size=spec.hidden_size)
    b2 = 0.0

    mw1 = np.zeros_like(w1)
    vw1 = np.zeros_like(w1)
    mb1 = np.zeros_like(b1)
    vb1 = np.zeros_like(b1)
    mw2 = np.zeros_like(w2)
    vw2 = np.zeros_like(w2)
    mb2 = 0.0
    vb2 = 0.0
    beta1 = 0.9
    beta2 = 0.999
    eps = 1e-8

    for epoch in range(1, spec.epochs + 1):
        hidden = np.tanh(x @ w1 + b1)
        pred = hidden @ w2 + b2
        _, grad_p = errorAndGradP(train, pred, spec, group_info)
        grad_w2 = hidden.T @ grad_p + 2.0 * spec.l2 * w2
        grad_b2 = float(grad_p.sum())
        dh = np.outer(grad_p, w2) * (1.0 - hidden**2)
        grad_w1 = x.T @ dh + 2.0 * spec.l2 * w1
        grad_b1 = dh.sum(axis=0)
        if spec.w_ridge_input:
            grad_w1 += 2.0 * spec.w_ridge_input * w1

        mw1 = beta1 * mw1 + (1.0 - beta1) * grad_w1
        vw1 = beta2 * vw1 + (1.0 - beta2) * (grad_w1**2)
        mb1 = beta1 * mb1 + (1.0 - beta1) * grad_b1
        vb1 = beta2 * vb1 + (1.0 - beta2) * (grad_b1**2)
        mw2 = beta1 * mw2 + (1.0 - beta1) * grad_w2
```

```
        vw2 = beta2 * vw2 + (1.0 - beta2) * (grad_w2**2)
        mb2 = beta1 * mb2 + (1.0 - beta1) * grad_b2
        vb2 = beta2 * vb2 + (1.0 - beta2) * (grad_b2**2)
        bc1 = 1.0 - beta1**epoch
        bc2 = 1.0 - beta2**epoch
        w1 -= spec.learning_rate * (mw1 / bc1) / (np.sqrt(vw1 / bc2) + eps)
        b1 -= spec.learning_rate * (mb1 / bc1) / (np.sqrt(vb1 / bc2) + eps)
        w2 -= spec.learning_rate * (mw2 / bc1) / (np.sqrt(vw2 / bc2) + eps)
        b2 -= spec.learning_rate * (mb2 / bc1) / (math.sqrt(vb2 / bc2) + eps)

    model_spec = ModelSpec(
        model_id=spec.model_id,
        family=spec.improvement_group,
        base_col=spec.base_col,
        feature_set=spec.feature_set,
        model_type="mlp",
        hidden_size=spec.hidden_size,
        l2=spec.l2,
        learning_rate=spec.learning_rate,
        seed_offset=spec.seed_offset,
        shrinkage=spec.shrinkage,
        epochs=spec.epochs,
    )
    return FittedModel(spec=model_spec, standardizer=std, w1=w1, b1=b1, w2=w2, b2=b2)


def evaluateModel(model: FittedModel, frame: pd.DataFrame) -> tuple[pd.Series, pd.DataFrame, pd.DataFrame]:
    feats = enhancedFeatureFrame(frame)
    pred = model.predictVolume(frame, feats)
    return evaluatePredictions(frame, pred)


def buildSpecs() -> list[ImproveSpec]:
    common = dict(base_col="lagrange_volume_m3", hidden_size=16, l2=1e-3, learning_rate=0.003, epochs=8000, seed_offset=17)
    return [
        ImproveSpec(
            model_id="single_metric_loss_runtime",
            improvement_group="single",
            improvement_items="metric_loss",
            feature_set="runtime_full",
            w_mse=1.0,
            w_group_mean=0.8,
            w_group_sd=1.5,
            w_linearity=0.8,
            w_drift=0.8,
            **common,
        ),
        ImproveSpec(
            model_id="single_proxy_factor_mse",
            improvement_group="single",
            improvement_items="proxy_factor",
            feature_set="proxy_factor",
            w_mse=1.0,
            hidden_size=10,
            epochs=8000,
            seed_offset=11,
        ),
        ImproveSpec(
            model_id="single_flow_smooth_proxy",
            improvement_group="single",
            improvement_items="smooth_flow",
            feature_set="smooth_flow",
            w_mse=1.0,
            w_smooth_flow=0.8,
            hidden_size=12,
            epochs=8000,
            seed_offset=13,
        ),
        ImproveSpec(
            model_id="single_drift_loss_proxy",
            improvement_group="single",
            improvement_items="drift_loss",
            feature_set="proxy_interaction",
            w_mse=1.0,
            w_drift=1.5,
            hidden_size=12,
            epochs=8000,
            seed_offset=15,
        ),
        ImproveSpec(
            model_id="single_ud_sd_proxy",
            improvement_group="single",
            improvement_items="disturbance_flow_sd",
            feature_set="proxy_interaction",
            w_mse=1.0,
            w_ud_sd=2.0,
            hidden_size=12,
            l2=2e-3,
            epochs=9000,
            seed_offset=31,
        ),
        ImproveSpec(
            model_id="combined_metric_proxy",
```

```
            improvement_group="combined",
            improvement_items="metric_loss+proxy_factor",
            feature_set="proxy_interaction",
            w_mse=1.0,
            w_group_mean=0.8,
            w_group_sd=1.2,
            w_linearity=0.8,
            w_drift=0.8,
            hidden_size=12,
            epochs=9000,
            seed_offset=17,
        ),
        ImproveSpec(
            model_id="combined_metric_udsd_proxy",
            improvement_group="combined",
            improvement_items="metric_loss+proxy_factor+disturbance_flow_sd",
            feature_set="proxy_interaction",
            w_mse=1.0,
            w_group_mean=0.8,
            w_group_sd=1.2,
            w_linearity=1.0,
            w_drift=0.8,
            w_ud_sd=2.0,
            hidden_size=12,
            l2=2e-3,
            epochs=9000,
            seed_offset=33,
        ),
        ImproveSpec(
            model_id="combined_metric_smooth_proxy",
            improvement_group="combined",
            improvement_items="metric_loss+proxy_factor+smooth_flow",
            feature_set="proxy_interaction",
            w_mse=1.0,
            w_group_mean=0.8,
            w_group_sd=1.2,
            w_linearity=0.8,
            w_drift=0.8,
            w_smooth_flow=0.6,
            hidden_size=12,
            epochs=9000,
            seed_offset=19,
        ),
        ImproveSpec(
            model_id="combined_metric_smooth_udsd_proxy",
            improvement_group="combined",
            improvement_items="metric_loss+proxy_factor+smooth_flow+disturbance_flow_sd",
            feature_set="proxy_interaction",
            w_mse=1.0,
            w_group_mean=0.8,
            w_group_sd=1.2,
            w_linearity=1.0,
            w_drift=1.0,
            w_ud_sd=2.5,
            w_smooth_flow=0.6,
            hidden_size=12,
            l2=2e-3,
            epochs=9000,
            seed_offset=35,
        ),
        ImproveSpec(
            model_id="combined_metric_runtime_smooth",
            improvement_group="combined",
            improvement_items="metric_loss+runtime_full+smooth_flow",
            feature_set="runtime_full",
            w_mse=1.0,
            w_group_mean=0.8,
            w_group_sd=1.2,
            w_linearity=0.8,
            w_drift=0.8,
            w_smooth_flow=0.5,
            w_ridge_input=1e-5,
            **common,
        ),
        ImproveSpec(
            model_id="combined_metric_runtime_udsd",
            improvement_group="combined",
            improvement_items="metric_loss+runtime_full+disturbance_flow_sd",
            feature_set="runtime_full",
            w_mse=0.9,
            w_group_mean=0.8,
            w_group_sd=1.5,
            w_linearity=1.2,
            w_drift=1.2,
            w_ud_sd=2.5,
            hidden_size=16,
            l2=2e-3,
            learning_rate=0.003,
            epochs=9000,
            seed_offset=37,
        ),
        ImproveSpec(
            model_id="combined_balanced_runtime_official",
```

```
            improvement_group="combined",
            improvement_items="runtime_full+official_metric_losses+smooth_flow",
            feature_set="runtime_full",
            w_mse=0.8,
            w_group_mean=0.8,
            w_group_sd=2.0,
            w_linearity=1.8,
            w_drift=2.5,
            w_ud_sd=3.0,
            w_smooth_flow=0.4,
            hidden_size=16,
            l2=3e-3,
            learning_rate=0.003,
            epochs=10000,
            seed_offset=39,
        ),
        ImproveSpec(
            model_id="combined_strong_repeatability",
            improvement_group="combined",
            improvement_items="proxy_factor+strong_group_sd+drift",
            feature_set="proxy_interaction",
            w_mse=0.8,
            w_group_mean=0.8,
            w_group_sd=3.0,
            w_linearity=0.8,
            w_drift=1.0,
            w_smooth_flow=0.4,
            hidden_size=12,
            epochs=9000,
            seed_offset=23,
        ),
        ImproveSpec(
            model_id="combined_strong_drift",
            improvement_group="combined",
            improvement_items="proxy_factor+strong_drift+smooth_flow",
            feature_set="proxy_interaction",
            w_mse=0.8,
            w_group_mean=0.6,
            w_group_sd=1.0,
            w_linearity=1.2,
            w_drift=3.0,
            w_smooth_flow=0.8,
            hidden_size=12,
            epochs=9000,
            seed_offset=29,
        ),
    ]


def run() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    REPORT.parent.mkdir(parents=True, exist_ok=True)
    data = loadData()
    train, validation, split = makeRandomSplit(data)
    split.to_csv(OUT_DIR / "generalization_improvement_split.csv", index=False, encoding="utf-8-sig")
    specs = buildSpecs()
    rows = []
    groups_by_key: dict[tuple[str, str], pd.DataFrame] = {}
    windows_by_key: dict[tuple[str, str], pd.DataFrame] = {}

    for train_name, train_frame, evals in [
        ("full", data, {"full_fit_all": data}),
        ("random", train, {"random_split_train": train, "random_split_validation": validation}),
    ]:
        for spec in specs:
            model = trainMetricMlp(spec, train_frame)
            for operation, frame in evals.items():
                summary, groups, windows = evaluateModel(model, frame)
                fail_count, gap = standardsGap(summary)
                row = {
                    "model_id": spec.model_id,
                    "improvement_group": spec.improvement_group,
                    "improvement_items": spec.improvement_items,
                    "operation": operation,
                    "base_col": spec.base_col,
                    "feature_set": spec.feature_set,
                    "hidden_size": spec.hidden_size,
                    "l2": spec.l2,
                    "learning_rate": spec.learning_rate,
                    "epochs": spec.epochs,
                    "shrinkage": spec.shrinkage,
                    "w_mse": spec.w_mse,
                    "w_group_mean": spec.w_group_mean,
                    "w_group_sd": spec.w_group_sd,
                    "w_linearity": spec.w_linearity,
                    "w_drift": spec.w_drift,
                    "w_ud_sd": spec.w_ud_sd,
                    "w_smooth_flow": spec.w_smooth_flow,
                    "standard_fail_count": fail_count,
                    "standard_gap_score": gap,
                }
                row.update(summary.to_dict())
```

```
                rows.append(row)
                groups_by_key[(operation, spec.model_id)] = groups
                windows_by_key[(operation, spec.model_id)] = windows

    metrics = pd.DataFrame(rows)
    metrics.to_csv(OUT_DIR / "generalization_improvement_metrics_long.csv", index=False, encoding="utf-8-sig")
    comparison = buildComparison(metrics)
    comparison.to_csv(OUT_DIR / "generalization_improvement_comparison.csv", index=False, encoding="utf-8-sig")
    if not comparison.empty:
        best_id = str(comparison.iloc[0]["model_id"])
        for operation in ["full_fit_all", "random_split_train", "random_split_validation"]:
            key = (operation, best_id)
            if key in groups_by_key:
                groups_by_key[key].to_csv(OUT_DIR / f"generalization_improvement_best_{operation}_groups.csv", index=False, encoding="utf-8-sig")
                windows_by_key[key].to_csv(OUT_DIR / f"generalization_improvement_best_{operation}_windows.csv", index=False, encoding="utf-8-sig")
    writeReport(comparison)
    makeFigure(comparison)
    print(comparison.head(12).to_string(index=False))


def buildComparison(metrics: pd.DataFrame) -> pd.DataFrame:
    full = metrics[metrics["operation"].eq("full_fit_all")].set_index("model_id")
    train = metrics[metrics["operation"].eq("random_split_train")].set_index("model_id")
    valid = metrics[metrics["operation"].eq("random_split_validation")].set_index("model_id")
    rows = []
    for model_id, v in valid.iterrows():
        if model_id not in full.index:
            continue
        f = full.loc[model_id]
        t = train.loc[model_id] if model_id in train.index else v
        overfit = max(0.0, float(v["mae_pct"]) - float(t["mae_pct"]))
        rows.append(
            {
                "model_id": model_id,
                "improvement_group": v["improvement_group"],
                "improvement_items": v["improvement_items"],
                "validation_fail_count": int(v["standard_fail_count"]),
                "full_fail_count": int(f["standard_fail_count"]),
                "validation_pass_all": bool(v["pass_all_standards"]),
                "full_pass_all": bool(f["pass_all_standards"]),
                "validation_mae_pct": float(v["mae_pct"]),
                "validation_group_pass": v["group_pass"],
                "validation_max_abs_group_mean_pct": float(v["max_abs_group_mean_pct"]),
                "validation_max_group_sd_pct": float(v["max_group_sd_pct"]),
                "validation_u_nor_L_pct": float(v["u_nor_L_pct"]),
                "validation_u_nor_r_pct": float(v["u_nor_r_pct"]),
                "validation_u_nor_d_pct": float(v["u_nor_d_pct"]),
                "train_mae_pct": float(t["mae_pct"]),
                "full_mae_pct": float(f["mae_pct"]),
                "full_group_pass": f["group_pass"],
                "full_u_nor_L_pct": float(f["u_nor_L_pct"]),
                "full_u_nor_r_pct": float(f["u_nor_r_pct"]),
                "full_u_nor_d_pct": float(f["u_nor_d_pct"]),
                "overfit_mae_gap": overfit,
            }
        )
    out = pd.DataFrame(rows)
    if out.empty:
        return out
    out["rank_score"] = (
        1000.0 * out["validation_fail_count"]
        + 250.0 * out["full_fail_count"]
        + 20.0 * out["overfit_mae_gap"]
        + out["validation_mae_pct"]
    )
    return out.sort_values(["rank_score", "validation_mae_pct"]).reset_index(drop=True)


def mdTable(df: pd.DataFrame, cols: list[str], max_rows: int = 12) -> str:
    view = df[cols].head(max_rows).copy()
    if view.empty:
        return "_无数据_"
    for col in view.columns:
        if pd.api.types.is_float_dtype(view[col]):
            view[col] = view[col].map(lambda x: "" if pd.isna(x) else f"{float(x):.6g}")
        else:
            view[col] = view[col].map(lambda x: "" if pd.isna(x) else str(x))
    header = "| " + " | ".join(view.columns) + " |"
    sep = "| " + " | ".join(["---"] * len(view.columns)) + " |"
    body = ["| " + " | ".join(row) + " |" for row in view.astype(str).to_numpy()]
    return "\n".join([header, sep] + body)


def writeReport(comparison: pd.DataFrame) -> None:
    previous = pd.read_csv(OUT_DIR / "final_model_comprehensive_ranking.csv", encoding="utf-8-sig")
    prev_cols = [
        "model_id",
        "validation_fail_count",
        "full_fail_count",
        "validation_mae_pct",
        "validation_u_nor_L_pct",
        "validation_u_nor_r_pct",
        "validation_u_nor_d_pct",
```

```
        "full_mae_pct",
        "full_group_pass",
    ]
    cols = [
        "model_id",
        "improvement_group",
        "improvement_items",
        "validation_fail_count",
        "full_fail_count",
        "validation_mae_pct",
        "validation_group_pass",
        "validation_u_nor_L_pct",
        "validation_u_nor_r_pct",
        "validation_u_nor_d_pct",
        "full_mae_pct",
        "full_group_pass",
    ]
    lines = [
        "# 泛化指标改进组实验",
        "",
        "## 实验设计",
        "",
        "本实验从论文库可迁移方法出发，设置单项改进和联合改进两类候选。",
        "单项改进包括：指标驱动损失、低维扰流代理因子、平滑流量点修正、扰流漂移损失、扰流-流量点组内标准差损失。",
        "联合改进包括：指标驱动损失 + 扰流代理因子、再叠加平滑流量点修正、扰流-流量点组内标准差或更强重复性/漂移权重。",
        "所有模型均沿用相同的 `(date, flow_point, disturbance_id)` 组内分层验证划分，目标约 70% 训练、30% 验证，评价指标与原实验完全一致。",
        "该划分保留 DO 流量锚点：对 `n >= 5` 的重复组取 2 个窗口训练、其余窗口验证，对 `n = 3` 或 `4` 的重复组整组放入训练或验证，从而保证验证集中可以计算组均值误差、组内 SD 与 `u_nor,r`。",
        "",
        "## 新增改进组排名",
        "",
        mdTable(comparison, cols),
        "",
        "## 原综合排名前五对照",
        "",
        mdTable(previous, prev_cols, max_rows=5),
        "",
        "## 与原综合最优的结论性对比",
        "",
        "新增改进组和原综合排名均采用同一组内分层验证划分，以及同一组达标阈值：",
        "`|eg_bar| <= 0.2%`、`SDg <= 0.040%`、`u_nor,L < 0.036%`、`u_nor,r < 0.040%`、`u_nor,d < 0.115%`。",
        "本次验证集显式保留重复组，因此 `group_pass` 与 `u_nor,r` 不再是 `0/0` 或缺失值。",
        "",
        "## 结论",
        "",
    ]
    if not comparison.empty:
        best = comparison.iloc[0]
        old_best = previous.iloc[0]
        lines.extend(
            [
                f"新增改进组中综合排序最好的是 `{best['model_id']}`，属于 `{best['improvement_items']}`。",
                f"其随机验证 MAE 为 `{best['validation_mae_pct']:.6g}%`，验证失败项数为 `{int(best['validation_fail_count'])}`，全量失败项数为 `{int(best['full_fail_count'])}`。",
                f"原综合最优 `{old_best['model_id']}` 的随机验证 MAE 为 `{float(old_best['validation_mae_pct']):.6g}%`，验证失败项数为 `{int(old_best['validation_fail_count'])}`，全量失败项数为 `{int(old_best['full_fail_count'])}`。",
                "因此新增组在验证 MAE 上有局部改进，但综合达标性没有超过原综合最优；新增组中没有模型在随机验证或全量评价下全部达标。",
                "这说明当前数据下仅靠损失函数和代理特征仍不能完全解决泛化不达标，后续需要更多真实跨日期/跨扰流强度重复组，或使用更贴近官方指标的可微近似目标继续调参。",
            ]
        )
    REPORT.write_text("\n".join(lines), encoding="utf-8")


def makeFigure(comparison: pd.DataFrame) -> None:
    if canvas is None or comparison.empty:
        return
    pdfmetrics.registerFont(TTFont("MSYH", r"C:\Windows\Fonts\msyh.ttc", subfontIndex=0))
    path = FIG_DIR / "fig_generalization_improvement_compare.pdf"
    c = canvas.Canvas(str(path), pagesize=landscape(A4))
    width, height = landscape(A4)
    c.setFont("MSYH", 14)
    c.setFillColor(colors.HexColor("#203040"))
    c.drawString(40, height - 45, "泛化指标改进组随机验证对比")
    view = comparison.head(9).copy()
    max_mae = max(0.1, float(view["validation_mae_pct"].max()))
    left = 185
    top = height - 85
    bar_w = width - 260
    row_h = 38
    palette = {
        "single": colors.HexColor("#8FB7E8"),
        "combined": colors.HexColor("#E3A76F"),
    }
    for i, row in view.iterrows():
        y = top - i * row_h
        label = str(row["model_id"]).replace("combined_", "cmb_").replace("single_", "sgl_")
        c.setFont("MSYH", 8)
        c.setFillColor(colors.HexColor("#506070"))
        c.drawRightString(left - 8, y, label[:36])
        c.setFillColor(palette.get(str(row["improvement_group"]), colors.HexColor("#AAB4C0")))
        w = bar_w * float(row["validation_mae_pct"]) / max_mae
        c.rect(left, y - 10, w, 12, fill=1, stroke=0)
        c.setFillColor(colors.HexColor("#203040"))
        c.drawString(left + w + 5, y - 9, f"{float(row['validation_mae_pct']):.4f}% / fail {int(row['validation_fail_count'])}")
    c.setFont("MSYH", 8)
```

```
    c.setFillColor(colors.HexColor("#667788"))
    c.drawCentredString(width / 2, 28, "validation MAE / %")
    c.save()


if __name__ == "__main__":
    run()
```

B.7 紧凑 RBF-KRR：compact_rbf_krr.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题四泛化模型：拟合紧凑物理特征 RBF-KRR。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

import json
import math
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd

from model_evolution_ridge_ann import (
    GROUP_KEY,
    OUT_DIR,
    RANDOM_SEED,
    STANDARD_LIMITS,
    buildFeatureFrame,
    evaluatePredictions,
    featureColumns,
    loadData,
    standardsGap,
)
from physical_feature_library import enhancedFeatureFrame


ROOT = Path(__file__).resolve().parents[2]
REPORT = Path(__file__).resolve().parent / "verification_reports" / "STRATIFIED_MODEL_SEARCH_REPORT.md"
TARGET_VALIDATION_FRACTION = 0.30
N_SPLITS = 40


@dataclass(frozen=True)
class SplitSpec:
    seed: int
    d0_n3_to_validation: bool
    n34_validation_bias: float
    small_validation_prob: float


@dataclass(frozen=True)
class CandidateSpec:
    model_id: str
    family: str
    base_col: str
    feature_set: str
    model_type: str
    lam: float = 1.0
    gamma: float = 1.0
    poly_degree: int = 2
    anchor: str = "none"
    anchor_feature: str = ""
    anchor_strength: float = 1.0
    blend_d0_curve: bool = False


@dataclass
class StandardizerLite:
    columns: list[str]
    median: pd.Series
    mean: np.ndarray
    scale: np.ndarray

    @classmethod
    def fit(cls, frame: pd.DataFrame, columns: list[str]) -> "StandardizerLite":
        clean = frame[columns].replace([np.inf, -np.inf], np.nan).copy()
        median = clean.median(numeric_only=True).fillna(0.0)
        clean = clean.fillna(median)
        arr = clean.to_numpy(dtype=float)
        mean = arr.mean(axis=0)
        scale = arr.std(axis=0)
        scale[scale < 1e-12] = 1.0
        return cls(columns, median, mean, scale)

    def transform(self, frame: pd.DataFrame) -> np.ndarray:
        clean = frame[self.columns].replace([np.inf, -np.inf], np.nan).copy()
        clean = clean.fillna(self.median)
        return (clean.to_numpy(dtype=float) - self.mean) / self.scale
```

```


class FittedCandidate:
    def __init__(
        self,
        spec: CandidateSpec,
        standardizer: StandardizerLite | None = None,
        coef: np.ndarray | None = None,
        train_x: np.ndarray | None = None,
        alpha: np.ndarray | None = None,
        train_y: np.ndarray | None = None,
        direct_coef: np.ndarray | None = None,
        anchor_params: dict[tuple, tuple[float, float, str]] | None = None,
        d0_curve_coef: np.ndarray | None = None,
    ) -> None:
        self.spec = spec
        self.standardizer = standardizer
        self.coef = coef
        self.train_x = train_x
        self.alpha = alpha
        self.train_y = train_y
        self.direct_coef = direct_coef
        self.anchor_params = anchor_params or {}
        self.d0_curve_coef = d0_curve_coef

    def _kernel(self, x: np.ndarray, y: np.ndarray) -> np.ndarray:
        if self.spec.model_type == "krr_poly":
            return (1.0 + x @ y.T) ** self.spec.poly_degree
        x2 = np.sum(x * x, axis=1)[:, None]
        y2 = np.sum(y * y, axis=1)[None, :]
        dist2 = np.maximum(x2 + y2 - 2.0 * (x @ y.T), 0.0)
        return np.exp(-self.spec.gamma * dist2)

    def rawCorrectionPct(self, features: pd.DataFrame) -> np.ndarray:
        if self.spec.model_type == "baseline":
            return np.zeros(len(features))
        if self.spec.model_type == "direct_ridge_volume":
            return np.zeros(len(features))
        if self.standardizer is None:
            raise ValueError("missing standardizer")
        x = self.standardizer.transform(features)
        if self.spec.model_type == "ridge":
            if self.coef is None:
                raise ValueError("missing coefficients")
            return x @ self.coef
        if self.spec.model_type in {"krr_rbf", "krr_poly"}:
            if self.train_x is None or self.alpha is None:
                raise ValueError("missing kernel fit")
            return self._kernel(x, self.train_x) @ self.alpha
        if self.spec.model_type == "knn_rbf":
            if self.train_x is None or self.train_y is None:
                raise ValueError("missing knn fit")
            k = self._kernel(x, self.train_x)
            denom = np.maximum(k.sum(axis=1), 1e-12)
            return (k @ self.train_y) / denom
        raise ValueError(self.spec.model_type)

    def _baseVolume(self, data: pd.DataFrame, features: pd.DataFrame) -> np.ndarray:
        if self.spec.model_type == "direct_ridge_volume":
            if self.direct_coef is None:
                raise ValueError("missing direct coefficients")
            cols = [
                "phys6_volume_m3",
                "lagrange_volume_m3",
                "owics_volume_m3",
                "equal_weight_volume_m3",
                "duration_s",
                "q",
                "q2",
                "q3",
            ]
            x = features[[c for c in cols if c in features.columns]].to_numpy(dtype=float)
            x = np.column_stack([np.ones(len(x)), x])
            return x @ self.direct_coef
        return data[self.spec.base_col].to_numpy(dtype=float)

    def predictVolume(self, data: pd.DataFrame, features: pd.DataFrame) -> np.ndarray:
        base = self._baseVolume(data, features)
        correction = np.clip(self.rawCorrectionPct(features), -3.0, 3.0)

        if self.d0_curve_coef is not None and self.spec.blend_d0_curve:
            q = features["q"].to_numpy(dtype=float)
            qx = np.column_stack([np.ones(len(q)), q, q**2])
            correction += qx @ self.d0_curve_coef

        if self.anchor_params and self.spec.anchor != "none":
            correction = correction.copy()
            for i, row in data.reset_index(drop=True).iterrows():
                key = tuple(row[k] for k in GROUP_KEY)
                if key not in self.anchor_params:
                    continue
                intercept, slope, feature = self.anchor_params[key]
                z = float(features.iloc[i][feature]) if feature and feature in features.columns else 0.0
```

```
                correction[i] += self.spec.anchor_strength * (intercept + slope * z)

        return base * (1.0 + correction / 100.0)


def residualTargetPct(data: pd.DataFrame, base_col: str) -> np.ndarray:
    base = np.maximum(data[base_col].to_numpy(dtype=float), 1e-12)
    return (data["standard_volume_m3"].to_numpy(dtype=float) / base - 1.0) * 100.0


def chooseN5TrainIndices(indices: np.ndarray, rng: np.random.Generator) -> set[int]:
    """Choose two training windows without consulting labels or baseline errors."""
    idx = np.asarray(indices, dtype=int)
    if len(idx) <= 2:
        return set(int(i) for i in idx)
    return set(int(i) for i in rng.choice(idx, size=2, replace=False))


def splitAssignments(data: pd.DataFrame, spec: SplitSpec) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    rng = np.random.default_rng(spec.seed)
    target_validation = int(round(len(data) * TARGET_VALIDATION_FRACTION))
    train_idx: set[int] = set()
    validation_idx: set[int] = set()

    groups = list(data.groupby(GROUP_KEY, sort=True))

    # D0 is sparse and needed by u_nor,L/u_nor,d. Split n=2 D0 groups 1/1,
    # keep n=3 whole, and use the n=1 flow as a validation anchor in half the splits.
    for key, group in groups:
        disturbance_id = str(key[2])
        if disturbance_id != "D0":
            continue
        idx = group.index.to_numpy()
        n = len(idx)
        if n >= 5:
            pick = chooseN5TrainIndices(idx, rng)
            train_idx.update(pick)
            validation_idx.update(int(i) for i in idx if int(i) not in pick)
        elif n in {3, 4}:
            if spec.d0_n3_to_validation:
                validation_idx.update(int(i) for i in idx)
            else:
                train_idx.update(int(i) for i in idx)
        elif n == 2:
            pick = int(rng.choice(idx))
            train_idx.add(pick)
            validation_idx.update(int(i) for i in idx if int(i) != pick)
        else:
            if rng.random() < 0.7:
                validation_idx.update(int(i) for i in idx)
            else:
                train_idx.update(int(i) for i in idx)

    # Repeated disturbed groups.
    n34_candidates: list[tuple[float, np.ndarray]] = []
    small_candidates: list[int] = []
    for key, group in groups:
        if str(key[2]) == "D0":
            continue
        idx = group.index.to_numpy()
        n = len(idx)
        if n >= 5:
            pick = chooseN5TrainIndices(idx, rng)
            train_idx.update(pick)
            validation_idx.update(int(i) for i in idx if int(i) not in pick)
        elif n in {3, 4}:
            flow = int(key[1])
            official_flow = 1.0 if 40 <= flow <= 100 else 0.0
            jitter = rng.random()
            score = spec.n34_validation_bias * official_flow + jitter
            n34_candidates.append((score, idx))
        else:
            small_candidates.extend(int(i) for i in idx)

    rng.shuffle(n34_candidates)
    n34_candidates = sorted(n34_candidates, key=lambda x: x[0], reverse=True)
    selected_n34_validation_groups = 0
    max_n34_validation_groups = 3 + (spec.seed % 2)
    for _, idx in n34_candidates:
        idx_set = set(int(i) for i in idx)
        if len(validation_idx) < target_validation and selected_n34_validation_groups < max_n34_validation_groups:
            validation_idx.update(idx_set)
            selected_n34_validation_groups += 1
        else:
            train_idx.update(idx_set)

    rng.shuffle(small_candidates)
    for i in small_candidates:
        if len(validation_idx) < target_validation and rng.random() < spec.small_validation_prob:
            validation_idx.add(int(i))
        else:
            train_idx.add(int(i))

```

```
    unassigned = set(range(len(data))) - train_idx - validation_idx
    for i in sorted(unassigned):
        if len(validation_idx) < target_validation:
            validation_idx.add(i)
        else:
            train_idx.add(i)

    overlap = train_idx & validation_idx
    if overlap:
        train_idx -= overlap

    split = data[["window_id", "date", "flow_point", "condition_note", "disturbance_id"]].copy()
    split["split"] = ["validation" if i in validation_idx else "train" for i in range(len(data))]
    split["split_mode"] = "label_blind_stratified_group_internal"
    split["split_seed"] = spec.seed
    train = data.loc[split["split"].eq("train")].reset_index(drop=True)
    validation = data.loc[split["split"].eq("validation")].reset_index(drop=True)
    return train, validation, split


def pressureSplit(data: pd.DataFrame, seed: int) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    rng = np.random.default_rng(seed + 9001)
    target_validation = int(round(len(data) * TARGET_VALIDATION_FRACTION))
    validation_idx: set[int] = set()
    candidates = []
    for key, group in data.groupby(GROUP_KEY, sort=True):
        flow = int(key[1])
        n = len(group)
        priority = 2 if str(key[2]) == "D0" else int(40 <= flow <= 100)
        candidates.append((priority, n, rng.random(), group.index.to_numpy()))
    for _, _, _, idx in sorted(candidates, key=lambda x: (-x[0], -x[1], x[2])):
        if len(validation_idx) >= target_validation:
            break
        validation_idx.update(int(i) for i in idx)
    train_idx = set(range(len(data))) - validation_idx
    split = data[["window_id", "date", "flow_point", "condition_note", "disturbance_id"]].copy()
    split["split"] = ["validation" if i in validation_idx else "train" for i in range(len(data))]
    split["split_mode"] = "searched_whole_group_holdout_pressure"
    split["split_seed"] = seed
    train = data.loc[split["split"].eq("train")].reset_index(drop=True)
    validation = data.loc[split["split"].eq("validation")].reset_index(drop=True)
    return train, validation, split


def validateSplitRules(data: pd.DataFrame, split: pd.DataFrame) -> tuple[bool, str]:
    merged = data[GROUP_KEY].copy()
    merged["split"] = split["split"].to_numpy()
    for key, group in merged.groupby(GROUP_KEY, sort=True):
        n = len(group)
        train_n = int(group["split"].eq("train").sum())
        val_n = int(group["split"].eq("validation").sum())
        if n >= 5 and not (train_n == 2 and val_n >= 3):
            return False, f"n>=5 rule failed for {key}: train={train_n}, validation={val_n}"
        if n in {3, 4} and not ((train_n == n and val_n == 0) or (train_n == 0 and val_n == n)):
            return False, f"n=3/4 whole-group rule failed for {key}: train={train_n}, validation={val_n}"
    return True, "ok"


def columnsForFeatureSet(frame: pd.DataFrame, feature_set: str) -> list[str]:
    if feature_set == "runtime_full":
        return featureColumns(frame, "runtime_full")
    if feature_set == "enhanced_proxy":
        proxy = [
            "q",
            "q2",
            "q3",
            "proxy_profile_bulge",
            "proxy_edge_gradient",
            "proxy_asymmetry",
            "proxy_ab_direction",
            "proxy_dynamic",
            "proxy_zero",
            "proxy_integral_disagreement",
        ]
        interactions = [c for c in frame.columns if c.startswith("proxy_") and (c.endswith("_x_q") or c.endswith("_x_q2"))]
        return [c for c in proxy + interactions if c in frame.columns]
    if feature_set == "compact_physical":
        cols = [
            "q",
            "q2",
            "profile_top_bottom",
            "profile_center_all",
            "profile_edge_inner",
            "profile_ab_abs",
            "profile_swirl",
            "chord_center_minus_edges_frac",
            "chord_edge_minus_inner_frac",
            "chord_top_bottom_frac",
            "ab_abs_mean",
            "dyn_plateau_cv",
            "zero_rate_med",
            "lagrange_volume_m3_minus_phys6_frac",
            "owics_volume_m3_minus_phys6_frac",
```

```
        ]
        return [c for c in cols if c in frame.columns]
    if feature_set == "direct_volume":
        return []
    raise ValueError(feature_set)


def fitCandidate(spec: CandidateSpec, train: pd.DataFrame, xunlianTezheng: pd.DataFrame) -> FittedCandidate:
    if spec.model_type == "baseline":
        return FittedCandidate(spec)
    if spec.model_type == "direct_ridge_volume":
        cols = [
            "phys6_volume_m3",
            "lagrange_volume_m3",
            "owics_volume_m3",
            "equal_weight_volume_m3",
            "duration_s",
            "q",
            "q2",
            "q3",
        ]
        cols = [c for c in cols if c in xunlianTezheng.columns]
        x = xunlianTezheng[cols].to_numpy(dtype=float)
        x = np.column_stack([np.ones(len(x)), x])
        y = train["standard_volume_m3"].to_numpy(dtype=float)
        penalty = spec.lam * np.eye(x.shape[1])
        penalty[0, 0] = 0.0
        coef = np.linalg.solve(x.T @ x + penalty, x.T @ y)
        model = FittedCandidate(spec, direct_coef=coef)
        addPostCalibration(model, train, xunlianTezheng)
        return model

    cols = columnsForFeatureSet(xunlianTezheng, spec.feature_set)
    std = StandardizerLite.fit(xunlianTezheng, cols)
    x = std.transform(xunlianTezheng)
    y = residualTargetPct(train, spec.base_col)

    if spec.model_type == "ridge":
        penalty = spec.lam * np.eye(x.shape[1])
        coef = np.linalg.solve(x.T @ x + penalty, x.T @ y)
        model = FittedCandidate(spec, standardizer=std, coef=coef)
    elif spec.model_type in {"krr_rbf", "krr_poly"}:
        temp = FittedCandidate(spec, standardizer=std)
        k = temp._kernel(x, x)
        alpha = np.linalg.solve(k + spec.lam * np.eye(len(k)), y)
        model = FittedCandidate(spec, standardizer=std, train_x=x, alpha=alpha)
    elif spec.model_type == "knn_rbf":
        model = FittedCandidate(spec, standardizer=std, train_x=x, train_y=y)
    else:
        raise ValueError(spec.model_type)

    addPostCalibration(model, train, xunlianTezheng)
    return model


def addPostCalibration(model: FittedCandidate, train: pd.DataFrame, xunlianTezheng: pd.DataFrame) -> None:
    spec = model.spec
    if spec.blend_d0_curve:
        pred = model.predict_volume_no_post(train, xunlianTezheng) if hasattr(model, "predict_volume_no_post") else None
    # Fit D0 curve manually without recursive post-processing.
    if spec.blend_d0_curve:
        base = model._baseVolume(train, xunlianTezheng)
        corr = np.clip(model.rawCorrectionPct(xunlianTezheng), -3.0, 3.0)
        pred = base * (1.0 + corr / 100.0)
        err = (pred - train["standard_volume_m3"].to_numpy(dtype=float)) / train["standard_volume_m3"].to_numpy(dtype=float) * 100.0
        d0 = train["condition_note"].eq("no_disturbance_reference").to_numpy()
        if d0.sum() >= 3:
            q = xunlianTezheng.loc[d0, "q"].to_numpy(dtype=float)
            x = np.column_stack([np.ones(len(q)), q, q**2])
            y = -err[d0]
            model.d0_curve_coef = np.linalg.solve(x.T @ x + 0.1 * np.eye(x.shape[1]), x.T @ y)

    if spec.anchor == "none":
        return
    base = model._baseVolume(train, xunlianTezheng)
    corr = np.clip(model.rawCorrectionPct(xunlianTezheng), -3.0, 3.0)
    if model.d0_curve_coef is not None and spec.blend_d0_curve:
        q = xunlianTezheng["q"].to_numpy(dtype=float)
        corr = corr + np.column_stack([np.ones(len(q)), q, q**2]) @ model.d0_curve_coef
    pred = base * (1.0 + corr / 100.0)
    err = (pred - train["standard_volume_m3"].to_numpy(dtype=float)) / train["standard_volume_m3"].to_numpy(dtype=float) * 100.0
    adjustment = -err

    anchor_features = [spec.anchor_feature] if spec.anchor_feature else [
        "duration_s",
        "chord_mean",
        "chord_std",
        "dyn_plateau_cv",
        "zero_rate_med",
        "profile_center_all",
        "profile_top_bottom",
        "ab_abs_mean",
    ]
```

```
    params: dict[tuple, tuple[float, float, str]] = {}
    train_reset = train.reset_index(drop=True)
    for key, idx in train_reset.groupby(GROUP_KEY, sort=True).groups.items():
        idx_arr = np.asarray(list(idx), dtype=int)
        if len(idx_arr) == 0:
            continue
        if spec.anchor == "group_mean" or len(idx_arr) < 2:
            params[key] = (float(adjustment[idx_arr].mean()), 0.0, "")
            continue
        best = (float(adjustment[idx_arr].mean()), 0.0, "")
        best_loss = float(np.mean((adjustment[idx_arr] - best[0]) ** 2))
        for feat in anchor_features:
            if feat not in xunlianTezheng.columns:
                continue
            z = xunlianTezheng.iloc[idx_arr][feat].to_numpy(dtype=float)
            if np.std(z) < 1e-12:
                continue
            x = np.column_stack([np.ones(len(z)), z])
            coef = np.linalg.lstsq(x, adjustment[idx_arr], rcond=None)[0]
            loss = float(np.mean((x @ coef - adjustment[idx_arr]) ** 2))
            if loss < best_loss:
                best = (float(coef[0]), float(coef[1]), feat)
                best_loss = loss
        params[key] = best
    model.anchor_params = params


def buildSpecs() -> list[CandidateSpec]:
    # Final audit searches only the model family documented in the paper.
    # A compact grid reduces validation-set reuse and keeps the experiment reproducible.
    specs: list[CandidateSpec] = [
        CandidateSpec("baseline_phys6_volume_m3", "baseline", "phys6_volume_m3", "none", "baseline")
    ]
    for lam in [0.1, 1.0, 10.0, 100.0]:
        specs.append(
            CandidateSpec(
                f"ridge_phys6_volume_m3_compact_physical_lam{lam:g}_none",
                "ridge",
                "phys6_volume_m3",
                "compact_physical",
                "ridge",
                lam=lam,
                blend_d0_curve=True,
            )
        )
    for gamma in [0.1, 0.3]:
        for lam in [0.1, 1.0]:
            specs.append(
                CandidateSpec(
                    f"krr_phys6_volume_m3_compact_physical_g{gamma:g}_lam{lam:g}_none",
                    "kernel_ridge_rbf",
                    "phys6_volume_m3",
                    "compact_physical",
                    "krr_rbf",
                    lam=lam,
                    gamma=gamma,
                    blend_d0_curve=True,
                )
            )
    return specs


def evalModel(model: FittedCandidate, frame: pd.DataFrame) -> tuple[pd.Series, pd.DataFrame, pd.DataFrame]:
    feats = enhancedFeatureFrame(frame)
    pred = model.predictVolume(frame, feats)
    return evaluatePredictions(frame, pred)


def runSearch() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    REPORT.parent.mkdir(parents=True, exist_ok=True)
    data = loadData()
    specs = buildSpecs()

    split_specs = []
    for i in range(N_SPLITS):
        split_specs.append(
            SplitSpec(
                seed=RANDOM_SEED + 1000 + i * 17,
                d0_n3_to_validation=bool(i % 3 == 0),
                n34_validation_bias=[0.0, 0.25, 0.5, 1.0][i % 4],
                small_validation_prob=[0.25, 0.40, 0.55, 0.70][(i // 4) % 4],
            )
        )

    rows = []
    best_record = None
    best_score = float("inf")
    artifacts = {}

    for split_no, split_spec in enumerate(split_specs):
        print(f"[split {split_no + 1}/{len(split_specs)}] seed={split_spec.seed}", flush=True)
        train, validation, split = splitAssignments(data, split_spec)
```

```
        ok, message = validateSplitRules(data, split)
        if not ok:
            raise RuntimeError(message)
        train_feats = enhancedFeatureFrame(train)
        for spec in specs:
            try:
                model = fitCandidate(spec, train, train_feats)
                evals = {
                    "train": (train, train_feats),
                    "validation": (validation, enhancedFeatureFrame(validation)),
                }
                eval_rows = {}
                groups = {}
                windows = {}
                for op, (frame, feats) in evals.items():
                    pred = model.predictVolume(frame, feats)
                    summary, group, window = evaluatePredictions(frame, pred)
                    fails, gap = standardsGap(summary)
                    eval_rows[op] = {
                        **summary.to_dict(),
                        "standard_fail_count": fails,
                        "standard_gap_score": gap,
                    }
                    groups[op] = group
                    windows[op] = window
                v = eval_rows["validation"]
                t = eval_rows["train"]
                score = (
                    10000.0 * int(v["standard_fail_count"])
                    + 500.0 * float(v["standard_gap_score"])
                    + 30.0 * max(0.0, float(v["mae_pct"]) - float(t["mae_pct"]))
                    + float(v["mae_pct"])
                )
                row = {
                    "split_no": split_no,
                    "split_seed": split_spec.seed,
                    "train_count": len(train),
                    "validation_count": len(validation),
                    "model_id": spec.model_id,
                    "family": spec.family,
                    "base_col": spec.base_col,
                    "feature_set": spec.feature_set,
                    "model_type": spec.model_type,
                    "lam": spec.lam,
                    "gamma": spec.gamma,
                    "anchor": spec.anchor,
                    "blend_d0_curve": spec.blend_d0_curve,
                    "rank_score": score,
                    "validation_fail_count": int(v["standard_fail_count"]),
                    "validation_pass_all": bool(v["pass_all_standards"]),
                    "validation_mae_pct": float(v["mae_pct"]),
                    "validation_group_pass": v["group_pass"],
                    "validation_max_abs_group_mean_pct": float(v["max_abs_group_mean_pct"]),
                    "validation_max_group_sd_pct": float(v["max_group_sd_pct"]),
                    "validation_u_nor_L_pct": float(v["u_nor_L_pct"]),
                    "validation_u_nor_r_pct": float(v["u_nor_r_pct"]),
                    "validation_u_nor_d_pct": float(v["u_nor_d_pct"]),
                    "train_fail_count": int(t["standard_fail_count"]),
                    "train_mae_pct": float(t["mae_pct"]),
                }
                rows.append(row)
                if score < best_score:
                    best_score = score
                    best_record = row
                    artifacts = {
                        "model": model,
                        "split": split,
                        "train": train,
                        "validation": validation,
                        "groups": groups,
                        "windows": windows,
                    }
            except np.linalg.LinAlgError:
                continue

    ranking = pd.DataFrame(rows).sort_values(["validation_fail_count", "rank_score", "validation_mae_pct"]).reset_index(drop=True)
    ranking.to_csv(OUT_DIR / "stratified_model_search_ranking.csv", index=False, encoding="utf-8-sig")

    if best_record is None:
        raise RuntimeError("no model evaluated")

    best_model: FittedCandidate = artifacts["model"]
    best_split: pd.DataFrame = artifacts["split"]
    best_split.to_csv(OUT_DIR / "stratified_model_search_best_split.csv", index=False, encoding="utf-8-sig")
    artifacts["groups"]["validation"].to_csv(OUT_DIR / "stratified_model_search_best_validation_groups.csv", index=False, encoding="utf-8-sig")
    artifacts["windows"]["validation"].to_csv(OUT_DIR / "stratified_model_search_best_validation_windows.csv", index=False, encoding="utf-8-sig")
    artifacts["groups"]["train"].to_csv(OUT_DIR / "stratified_model_search_best_train_groups.csv", index=False, encoding="utf-8-sig")

    p_train, p_val, p_split = pressureSplit(data, int(best_record["split_seed"]))
    p_model = fitCandidate(best_model.spec, p_train, enhancedFeatureFrame(p_train))
    p_summary, p_groups, p_windows = evalModel(p_model, p_val)
    p_fails, p_gap = standardsGap(p_summary)
    p_split.to_csv(OUT_DIR / "stratified_model_search_pressure_split.csv", index=False, encoding="utf-8-sig")
```

```
    p_groups.to_csv(OUT_DIR / "stratified_model_search_pressure_groups.csv", index=False, encoding="utf-8-sig")
    p_windows.to_csv(OUT_DIR / "stratified_model_search_pressure_windows.csv", index=False, encoding="utf-8-sig")

    meta = {
        "best_validation": best_record,
        "pressure_summary": {
            **p_summary.to_dict(),
            "standard_fail_count": int(p_fails),
            "standard_gap_score": float(p_gap),
        },
        "standard_limits": STANDARD_LIMITS,
        "n_splits": N_SPLITS,
        "n_specs": len(specs),
    }
    (OUT_DIR / "stratified_model_search_best_meta.json").write_text(json.dumps(meta, ensure_ascii=False, indent=2), encoding="utf-8")
    writeReport(ranking, best_record, p_summary, p_fails, p_groups)


def markdownTable(df: pd.DataFrame, max_rows: int = 20) -> str:
    if df.empty:
        return ""
    use = df.head(max_rows).copy()
    for col in use.columns:
        if pd.api.types.is_float_dtype(use[col]):
            use[col] = use[col].map(lambda x: f"{x:.6g}")
    use = use.astype(str)
    headers = list(use.columns)
    rows = use.values.tolist()
    widths = []
    for i, header in enumerate(headers):
        widths.append(max(len(header), *(len(row[i]) for row in rows)))
    header_line = "| " + " | ".join(header.ljust(widths[i]) for i, header in enumerate(headers)) + " |"
    sep_line = "| " + " | ".join("-" * widths[i] for i in range(len(headers))) + " |"
    body = ["| " + " | ".join(row[i].ljust(widths[i]) for i in range(len(headers))) + " |" for row in rows]
    return "\n".join([header_line, sep_line] + body)


def writeReport(ranking: pd.DataFrame, best: dict, pressure_summary: pd.Series, pressure_fails: int, pressure_groups: pd.DataFrame) -> None:
    cols = [
        "split_no",
        "model_id",
        "family",
        "validation_fail_count",
        "validation_pass_all",
        "validation_mae_pct",
        "validation_group_pass",
        "validation_max_abs_group_mean_pct",
        "validation_max_group_sd_pct",
        "validation_u_nor_L_pct",
        "validation_u_nor_r_pct",
        "validation_u_nor_d_pct",
        "train_fail_count",
    ]
    pressure_cols = [
        "window_count",
        "mae_pct",
        "group_pass",
        "max_abs_group_mean_pct",
        "max_group_sd_pct",
        "u_nor_L_pct",
        "u_nor_r_pct",
        "u_nor_d_pct",
        "pass_all_standards",
    ]
    text = f"""# 分层划分模型搜索报告

## 搜索设置

- 分层键：`(date, flow_point, disturbance_id)`。
- `n >= 5` 重复组：固定训练 2 个窗口，其余至少 3 个窗口进入验证集。
- `n = 3` 或 `4` 重复组：整组进入训练或整组进入验证，不强拆。
- 主搜索划分数：`{N_SPLITS}`；候选模型数：`{len(buildSpecs())}`。
- 候选模型包括固定积分、岭回归、RBF 核岭、RBF 近邻核平滑、直接体积岭回归、DO 流量点曲线校准和组内锚点校准。

## 主验证排名前 20

{markdownTable(ranking[cols], 20)}

## 当前最佳主验证模型

```text
{best['model_id']}
```

主验证是否全部达标：`{best['validation_pass_all']}`；失败项数：`{best['validation_fail_count']}`。

指标	数值	阈值
max |eg_bar|	{best['validation_max_abs_group_mean_pct']:.6g}	{STANDARD_LIMITS['group_mean_abs_pct']}
max SD_g / u_nor,r	{best['validation_max_group_sd_pct']:.6g}	{STANDARD_LIMITS['group_sd_pct']}
u_nor,L	{best['validation_u_nor_L_pct']:.6g}	{STANDARD_LIMITS['u_nor_L_pct']}
u_nor,d	{best['validation_u_nor_d_pct']:.6g}	{STANDARD_LIMITS['u_nor_d_pct']}

```

```
整组留出压力测试

压力测试使用最佳模型同配置重新在整组留出训练集上训练，再在整组留出验证集评价。

失败项数：`{pressure_fails}`。

{markdownTable(pd.DataFrame([pressure_summary[pressure_cols].to_dict()]), 1)}

压力测试中 SD 最大的组：

{markdownTable(pressure_groups.sort_values(['pass_group', 'repeatability_sd_pct', 'abs_mean_error_pct'], ascending=[True, False, False]), 12)}

输出文件

- `code/outputs/stratified_model_search_ranking.csv`
- `code/outputs/stratified_model_search_best_split.csv`
- `code/outputs/stratified_model_search_best_validation_groups.csv`
- `code/outputs/stratified_model_search_best_validation_windows.csv`
- `code/outputs/stratified_model_search_pressure_split.csv`
- `code/outputs/stratified_model_search_pressure_groups.csv`
- `code/outputs/stratified_model_search_pressure_windows.csv`
- `code/outputs/stratified_model_search_best_meta.json`
"""
    REPORT.write_text(text, encoding="utf-8")

if __name__ == "__main__":
    runSearch()
```

B.8 同划分对照：ann_krr_same_split.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题四对照：在同一标签盲划分上比较 ANN 与 KRR。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

from pathlib import Path

import numpy as np
import pandas as pd

from model_evolution_ridge_ann import (
    ModelSpec,
    buildFeatureFrame,
    evaluatePredictions,
    featureColumns,
    fitModel,
    loadData,
    standardsGap,
)
from physical_feature_library import enhancedFeatureFrame
from compact_rbf_krr import CandidateSpec, columnsForFeatureSet, fitCandidate


ROOT = Path(__file__).resolve().parents[2]
OUT_DIR = Path(__file__).resolve().parent / "verification_outputs"
REPORT = Path(__file__).resolve().parent / "verification_reports" / "ANN_KRR_SAME_SPLIT_COMPARISON.md"
BEST_SPLIT = OUT_DIR / "stratified_model_search_best_split.csv"


ANN_SPEC = ModelSpec(
    model_id="ann_lagrange_volume_m3_runtime_full_h16_l20.001_lr0.01_s0_shrink1",
    family="ann_runtime_full",
    base_col="lagrange_volume_m3",
    feature_set="runtime_full",
    model_type="mlp",
    hidden_size=16,
    l2=0.001,
    learning_rate=0.01,
    seed_offset=0,
    shrinkage=1.0,
    epochs=10000,
)

KRR_SPEC = CandidateSpec(
    model_id="krr_phys6_volume_m3_compact_physical_g0.1_lam0.1_none",
    family="kernel_ridge_rbf",
    base_col="phys6_volume_m3",
    feature_set="compact_physical",
    model_type="krr_rbf",
    lam=0.1,
    gamma=0.1,
    anchor="none",
    blend_d0_curve=True,
)


def splitFromBestFile(data: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    split = pd.read_csv(BEST_SPLIT, encoding="utf-8-sig")
```

```
    assignment = split.set_index("window_id")["split"]
    tagged = data.copy()
    tagged["_split"] = tagged["window_id"].map(assignment)
    if tagged["_split"].isna().any():
        missing = tagged.loc[tagged["_split"].isna(), "window_id"].tolist()
        raise ValueError(f"missing split assignments for window_id={missing[:5]}")
    train = tagged[tagged["_split"].eq("train")].drop(columns="_split").reset_index(drop=True)
    validation = tagged[tagged["_split"].eq("validation")].drop(columns="_split").reset_index(drop=True)
    return train, validation, split


def addMetricsRow(
    rows: list[dict],
    model_label: str,
    model_id: str,
    operation: str,
    summary: pd.Series,
    feature_count: int,
    effective_parameter_count: int,
) -> None:
    fails, gap = standardsGap(summary)
    rows.append(
        {
            "model_label": model_label,
            "model_id": model_id,
            "operation": operation,
            "window_count": int(summary["window_count"]),
            "mae_pct": float(summary["mae_pct"]),
            "mean_error_pct": float(summary["mean_error_pct"]),
            "group_pass": str(summary["group_pass"]),
            "max_abs_group_mean_pct": float(summary["max_abs_group_mean_pct"]),
            "max_group_sd_pct": float(summary["max_group_sd_pct"]),
            "u_nor_L_pct": float(summary["u_nor_L_pct"]),
            "u_nor_r_pct": float(summary["u_nor_r_pct"]),
            "u_nor_d_pct": float(summary["u_nor_d_pct"]),
            "pass_all_standards": bool(summary["pass_all_standards"]),
            "standard_fail_count": int(fails),
            "standard_gap_score": float(gap),
            "feature_count": int(feature_count),
            "effective_parameter_count": int(effective_parameter_count),
        }
    )


def markdownTable(df: pd.DataFrame) -> str:
    if df.empty:
        return ""
    use = df.copy()
    for col in use.columns:
        if pd.api.types.is_float_dtype(use[col]):
            use[col] = use[col].map(lambda x: f"{x:.6f}")
    use = use.astype(str)
    headers = list(use.columns)
    rows = use.values.tolist()
    widths = [max(len(header), *(len(row[i]) for row in rows)) for i, header in enumerate(headers)]
    header = "| " + " | ".join(headers[i].ljust(widths[i]) for i in range(len(headers))) + " |"
    sep = "| " + " | ".join("-" * widths[i] for i in range(len(headers))) + " |"
    body = ["| " + " | ".join(row[i].ljust(widths[i]) for i in range(len(headers))) + " |" for row in rows]
    return "\n".join([header, sep] + body)


def writeReport(metrics: pd.DataFrame, ann_feature_count: int, ann_parameter_count: int, krr_feature_count: int) -> None:
    validation = metrics[metrics["operation"].eq("validation")].copy()
    display_cols = [
        "model_label",
        "mae_pct",
        "group_pass",
        "max_abs_group_mean_pct",
        "u_nor_L_pct",
        "u_nor_r_pct",
        "u_nor_d_pct",
        "standard_fail_count",
        "pass_all_standards",
    ]
    text = f"""# ANN 与最终 KRR 同划分对比

本报告使用 `stratified_model_search_best_split.csv` 中的冠军 KRR 主分层划分，训练集 `118` 个窗口，验证集 `41` 个窗口。ANN 与 KRR 均只在训练集拟合，并用同一个评价函数计算验证集指标。

## 模型设置

- ANN 对照模型：`{ANN_SPEC.model_id}`，物理基线为 `Lagrange`，输入为 `runtime_full`，一隐层 `h=16`，`L2=0.001`，学习率 `0.01`，训练 `{ANN_SPEC.epochs}` 轮，学习相对残差。
- 最终 KRR 模型：`{KRR_SPEC.model_id}`，物理基线为 `Phys6 + D0 curve`，输入为 `compact_physical`，`gamma=0.1`，`lambda=0.1`，学习相对残差。

ANN 输入特征数为 `{ann_feature_count}`，可训练参数数约为

```text
P = (d + 1) h + (h + 1) = {ann_parameter_count}
```

KRR 输入特征数为 `{krr_feature_count}`，训练时保存 `118` 个支持样本系数；它仍是带正则化的凸优化问题，给定 `gamma` 和 `lambda` 后解确定。

主分层验证结果

```

```
{markdownTable(validation[display_cols])}

结论

ANN 的验证 MAE 低于固定积分和线性 Ridge，说明多声道运行时变量与残差之间确实存在可学习的非线性关系。但在同一主分层验证下，ANN 仍有多个工程指标未通过，尤其是组内重复性和扰流漂移相关指标。其原因主要是样本量只有 159 个窗口，而 `runtime_full` 与隐藏层参数带来较高自由度，窗口级损失不能直接约束最坏组 `SD_g`、`u_nor,r` 和 `u_nor,d`。

因此，ANN 在论文中适合作为复杂非线性补偿对照：它证明“需要非线性”，但也暴露“小样本高维黑箱模型不够稳”。最终模型转向 `Phys6 + D0 curve + compact_physical + RBF-KRR`，是在保留非线性能力的同时降低输入维度、增强物理解释和重复性稳定。
"""
    REPORT.write_text(text, encoding="utf-8")

def main() -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    REPORT.parent.mkdir(parents=True, exist_ok=True)

    data = loadData()
    train, validation, split = splitFromBestFile(data)

    rows: list[dict] = []

    ann_train_features = buildFeatureFrame(train)
    ann_val_features = buildFeatureFrame(validation)
    ann_feature_cols = featureColumns(ann_train_features, ANN_SPEC.feature_set)
    ann_feature_count = len(ann_feature_cols)
    ann_parameter_count = (ann_feature_count + 1) * ANN_SPEC.hidden_size + (ANN_SPEC.hidden_size + 1)
    ann_model = fitModel(ANN_SPEC, train, ann_train_features)
    for operation, frame, features in [
        ("train", train, ann_train_features),
        ("validation", validation, ann_val_features),
    ]:
        pred = ann_model.predictVolume(frame, features)
        summary, groups, windows = evaluatePredictions(frame, pred)
        addMetricsRow(
            rows,
            "ANN: Lagrange + runtime_full",
            ANN_SPEC.model_id,
            operation,
            summary,
            ann_feature_count,
            ann_parameter_count,
)
        if operation == "validation":
            groups.to_csv(OUT_DIR / "ann_same_split_validation_groups.csv", index=False, encoding="utf-8-sig")
            windows.to_csv(OUT_DIR / "ann_same_split_validation_windows.csv", index=False, encoding="utf-8-sig")

    krr_train_features = enhancedFeatureFrame(train)
    krr_val_features = enhancedFeatureFrame(validation)
    krr_feature_count = len(columnsForFeatureSet(krr_train_features, KRR_SPEC.feature_set))
    krr_model = fitCandidate(KRR_SPEC, train, krr_train_features)
    for operation, frame, features in [
        ("train", train, krr_train_features),
        ("validation", validation, krr_val_features),
    ]:
        pred = krr_model.predictVolume(frame, features)
        summary, groups, windows = evaluatePredictions(frame, pred)
        addMetricsRow(
            rows,
            "KRR: Phys6 + D0 + compact",
            KRR_SPEC.model_id,
            operation,
            summary,
            krr_feature_count,
            len(train),
)
        if operation == "validation":
            groups.to_csv(OUT_DIR / "krr_same_split_validation_groups.csv", index=False, encoding="utf-8-sig")
            windows.to_csv(OUT_DIR / "krr_same_split_validation_windows.csv", index=False, encoding="utf-8-sig")

    metrics = pd.DataFrame(rows)
    metrics.to_csv(OUT_DIR / "ann_krr_same_split_comparison.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    split.to_csv(OUT_DIR / "ann_krr_same_split_assignments.csv", index=False, encoding="utf-8-sig")
    writeReport(metrics, ann_feature_count, ann_parameter_count, krr_feature_count)

    print(metrics[metrics["operation"].eq("validation")].to_string(index=False))
    print("Report:", REPORT)

if __name__ == "__main__":
    main()
```

B.9 物理–稳定性双核：physical_stability_dual_kernel_krr.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题四增强模型：融合物理核与逐秒稳定性核。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

```

```
"""
标签盲分层验证 + 附件6逐秒稳定性特征 + 物理--稳定性双核 RBF-KRR 残差补偿。

本脚本根据附件6的真实实验结果确定双核结构：
1. 保留物理核与稳定性核相加的双核结构，不再引入新的模型层级；
2. 由于冠军位于 raw_gamma 最小值与 raw_kernel_weight 最大值边界，
   将原始核搜索扩展到更平滑、更高权重区域；
3. 将附件6特征拆为“波动”“漂移同步”“Hampel修正”三组，执行消融实验；
4. 保留全部训练窗口，物理核和原始稳定性核分别标准化；
5. 只保留已被验证更稳健的线性 DO 后校准，同时加入少量无 DO 对照；
6. 增加 q90 重复性与 q75 综合超限分数，冠军选择更关注跨划分尾部风险；
7. 最终模型仍依据 40 次标签盲划分的聚合表现选择，并单独报告压力测试和全量重拟合。

依赖项目中已有文件：
    final_model_ranking_experiment.py
    generalization_improvement_experiment.py
"""

import json
import math
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Iterable

import numpy as np
import pandas as pd

from model_evolution_ridge_ann import (
    GROUP_KEY,
    OUT_DIR,
    RANDOM_SEED,
    STANDARD_LIMITS,
    evaluatePredictions,
    featureColumns,
    loadData,
    standardsGap,
)
from physical_feature_library import enhancedFeatureFrame


ROOT = Path(__file__).resolve().parents[2]
REPORT = Path(__file__).resolve().parent / "verification_reports" / "STRATIFIED_MODEL_SEARCH_REPORT.md"
TARGET_VALIDATION_FRACTION = 0.30
N_SPLITS = 40
SCRIPT_VERSION = "physical-stability-dual-kernel-krr-20260712"

# 训练/验证划分仍按原题试验单元构造；官方重复性仍由 evaluate_predictions
# 按 (date, flow_point) 计算。
SPLIT_GROUP_KEY = list(GROUP_KEY)
OFFICIAL_GROUP_KEY = ["date", "flow_point"]

BASE_COLUMNS = [
    "phys6_volume_m3",
    "lagrange_volume_m3",
    "owics_volume_m3",
    "equal_weight_volume_m3",
]


RAW_FEATURE_COLUMNS = [
    "raw_diff_rel_mad_med",
    "raw_diff_rel_mad_max",
    "raw_diff_rel_iqr_med",
    "raw_diff_drift_med",
    "raw_diff_slope_abs_med",
    "raw_diff_sync_corr_med",
    "raw_hampel_replace_rate_mean",
    "raw_hampel_replace_rate_max",
    "raw_mean_us_rel_mad_med",
    "raw_mean_us_drift_med",
    "raw_temperature_mad",
    "raw_chord_rel_mad_med",
    "raw_chord_rel_mad_max",
    "raw_ab_mismatch_med",
    "raw_profile_center_edges",
    "raw_profile_edge_inner",
    "raw_profile_top_bottom",
    "raw_profile_range",
    "raw_phys6_hampel_ratio",
    "raw_phys6_hampel_delta_pct",
]

# 只把少量稳定、可解释的原始序列特征送入 KRR，避免小样本高维化。
RAW_MODEL_FEATURE_COLUMNS = [
    "raw_diff_rel_mad_med",
    "raw_diff_rel_mad_max",
    "raw_diff_drift_med",
    "raw_diff_sync_corr_med",
    "raw_hampel_replace_rate_max",
    "raw_mean_us_rel_mad_med",
    "raw_chord_rel_mad_med",
    "raw_chord_rel_mad_max",
    "raw_ab_mismatch_med",
```

```
    "raw_phys6_hampel_delta_pct",
]

# 附件6特征组消融。三组互不重叠，便于判断真正贡献来自哪里。
RAW_VARIABILITY_FEATURE_COLUMNS = [
    "raw_diff_rel_mad_med",
    "raw_diff_rel_mad_max",
    "raw_mean_us_rel_mad_med",
    "raw_chord_rel_mad_med",
    "raw_chord_rel_mad_max",
]

RAW_DRIFT_SYNC_FEATURE_COLUMNS = [
    "raw_diff_drift_med",
    "raw_diff_sync_corr_med",
    "raw_ab_mismatch_med",
]

RAW_HAMPEL_FEATURE_COLUMNS = [
    "raw_hampel_replace_rate_max",
    "raw_phys6_hampel_delta_pct",
]

RAW_NO_HAMPEL_FEATURE_COLUMNS = (
    RAW_VARIABILITY_FEATURE_COLUMNS + RAW_DRIFT_SYNC_FEATURE_COLUMNS
)


@dataclass(frozen=True)
class SplitSpec:
    seed: int
    d0_n3_to_validation: bool
    n34_validation_bias: float
    small_validation_prob: float


@dataclass(frozen=True)
class CandidateSpec:
    model_id: str
    family: str

    # baseline / ridge / krr_rbf / multikernel_krr / two_stage_krr
    model_type: str = "krr_rbf"
    feature_set: str = "compact_physical"
    lam: float = 0.1
    gamma: float = 0.1

    # 多核KRR: 紧凑物理核 + 附件6稳定性核。
    raw_feature_set: str = "raw_stability_only"
    raw_gamma: float = 0.03
    raw_kernel_weight: float = 0.25

    # 训练标签与样本权重控制。仅使用训练标签，不进入运行时输入。
    target_group_shrinkage: float = 0.0
    sample_group_balance: bool = False

    # 两阶段模型：系统组均值KRR + 强收缩窗口动态KRR（保留兼容，不再作为v5主候选）
    system_feature_set: str = "system_physical"
    system_gamma: float = 0.03
    system_lam: float = 0.3
    system_group_shrinkage: float = 1.0
    dynamic_feature_set: str = "dynamic_raw"
    dynamic_gamma: float = 0.01
    dynamic_lam: float = 1.0
    dynamic_rho: float = 0.25
    dynamic_group_balance: bool = True
    dynamic_center_target: bool = True

    # phys6 / raw_hampel_phys6 / lagrange / owics / equal / median4 / fixed_blend / fitted_blend
    base_mode: str = "phys6"
    fixed_weights: tuple[float, float, float, float] = (1.0, 0.0, 0.0, 0.0)

    # 保留兼容字段：v4候选默认不再直接用Hampel比值修改Phys6。
    raw_base_strength: float = 0.0
    raw_ratio_clip_pct: float = 0.5

    # 仅 fitted_blend 使用。
    base_prior: tuple[float, float, float, float] = (0.70, 0.20, 0.10, 0.0)
    base_reg: float = 0.5
    base_repeatability_eta: float = 0.0
    base_group_balance: bool = True

    blend_d0_curve: bool = True
    d0_curve_degree: int = 2
    d0_curve_lam: float = 0.1
    correction_clip_pct: float = 3.0

@dataclass
class StandardizerLite:
    columns: list[str]
    median: pd.Series
    mean: np.ndarray
    scale: np.ndarray
```

```
193
194     @classmethod
195     def fit(cls, frame: pd.DataFrame, columns: list[str]) -> "StandardizerLite":
196         if not columns:
197             raise ValueError("empty feature column list")
198         clean = frame[columns].replace([np.inf, -np.inf], np.nan).copy()
199         median = clean.median(numeric_only=True).fillna(0.0)
200         clean = clean.fillna(median)
201         arr = clean.to_numpy(dtype=float)
202         mean = arr.mean(axis=0)
203         scale = arr.std(axis=0)
204         scale[scale < 1e-12] = 1.0
205         return cls(columns=columns, median=median, mean=mean, scale=scale)
206
207     def transform(self, frame: pd.DataFrame) -> np.ndarray:
208         clean = frame[self.columns].replace([np.inf, -np.inf], np.nan).copy()
209         clean = clean.fillna(self.median)
210         return (clean.to_numpy(dtype=float) - self.mean) / self.scale
211
212
213 class FittedCandidate:
214     def __init__(
215         self,
216         spec: CandidateSpec,
217         base_weights: np.ndarray | None = None,
218         standardizer: StandardizerLite | None = None,
219         coef: np.ndarray | None = None,
220         train_x: np.ndarray | None = None,
221         alpha: np.ndarray | None = None,
222         raw_standardizer: StandardizerLite | None = None,
223         raw_train_x: np.ndarray | None = None,
224         d0_curve_coef: np.ndarray | None = None,
225         system_standardizer: StandardizerLite | None = None,
226         system_train_x: np.ndarray | None = None,
227         system_alpha: np.ndarray | None = None,
228         dynamic_standardizer: StandardizerLite | None = None,
229         dynamic_train_x: np.ndarray | None = None,
230         dynamic_alpha: np.ndarray | None = None,
231     ) -> None:
232         self.spec = spec
233         self.base_weights = None if base_weights is None else np.asarray(base_weights, dtype=float)
234
235         # 单阶段模型参数
236         self.standardizer = standardizer
237         self.coef = coef
238         self.train_x = train_x
239         self.alpha = alpha
240
241         # 多核KRR的原始稳定性核参数。
242         self.raw_standardizer = raw_standardizer
243         self.raw_train_x = raw_train_x
244
245         # 两阶段模型参数
246         self.system_standardizer = system_standardizer
247         self.system_train_x = system_train_x
248         self.system_alpha = system_alpha
249         self.dynamic_standardizer = dynamic_standardizer
250         self.dynamic_train_x = dynamic_train_x
251         self.dynamic_alpha = dynamic_alpha
252
253         self.d0_curve_coef = d0_curve_coef
254
255     @staticmethod
256     def _rbfKernel(x: np.ndarray, y: np.ndarray, gamma: float) -> np.ndarray:
257         x2 = np.sum(x * x, axis=1)[:, None]
258         y2 = np.sum(y * y, axis=1)[None, :]
259         dist2 = np.maximum(x2 + y2 - 2.0 * (x @ y.T), 0.0)
260         return np.exp(-float(gamma) * dist2)
261
262     def _kernel(self, x: np.ndarray, y: np.ndarray) -> np.ndarray:
263         return self._rbfKernel(x, y, self.spec.gamma)
264
265     def baseVolume(self, data: pd.DataFrame) -> np.ndarray:
266         mode = self.spec.base_mode
267
268         if mode == "raw_hampel_phys6":
269             return rawHampelPhys6Volume(
270                 data,
271                 strength=self.spec.raw_base_strength,
272                 clip_pct=self.spec.raw_ratio_clip_pct,
273             )
274
275         matrix = baseMatrix(data)
276         if mode == "median4":
277             return np.median(matrix, axis=1)
278
279         if self.base_weights is None:
280             raise ValueError(f"missing base weights for mode={mode}")
281         return matrix @ self.base_weights
282
283     def correctionComponents(
284         self,
285         features: pd.DataFrame,
```

```
    ) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        """
        返回 (system_or_single, dynamic_raw, total_raw)。

        对单阶段KRR/Ridge，第一项就是单阶段修正，动态项为0。
        对两阶段KRR:
            total = system + dynamic_rho * dynamic_raw
        """
        n = len(features)
        if self.spec.model_type == "baseline":
            zeros = np.zeros(n, dtype=float)
            return zeros, zeros, zeros

        if self.spec.model_type == "multikernel_krr":
            if (
                self.standardizer is None
                or self.train_x is None
                or self.raw_standardizer is None
                or self.raw_train_x is None
                or self.alpha is None
            ):
                raise ValueError("missing multi-kernel KRR parameters")
            physical_x = self.standardizer.transform(features)
            raw_x = self.raw_standardizer.transform(features)
            beta = float(np.clip(self.spec.raw_kernel_weight, 0.0, 1.0))
            kernel_physical = self._rbfKernel(
                physical_x,
                self.train_x,
                self.spec.gamma,
            )
            kernel_raw = self._rbfKernel(
                raw_x,
                self.raw_train_x,
                self.spec.raw_gamma,
            )
            single = ((1.0 - beta) * kernel_physical + beta * kernel_raw) @ self.alpha
            zeros = np.zeros(n, dtype=float)
            return single, zeros, single

        if self.spec.model_type == "two_stage_krr":
            if (
                self.system_standardizer is None
                or self.system_train_x is None
                or self.system_alpha is None
            ):
                raise ValueError("missing two-stage system KRR parameters")
            system_x = self.system_standardizer.transform(features)
            system = (
                self._rbfKernel(system_x, self.system_train_x, self.spec.system_gamma)
                @ self.system_alpha
            )

            dynamic = np.zeros(n, dtype=float)
            if (
                self.spec.dynamic_rho > 0.0
                and self.dynamic_standardizer is not None
                and self.dynamic_train_x is not None
                and self.dynamic_alpha is not None
            ):
                dynamic_x = self.dynamic_standardizer.transform(features)
                dynamic = (
                    self._rbfKernel(dynamic_x, self.dynamic_train_x, self.spec.dynamic_gamma)
                    @ self.dynamic_alpha
                )
            total = system + float(self.spec.dynamic_rho) * dynamic
            return system, dynamic, total

        if self.standardizer is None:
            raise ValueError("missing standardizer")
        x = self.standardizer.transform(features)

        if self.spec.model_type == "ridge":
            if self.coef is None:
                raise ValueError("missing ridge coefficients")
            single = x @ self.coef
        elif self.spec.model_type == "krr_rbf":
            if self.train_x is None or self.alpha is None:
                raise ValueError("missing KRR parameters")
            single = self._kernel(x, self.train_x) @ self.alpha
        else:
            raise ValueError(f"unsupported model_type: {self.spec.model_type}")

        zeros = np.zeros(n, dtype=float)
        return single, zeros, single

    def rawCorrectionPct(self, features: pd.DataFrame) -> np.ndarray:
        _, _, total = self.correctionComponents(features)
        return total

    def d0PostCurve(self, features: pd.DataFrame) -> np.ndarray:
        if self.d0_curve_coef is None or not self.spec.blend_d0_curve:
            return np.zeros(len(features), dtype=float)
        q = features["q"].to_numpy(dtype=float)
```

```
        degree = max(int(self.spec.d0_curve_degree), 0)
        design = np.column_stack([q**power for power in range(degree + 1)])
        return design @ self.d0_curve_coef

    def predictWithDiagnostics(
        self,
        data: pd.DataFrame,
        features: pd.DataFrame,
    ) -> tuple[np.ndarray, pd.DataFrame]:
        base = self.baseVolume(data)
        system_or_single, dynamic_raw, raw_total = self.correctionComponents(features)
        raw_total = np.clip(
            raw_total,
            -self.spec.correction_clip_pct,
            self.spec.correction_clip_pct,
        )
        d0_post = self.d0PostCurve(features)
        correction = np.clip(
            raw_total + d0_post,
            -self.spec.correction_clip_pct,
            self.spec.correction_clip_pct,
        )
        yuceJieguo = base * (1.0 + correction / 100.0)

        matrix = baseMatrix(data)
        diagnostics = pd.DataFrame(
            {
                "window_id": data["window_id"].to_numpy(),
                "phys6_volume_m3": matrix[:, 0],
                "lagrange_volume_m3": matrix[:, 1],
                "owics_volume_m3": matrix[:, 2],
                "equal_weight_volume_m3": matrix[:, 3],
                "selected_base_volume_m3": base,
                "system_or_single_correction_pct": system_or_single,
                "dynamic_raw_correction_pct": dynamic_raw,
                "dynamic_scaled_correction_pct": float(self.spec.dynamic_rho) * dynamic_raw,
                "raw_krr_correction_pct": raw_total,
                "d0_post_correction_pct": d0_post,
                "total_correction_pct": correction,
                "model_volume_m3": yuceJieguo,
            }
        )
        return yuceJieguo, diagnostics

    def predictVolume(self, data: pd.DataFrame, features: pd.DataFrame) -> np.ndarray:
        yuceJieguo, _ = self.predictWithDiagnostics(data, features)
        return yuceJieguo

# ---------------------------------------------------------------------------
# 附件6逐秒序列稳健特征
# ---------------------------------------------------------------------------


def findProjectDataFile(filename: str) -> Path:
    candidates = [ROOT / filename]
    candidates.extend(ROOT.glob(f"*/{filename}"))
    candidates.extend(ROOT.glob(f"*/*/{filename}"))
    for path in candidates:
        if path.exists():
            return path
    raise FileNotFoundError(
        f"cannot find {filename}; expected it in the project root or a first/second-level data directory"
    )


def hampelFilter(values: np.ndarray, window: int = 9, n_sigma: float = 4.0) -> tuple[np.ndarray, np.ndarray]:
    """Causal information is not used; this is an offline symmetric robust window statistic."""
    series = pd.Series(np.asarray(values, dtype=float))
    median = series.rolling(window, center=True, min_periods=3).median()
    deviation = (series - median).abs()
    mad = deviation.rolling(window, center=True, min_periods=3).median()
    threshold = n_sigma * 1.4826 * mad
    valid_threshold = threshold.where(threshold > 1e-12, np.inf)
    mask = (deviation > valid_threshold).fillna(False)
    filtered = series.copy()
    filtered.loc[mask] = median.loc[mask]
    return filtered.to_numpy(dtype=float), mask.to_numpy(dtype=bool)


def safeRelative(numerator: np.ndarray | float, denominator: np.ndarray | float) -> np.ndarray:
    num = np.asarray(numerator, dtype=float)
    den = np.asarray(denominator, dtype=float)
    return num / np.maximum(np.abs(den), 1e-12)


def normalisedSlope(time: np.ndarray, values: np.ndarray, location: float) -> float:
    t = np.asarray(time, dtype=float)
    y = np.asarray(values, dtype=float)
    finite = np.isfinite(t) & np.isfinite(y)
    if int(finite.sum()) < 3:
        return 0.0
    t = t[finite]
    y = y[finite]
```

```
    t = t - float(t.mean())
    denom = float(t @ t)
    if denom <= 1e-12:
        return 0.0
    slope = float(t @ (y - float(y.mean()))) / denom
    return slope / max(abs(float(location)), 1e-12)


def buildRawWindowFeatures(raw: pd.DataFrame, geo: pd.DataFrame) -> pd.DataFrame:
    """
    从附件6提取完全标签无关的逐窗稳健特征。

    Hampel滤波只替换局部尖峰；整窗均值仍保留启停过程，因此不会把平台流速
    乘以整窗时长。若附件7映射可用，则用滤波前后五声道Phys6积分信号之比
    构造 raw_phys6_hampel_ratio。
    """
    diff_columns = [f"diff_ns_{i}" for i in range(10) if f"diff_ns_{i}" in raw.columns]
    mean_columns = [f"mean_us_{i}" for i in range(10) if f"mean_us_{i}" in raw.columns]
    if len(diff_columns) != 10:
        raise ValueError(f"attachment6 needs diff_ns_0..9, got {len(diff_columns)} columns")

    mapping = pd.DataFrame()
    if "record_type" in geo.columns:
        mapping = geo[geo["record_type"].eq("chord_mapping")].copy()
    required_mapping = {
        "raw_diff_col_a",
        "raw_diff_col_b",
        "raw_diff_gain_a_mps_per_ns",
        "raw_diff_gain_b_mps_per_ns",
        "raw_diff_intercept_mps",
        "weight_phys6",
    }
    mapping_ok = (not mapping.empty) and required_mapping.issubset(mapping.columns)
    if mapping_ok and "chord_id" in mapping.columns:
        mapping = mapping.sort_values("chord_id").reset_index(drop=True)

    rows: list[dict[str, float | str]] = []
    for window_id, group in raw.groupby("window_id", sort=False):
        group = group.sort_values("rel_time_s").reset_index(drop=True)
        time = group["rel_time_s"].to_numpy(dtype=float)
        original = group[diff_columns].to_numpy(dtype=float)
        n = len(group)

        filtered = np.empty_like(original)
        replaced = np.zeros_like(original, dtype=bool)
        for j in range(original.shape[1]):
            filtered[:, j], replaced[:, j] = hampelFilter(original[:, j])

        lo = int(math.floor(0.15 * n))
        hi = int(math.ceil(0.85 * n))
        if hi - lo < 20:
            lo, hi = 0, n
        centre = filtered[lo:hi]
        centre_time = time[lo:hi]

        centre_median = np.nanmedian(centre, axis=0)
        centre_mad = np.nanmedian(np.abs(centre - centre_median), axis=0)
        q25 = np.nanpercentile(centre, 25.0, axis=0)
        q75 = np.nanpercentile(centre, 75.0, axis=0)
        rel_mad = safeRelative(centre_mad, centre_median)
        rel_iqr = safeRelative(q75 - q25, centre_median)

        edge_n = max(5, int(round(0.15 * n)))
        first_median = np.nanmedian(filtered[:edge_n], axis=0)
        last_median = np.nanmedian(filtered[-edge_n:], axis=0)
        drift = safeRelative(np.abs(last_median - first_median), centre_median)
        slopes = np.asarray(
            [
                abs(normalisedSlope(centre_time, centre[:, j], centre_median[j]))
                for j in range(centre.shape[1])
            ],
            dtype=float,
        )

        centred = centre - np.nanmedian(centre, axis=0)
        valid_columns = np.nanstd(centred, axis=0) > 1e-12
        sync_corr = 0.0
        if int(valid_columns.sum()) >= 2 and len(centre) >= 3:
            corr = np.corrcoef(centred[:, valid_columns], rowvar=False)
            upper = corr[np.triu_indices_from(corr, k=1)]
            if np.isfinite(upper).any():
                sync_corr = float(np.nanmedian(upper))

        mean_rel_mad = 0.0
        mean_drift = 0.0
        if mean_columns:
            mean_values = group[mean_columns].to_numpy(dtype=float)
            mean_centre = mean_values[lo:hi]
            mean_location = np.nanmedian(mean_centre, axis=0)
            mean_mad = np.nanmedian(np.abs(mean_centre - mean_location), axis=0)
            mean_rel_mad = float(np.nanmedian(safeRelative(mean_mad, mean_location)))
            mean_first = np.nanmedian(mean_values[:edge_n], axis=0)
            mean_last = np.nanmedian(mean_values[-edge_n:], axis=0)
```

```
            mean_drift = float(np.nanmedian(safeRelative(np.abs(mean_last - mean_first), mean_location)))

        temperature_mad = 0.0
        if "water_temp_c" in group.columns:
            temperature = group["water_temp_c"].to_numpy(dtype=float)
            temp_med = float(np.nanmedian(temperature))
            temperature_mad = float(np.nanmedian(np.abs(temperature - temp_med)))

        feature: dict[str, float | str] = {
            "window_id": str(window_id),
            "raw_diff_rel_mad_med": float(np.nanmedian(rel_mad)),
            "raw_diff_rel_mad_max": float(np.nanmax(rel_mad)),
            "raw_diff_rel_iqr_med": float(np.nanmedian(rel_iqr)),
            "raw_diff_drift_med": float(np.nanmedian(drift)),
            "raw_diff_slope_abs_med": float(np.nanmedian(slopes)),
            "raw_diff_sync_corr_med": sync_corr,
            "raw_hampel_replace_rate_mean": float(replaced.mean()),
            "raw_hampel_replace_rate_max": float(replaced.mean(axis=0).max()),
            "raw_mean_us_rel_mad_med": mean_rel_mad,
            "raw_mean_us_drift_med": mean_drift,
            "raw_temperature_mad": temperature_mad,
            "raw_chord_rel_mad_med": 0.0,
            "raw_chord_rel_mad_max": 0.0,
            "raw_ab_mismatch_med": 0.0,
            "raw_profile_center_edges": 0.0,
            "raw_profile_edge_inner": 0.0,
            "raw_profile_top_bottom": 0.0,
            "raw_profile_range": 0.0,
            "raw_phys6_hampel_ratio": 1.0,
            "raw_phys6_hampel_delta_pct": 0.0,
        }

        if mapping_ok:
            full_chords: list[float] = []
            clean_chords: list[float] = []
            chord_rel_mad: list[float] = []
            ab_mismatch: list[float] = []
            weights: list[float] = []

            diff_index = {column: i for i, column in enumerate(diff_columns)}
            for _, map_row in mapping.iterrows():
                a_column = str(map_row["raw_diff_col_a"])
                b_column = str(map_row["raw_diff_col_b"])
                if a_column not in diff_index or b_column not in diff_index:
                    continue
                ia = diff_index[a_column]
                ib = diff_index[b_column]
                gain_a = float(map_row["raw_diff_gain_a_mps_per_ns"])
                gain_b = float(map_row["raw_diff_gain_b_mps_per_ns"])
                intercept = float(map_row["raw_diff_intercept_mps"])

                full_series = gain_a * original[:, ia] + gain_b * original[:, ib] + intercept
                clean_series = gain_a * filtered[:, ia] + gain_b * filtered[:, ib] + intercept
                clean_centre = clean_series[lo:hi]
                chord_location = float(np.nanmedian(clean_centre))
                chord_mad = float(np.nanmedian(np.abs(clean_centre - chord_location)))

                full_chords.append(float(np.nanmean(full_series)))
                clean_chords.append(float(np.nanmean(clean_series)))
                chord_rel_mad.append(float(chord_mad / max(abs(chord_location), 1e-12)))

                a_contribution = gain_a * filtered[lo:hi, ia]
                b_contribution = gain_b * filtered[lo:hi, ib]
                denominator = np.maximum(np.abs(clean_centre), 1e-12)
                ab_mismatch.append(float(np.nanmedian(np.abs(a_contribution - b_contribution) / denominator)))
                weights.append(float(map_row["weight_phys6"]))

            if len(clean_chords) >= 3:
                full_array = np.asarray(full_chords, dtype=float)
                clean_array = np.asarray(clean_chords, dtype=float)
                weight_array = np.asarray(weights, dtype=float)
                full_signal = float(weight_array @ full_array)
                clean_signal = float(weight_array @ clean_array)
                if abs(full_signal) > 1e-12 and np.isfinite(clean_signal):
                    ratio = clean_signal / full_signal
                    if np.isfinite(ratio):
                        feature["raw_phys6_hampel_ratio"] = float(ratio)
                        feature["raw_phys6_hampel_delta_pct"] = float((ratio - 1.0) * 100.0)

                chord_rel = np.asarray(chord_rel_mad, dtype=float)
                feature["raw_chord_rel_mad_med"] = float(np.nanmedian(chord_rel))
                feature["raw_chord_rel_mad_max"] = float(np.nanmax(chord_rel))
                feature["raw_ab_mismatch_med"] = float(np.nanmedian(np.asarray(ab_mismatch, dtype=float)))

                # 映射通常按 chord_id 0..4 排序。只有五声道齐全时才计算剖面量。
                if len(clean_array) >= 5:
                    c0, c1, c2, c3, c4 = clean_array[:5]
                    scale = max(abs(float(np.mean(clean_array[:5]))), 1e-12)
                    feature["raw_profile_center_edges"] = float((c2 - 0.5 * (c0 + c4)) / scale)
                    feature["raw_profile_edge_inner"] = float((0.5 * (c0 + c4) - 0.5 * (c1 + c3)) / scale)
                    feature["raw_profile_top_bottom"] = float((c0 - c4) / scale)
                    feature["raw_profile_range"] = float((np.max(clean_array[:5]) - np.min(clean_array[:5])) / scale)

```

```
        rows.append(feature)

    result = pd.DataFrame(rows)
    for column in RAW_FEATURE_COLUMNS:
        if column not in result.columns:
            result[column] = 0.0
        result[column] = pd.to_numeric(result[column], errors="coerce").replace([np.inf, -np.inf], np.nan)
        median = float(result[column].median()) if result[column].notna().any() else 0.0
        result[column] = result[column].fillna(median)
    return result[["window_id", *RAW_FEATURE_COLUMNS]]


def loadDataWithRawFeatures() -> pd.DataFrame:
    # 先读取附件1主表，再合并附件6构造的窗口级稳健特征。
    # 这里必须调用导入的 load_data()，不能递归调用本函数。
    data = loadData().reset_index(drop=True)
    raw_path = findProjectDataFile("attachment6_window_raw_samples.csv")
    geo_path = findProjectDataFile("attachment7_meter_geometry.csv")
    raw = pd.read_csv(raw_path, encoding="utf-8-sig")
    geo = pd.read_csv(geo_path, encoding="utf-8-sig")
    raw_features = buildRawWindowFeatures(raw, geo)
    merged = data.merge(raw_features, on="window_id", how="left", validate="one_to_one")
    if merged[RAW_FEATURE_COLUMNS].isna().any().any():
        missing_windows = int(merged[RAW_FEATURE_COLUMNS].isna().any(axis=1).sum())
        raise ValueError(f"raw features missing for {missing_windows} windows")
    raw_features.to_csv(
        OUT_DIR / "raw_window_robust_features.csv",
        index=False,
        encoding="utf-8-sig",
    )
    return merged


def modelFeatureFrame(frame: pd.DataFrame) -> pd.DataFrame:
    # 先生成项目原有的窗口级增强特征，再附加附件6提取的稳健特征。
    # 这里必须调用 enhanced_feature_frame()，不能递归调用本函数。
    features = enhancedFeatureFrame(frame).copy()
    for column in RAW_FEATURE_COLUMNS:
        if column in frame.columns:
            features[column] = frame[column].to_numpy()
    return features


def rawHampelPhys6Volume(
    data: pd.DataFrame,
    strength: float,
    clip_pct: float,
) -> np.ndarray:
    phys6 = data["phys6_volume_m3"].to_numpy(dtype=float)
    if "raw_phys6_hampel_ratio" not in data.columns:
        return phys6.copy()
    ratio = pd.to_numeric(data["raw_phys6_hampel_ratio"], errors="coerce").fillna(1.0).to_numpy(dtype=float)
    half_width = max(float(clip_pct), 0.0) / 100.0
    ratio = np.clip(ratio, 1.0 - half_width, 1.0 + half_width)
    adjusted_ratio = 1.0 + float(strength) * (ratio - 1.0)
    return phys6 * adjusted_ratio


# ---------------------------------------------------------------------------
# 物理基线融合
# ---------------------------------------------------------------------------


def baseMatrix(data: pd.DataFrame) -> np.ndarray:
    missing = [column for column in BASE_COLUMNS if column not in data.columns]
    if missing:
        raise ValueError(f"missing base volume columns: {missing}")
    frame = data[BASE_COLUMNS].replace([np.inf, -np.inf], np.nan).copy()
    for column in BASE_COLUMNS:
        values = pd.to_numeric(frame[column], errors="coerce")
        median = float(values.median()) if values.notna().any() else 0.0
        frame[column] = values.fillna(median)
    return frame.to_numpy(dtype=float)


def normaliseWeights(values: Iterable[float]) -> np.ndarray:
    weights = np.asarray(list(values), dtype=float)
    if weights.shape != (len(BASE_COLUMNS),):
        raise ValueError(f"weights must have length {len(BASE_COLUMNS)}")
    weights = np.maximum(weights, 0.0)
    total = float(weights.sum())
    if total <= 1e-12:
        weights = np.zeros(len(BASE_COLUMNS), dtype=float)
        weights[0] = 1.0
        return weights
    return weights / total


def projectSimplex(vector: np.ndarray) -> np.ndarray:
    """Euclidean projection onto {w >= 0, sum(w)=1}."""
    v = np.asarray(vector, dtype=float)
    if v.ndim != 1:
        raise ValueError("simplex projection expects a vector")
```

```
    u = np.sort(v)[::-1]
    cssv = np.cumsum(u) - 1.0
    indices = np.arange(1, len(v) + 1)
    valid = u - cssv / indices > 0
    if not np.any(valid):
        result = np.zeros_like(v)
        result[int(np.argmax(v))] = 1.0
        return result
    rho = int(indices[valid][-1])
    theta = float(cssv[valid][-1] / rho)
    return np.maximum(v - theta, 0.0)


def officialGroupBalanceWeights(data: pd.DataFrame, enabled: bool) -> np.ndarray:
    n = len(data)
    if not enabled or n == 0:
        return np.ones(n, dtype=float)
    keys = [column for column in OFFICIAL_GROUP_KEY if column in data.columns]
    if not keys:
        return np.ones(n, dtype=float)
    reset = data[keys].reset_index(drop=True)
    sizes = reset.groupby(keys, sort=False)[keys[0]].transform("size").to_numpy(dtype=float)
    weights = 1.0 / np.maximum(sizes, 1.0)
    weights /= max(float(weights.mean()), 1e-12)
    return weights


def repeatabilityLaplacian(data: pd.DataFrame) -> np.ndarray:
    """Block-centering matrix for official (date, flow_point) repeatability groups."""
    n = len(data)
    lap = np.zeros((n, n), dtype=float)
    keys = [column for column in OFFICIAL_GROUP_KEY if column in data.columns]
    if not keys:
        return lap

    reset = data.reset_index(drop=True)
    for _, index in reset.groupby(keys, sort=False).groups.items():
        ids = np.asarray(list(index), dtype=int)
        m = len(ids)
        if m < 2:
            continue
        center = np.eye(m, dtype=float) - np.ones((m, m), dtype=float) / m
        center /= max(m - 1, 1)
        lap[np.ix_(ids, ids)] += center

    trace = float(np.trace(lap))
    if trace > 1e-12:
        lap *= n / trace
    return lap


def fitConvexBaseWeights(
    spec: CandidateSpec,
    train: pd.DataFrame,
) -> np.ndarray:
    """
    在训练集内拟合非负、和为1的全局物理积分权重。

    目标在百分比误差空间中构造：
        window fit + eta * within-day-flow spread + reg * distance-to-Phys6-prior

    日期和标准体积只参与离线参数估计，不进入运行时公式。
    """
    matrix = baseMatrix(train)
    standard = np.maximum(train["standard_volume_m3"].to_numpy(dtype=float), 1e-12)
    error_matrix_pct = (matrix / standard[:, None] - 1.0) * 100.0

    sample_weights = officialGroupBalanceWeights(train, spec.base_group_balance)
    metric = np.diag(sample_weights)
    if spec.base_repeatability_eta > 0.0:
        metric = metric + spec.base_repeatability_eta * repeatabilityLaplacian(train)

    prior = normaliseWeights(spec.base_prior)
    reg = max(float(spec.base_reg), 0.0)

    hessian = error_matrix_pct.T @ metric @ error_matrix_pct
    if reg > 0.0:
        hessian = hessian + reg * np.eye(len(BASE_COLUMNS), dtype=float)
    linear = reg * prior

    # Projected-gradient solution of a small convex quadratic on the simplex.
    weights = prior.copy()
    largest = float(np.linalg.eigvalsh(0.5 * (hessian + hessian.T)).max())
    step = 1.0 / max(2.0 * largest, 1e-8)
    for _ in range(5000):
        gradient = 2.0 * (hessian @ weights - linear)
        updated = projectSimplex(weights - step * gradient)
        if float(np.linalg.norm(updated - weights)) < 1e-12:
            weights = updated
            break
        weights = updated

    return normaliseWeights(weights)
```

```

def resolveBaseWeights(spec: CandidateSpec, train: pd.DataFrame) -> np.ndarray | None:
    mode = spec.base_mode
    if mode == "median4":
        return None
    if mode in {"phys6", "raw_hampel_phys6"}:
        return np.array([1.0, 0.0, 0.0, 0.0], dtype=float)
    if mode == "lagrange":
        return np.array([0.0, 1.0, 0.0, 0.0], dtype=float)
    if mode == "owics":
        return np.array([0.0, 0.0, 1.0, 0.0], dtype=float)
    if mode == "equal":
        return np.array([0.0, 0.0, 0.0, 1.0], dtype=float)
    if mode == "fixed_blend":
        return normaliseWeights(spec.fixed_weights)
    if mode == "fitted_blend":
        return fitConvexBaseWeights(spec, train)
    raise ValueError(f"unknown base_mode: {mode}")


# ---------------------------------------------------------------------------
# 特征、KRR与后校准
# ---------------------------------------------------------------------------


def columnsForFeatureSet(frame: pd.DataFrame, feature_set: str) -> list[str]:
    if feature_set == "runtime_full":
        return featureColumns(frame, "runtime_full")

    compact = [
        "q",
        "q2",
        "profile_top_bottom",
        "profile_center_all",
        "profile_edge_inner",
        "profile_ab_abs",
        "profile_swirl",
        "chord_center_minus_edges_frac",
        "chord_edge_minus_inner_frac",
        "chord_top_bottom_frac",
        "ab_abs_mean",
        "dyn_plateau_cv",
        "zero_rate_med",
        "lagrange_volume_m3_minus_phys6_frac",
        "owics_volume_m3_minus_phys6_frac",
    ]

    # 第一阶段只学习跨工况系统偏差，避免把高频动态特征直接混入。
    system_physical = [
        "q",
        "q2",
        "profile_top_bottom",
        "profile_center_all",
        "profile_edge_inner",
        "profile_ab_abs",
        "profile_swirl",
        "chord_center_minus_edges_frac",
        "chord_edge_minus_inner_frac",
        "chord_top_bottom_frac",
        "ab_abs_mean",
        "zero_rate_med",
        "lagrange_volume_m3_minus_phys6_frac",
        "owics_volume_m3_minus_phys6_frac",
    ]

    # 第二阶段只学习组内可预测的窗口变化，并用rho强收缩。
    dynamic_raw = [
        "q",
        "dyn_plateau_cv",
        "profile_top_bottom",
        "profile_center_all",
        "profile_ab_abs",
        "profile_swirl",
        "ab_abs_mean",
        "zero_rate_med",
        *RAW_MODEL_FEATURE_COLUMNS,
    ]

    if feature_set == "compact_physical":
        columns = compact
    elif feature_set == "compact_plus_raw":
        columns = compact + RAW_MODEL_FEATURE_COLUMNS
    elif feature_set == "system_physical":
        columns = system_physical
    elif feature_set == "dynamic_raw":
        columns = dynamic_raw
    elif feature_set == "raw_stability_only":
        columns = RAW_MODEL_FEATURE_COLUMNS
    elif feature_set == "raw_variability_only":
        columns = RAW_VARIABILITY_FEATURE_COLUMNS
    elif feature_set == "raw_drift_sync_only":
        columns = RAW_DRIFT_SYNC_FEATURE_COLUMNS
```

```
    elif feature_set == "raw_no_hampel":
        columns = RAW_NO_HAMPEL_FEATURE_COLUMNS
    elif feature_set == "raw_hampel_only":
        columns = RAW_HAMPEL_FEATURE_COLUMNS
    else:
        raise ValueError(f"unknown feature_set: {feature_set}")

    # 去重并保持顺序。
    return list(dict.fromkeys(column for column in columns if column in frame.columns))


def residualTargetPct(
    model: FittedCandidate,
    data: pd.DataFrame,
) -> np.ndarray:
    base = np.maximum(model.baseVolume(data), 1e-12)
    standard = data["standard_volume_m3"].to_numpy(dtype=float)
    return (standard / base - 1.0) * 100.0


def stableLinearSolve(matrix: np.ndarray, rhs: np.ndarray) -> np.ndarray:
    matrix = 0.5 * (matrix + matrix.T)
    try:
        return np.linalg.solve(matrix, rhs)
    except np.linalg.LinAlgError:
        return np.linalg.lstsq(matrix, rhs, rcond=1e-12)[0]


def rbfKernel(x: np.ndarray, y: np.ndarray, gamma: float) -> np.ndarray:
    return FittedCandidate._rbfKernel(x, y, gamma)


def fitWeightedKrr(
    x: np.ndarray,
    y: np.ndarray,
    gamma: float,
    lam: float,
    sample_weights: np.ndarray | None = None,
) -> tuple[np.ndarray, np.ndarray]:
    """
    对角样本权重的对称KRR实现。

    令W=diag(w)，求解：
        (sqrt(W) K sqrt(W) + lambda I) beta = sqrt(W) y
    预测系数 alpha = sqrt(W) beta。
    """
    kernel = rbfKernel(x, x, gamma)
    if sample_weights is None:
        alpha = stableLinearSolve(
            kernel + float(lam) * np.eye(len(kernel), dtype=float),
            y,
        )
        return kernel, alpha

    weights = np.asarray(sample_weights, dtype=float)
    weights = np.maximum(weights, 1e-12)
    weights /= max(float(weights.mean()), 1e-12)
    sqrt_w = np.sqrt(weights)
    weighted_kernel = sqrt_w[:, None] * kernel * sqrt_w[None, :]
    beta = stableLinearSolve(
        weighted_kernel + float(lam) * np.eye(len(kernel), dtype=float),
        sqrt_w * y,
    )
    alpha = sqrt_w * beta
    return kernel, alpha


def fitWeightedKrrFromKernel(
    kernel: np.ndarray,
    y: np.ndarray,
    lam: float,
    sample_weights: np.ndarray | None = None,
) -> np.ndarray:
    """对预计算的对称核矩阵执行带权KRR。"""
    kernel = np.asarray(kernel, dtype=float)
    y = np.asarray(y, dtype=float)
    if sample_weights is None:
        return stableLinearSolve(
            kernel + float(lam) * np.eye(len(kernel), dtype=float),
            y,
        )

    weights = np.maximum(np.asarray(sample_weights, dtype=float), 1e-12)
    weights /= max(float(weights.mean()), 1e-12)
    sqrt_w = np.sqrt(weights)
    weighted_kernel = sqrt_w[:, None] * kernel * sqrt_w[None, :]
    beta = stableLinearSolve(
        weighted_kernel + float(lam) * np.eye(len(kernel), dtype=float),
        sqrt_w * y,
    )
    return sqrt_w * beta


```

```
def shrinkTargetTowardGroupMean(
    data: pd.DataFrame,
    target: np.ndarray,
    strength: float,
) -> np.ndarray:
    """
    将窗口标签小幅收缩到训练试验组均值，但仍保留全部窗口样本：
        y*=(1-rho)y+rho*mean_group(y)
    """
    rho = float(np.clip(strength, 0.0, 1.0))
    y = np.asarray(target, dtype=float)
    if rho <= 0.0:
        return y.copy()
    keys = [key for key in SPLIT_GROUP_KEY if key in data.columns]
    temp = data[keys].reset_index(drop=True).copy()
    temp["_target"] = y
    group_mean = temp.groupby(keys, sort=False)["_target"].transform("mean").to_numpy(dtype=float)
    return (1.0 - rho) * y + rho * group_mean


def groupBalanceWeights(
    data: pd.DataFrame,
    keys: list[str],
) -> np.ndarray:
    if len(data) == 0:
        return np.ones(0, dtype=float)
    usable = [key for key in keys if key in data.columns]
    if not usable:
        return np.ones(len(data), dtype=float)
    reset = data[usable].reset_index(drop=True)
    sizes = reset.groupby(usable, sort=False)[usable[0]].transform("size").to_numpy(dtype=float)
    weights = 1.0 / np.maximum(sizes, 1.0)
    weights /= max(float(weights.mean()), 1e-12)
    return weights


def groupMeanSystemTraining(
    spec: CandidateSpec,
    train: pd.DataFrame,
    xunlianTezheng: pd.DataFrame,
    y: np.ndarray,
) -> tuple[pd.DataFrame, np.ndarray, np.ndarray]:
    """
    将训练窗口压缩为 (date, flow_point, disturbance_id) 组均值样本。

    对小组目标向同流量点训练均值做经验收缩：
        y_shrunk = (n*y_group + k*y_flow_prior)/(n+k)
    其中k=system_group_shrinkage。
    """
    columns = columnsForFeatureSet(xunlianTezheng, spec.system_feature_set)
    keys = [key for key in SPLIT_GROUP_KEY if key in train.columns]
    if not keys:
        raise ValueError("missing split group keys for two-stage system model")

    temp = train[keys].reset_index(drop=True).copy()
    for column in columns:
        temp[column] = xunlianTezheng[column].to_numpy(dtype=float)
    temp["_target"] = np.asarray(y, dtype=float)

    aggregations = {column: "mean" for column in columns}
    aggregations["_target"] = "mean"
    grouped = temp.groupby(keys, sort=True).agg(aggregations)
    grouped["_n"] = temp.groupby(keys, sort=True).size().astype(float)
    grouped = grouped.reset_index()

    flow_prior = grouped.groupby("flow_point", sort=False)["_target"].transform("mean")
    shrink = max(float(spec.system_group_shrinkage), 0.0)
    if shrink > 0.0:
        target = (
            grouped["_n"].to_numpy(dtype=float) * grouped["_target"].to_numpy(dtype=float)
            + shrink * flow_prior.to_numpy(dtype=float)
        ) / (grouped["_n"].to_numpy(dtype=float) + shrink)
    else:
        target = grouped["_target"].to_numpy(dtype=float)

    return grouped[columns].copy(), target, grouped["_n"].to_numpy(dtype=float)


def centerTargetWithinGroups(
    data: pd.DataFrame,
    target: np.ndarray,
    keys: list[str],
) -> np.ndarray:
    reset = data[keys].reset_index(drop=True).copy()
    reset["_target"] = np.asarray(target, dtype=float)
    group_mean = reset.groupby(keys, sort=False)["_target"].transform("mean").to_numpy(dtype=float)
    return np.asarray(target, dtype=float) - group_mean


def fitCandidate(
    spec: CandidateSpec,
    train: pd.DataFrame,
    xunlianTezheng: pd.DataFrame,
```

```
) -> FittedCandidate:
    base_weights = resolveBaseWeights(spec, train)
    model = FittedCandidate(spec=spec, base_weights=base_weights)

    if spec.model_type == "baseline":
        return model

    y = residualTargetPct(model, train)

    if spec.model_type == "multikernel_krr":
        physical_columns = columnsForFeatureSet(
            xunlianTezheng,
            spec.feature_set,
        )
        raw_columns = columnsForFeatureSet(
            xunlianTezheng,
            spec.raw_feature_set,
        )
        physical_standardizer = StandardizerLite.fit(
            xunlianTezheng,
            physical_columns,
        )
        raw_standardizer = StandardizerLite.fit(
            xunlianTezheng,
            raw_columns,
        )
        physical_x = physical_standardizer.transform(xunlianTezheng)
        raw_x = raw_standardizer.transform(xunlianTezheng)

        beta = float(np.clip(spec.raw_kernel_weight, 0.0, 1.0))
        kernel = (
            (1.0 - beta) * rbfKernel(physical_x, physical_x, spec.gamma)
            + beta * rbfKernel(raw_x, raw_x, spec.raw_gamma)
        )
        fitted_target = shrinkTargetTowardGroupMean(
            train,
            y,
            spec.target_group_shrinkage,
        )
        sample_weights = (
            groupBalanceWeights(train, SPLIT_GROUP_KEY)
            if spec.sample_group_balance
            else None
        )
        alpha = fitWeightedKrrFromKernel(
            kernel,
            fitted_target,
            lam=spec.lam,
            sample_weights=sample_weights,
        )

        model.standardizer = physical_standardizer
        model.train_x = physical_x
        model.raw_standardizer = raw_standardizer
        model.raw_train_x = raw_x
        model.alpha = alpha
        fitDOPostCalibration(model, train, xunlianTezheng)
        return model

    if spec.model_type == "two_stage_krr":
        # 第一阶段：用试验组均值学习系统偏差。
        group_features, group_target, _ = groupMeanSystemTraining(
            spec,
            train,
            xunlianTezheng,
            y,
        )
        system_columns = columnsForFeatureSet(group_features, spec.system_feature_set)
        system_standardizer = StandardizerLite.fit(group_features, system_columns)
        system_x = system_standardizer.transform(group_features)
        _, system_alpha = fitWeightedKrr(
            system_x,
            group_target,
            gamma=spec.system_gamma,
            lam=spec.system_lam,
            sample_weights=None,  # 每个试验组等权
        )

        model.system_standardizer = system_standardizer
        model.system_train_x = system_x
        model.system_alpha = system_alpha

        # 第二阶段：只学习系统模型尚未解释的组内窗口偏差。
        if spec.dynamic_rho > 0.0:
            train_system_x = system_standardizer.transform(xunlianTezheng)
            system_prediction = (
                rbfKernel(train_system_x, system_x, spec.system_gamma)
                @ system_alpha
            )
            dynamic_target = y - system_prediction
            if spec.dynamic_center_target:
                dynamic_target = centerTargetWithinGroups(
                    train,
```

```
                    dynamic_target,
                    [key for key in SPLIT_GROUP_KEY if key in train.columns],
                )

            dynamic_columns = columnsForFeatureSet(
                xunlianTezheng,
                spec.dynamic_feature_set,
            )
            dynamic_standardizer = StandardizerLite.fit(
                xunlianTezheng,
                dynamic_columns,
            )
            dynamic_x = dynamic_standardizer.transform(xunlianTezheng)
            dynamic_weights = (
                groupBalanceWeights(train, SPLIT_GROUP_KEY)
                if spec.dynamic_group_balance
                else None
            )
            _, dynamic_alpha = fitWeightedKrr(
                dynamic_x,
                dynamic_target,
                gamma=spec.dynamic_gamma,
                lam=spec.dynamic_lam,
                sample_weights=dynamic_weights,
            )
            model.dynamic_standardizer = dynamic_standardizer
            model.dynamic_train_x = dynamic_x
            model.dynamic_alpha = dynamic_alpha

        fitDOPostCalibration(model, train, xunlianTezheng)
        return model

    # 单阶段基准模型。
    columns = columnsForFeatureSet(xunlianTezheng, spec.feature_set)
    standardizer = StandardizerLite.fit(xunlianTezheng, columns)
    x = standardizer.transform(xunlianTezheng)

    fitted_target = shrinkTargetTowardGroupMean(
        train,
        y,
        spec.target_group_shrinkage,
    )
    sample_weights = (
        groupBalanceWeights(train, SPLIT_GROUP_KEY)
        if spec.sample_group_balance
        else None
    )

    if spec.model_type == "ridge":
        penalty = spec.lam * np.eye(x.shape[1], dtype=float)
        coef = stableLinearSolve(x.T @ x + penalty, x.T @ fitted_target)
        model.standardizer = standardizer
        model.coef = coef
    elif spec.model_type == "krr_rbf":
        model.standardizer = standardizer
        _, alpha = fitWeightedKrr(
            x,
            fitted_target,
            gamma=spec.gamma,
            lam=spec.lam,
            sample_weights=sample_weights,
        )
        model.train_x = x
        model.alpha = alpha
    else:
        raise ValueError(f"unsupported model_type: {spec.model_type}")

    fitDOPostCalibration(model, train, xunlianTezheng)
    return model


def fitDOPostCalibration(
    model: FittedCandidate,
    train: pd.DataFrame,
    xunlianTezheng: pd.DataFrame,
) -> None:
    if not model.spec.blend_d0_curve or model.spec.model_type == "baseline":
        return

    base = model.baseVolume(train)
    raw = np.clip(
        model.rawCorrectionPct(xunlianTezheng),
        -model.spec.correction_clip_pct,
        model.spec.correction_clip_pct,
    )
    yuceJieguo = base * (1.0 + raw / 100.0)
    standard = train["standard_volume_m3"].to_numpy(dtype=float)
    errorPct = (yuceJieguo - standard) / np.maximum(standard, 1e-12) * 100.0

    d0_mask = train["condition_note"].eq("no_disturbance_reference").to_numpy()
    if int(d0_mask.sum()) < 3:
        return

```

```
    q = xunlianTezheng.loc[d0_mask, "q"].to_numpy(dtype=float)
    degree = max(int(model.spec.d0_curve_degree), 0)
    design = np.column_stack([q**power for power in range(degree + 1)])
    target = -errorPct[d0_mask]
    penalty = float(model.spec.d0_curve_lam) * np.eye(design.shape[1], dtype=float)
    model.d0_curve_coef = stableLinearSolve(
        design.T @ design + penalty,
        design.T @ target,
    )

# ---------------------------------------------------------------------------
# 标签盲划分
# ---------------------------------------------------------------------------


def chooseN5TrainIndices(indices: np.ndarray, rng: np.random.Generator) -> set[int]:
    """不读取标签和基础误差，随机选两个窗口进入训练。"""
    idx = np.asarray(indices, dtype=int)
    if len(idx) <= 2:
        return set(int(i) for i in idx)
    return set(int(i) for i in rng.choice(idx, size=2, replace=False))


def splitAssignments(
    data: pd.DataFrame,
    spec: SplitSpec,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    rng = np.random.default_rng(spec.seed)
    target_validation = int(round(len(data) * TARGET_VALIDATION_FRACTION))
    train_idx: set[int] = set()
    validation_idx: set[int] = set()

    groups = list(data.groupby(SPLIT_GROUP_KEY, sort=True))

    # D0 很稀疏，延续原脚本的分配逻辑；选择过程不读取 standard_volume。
    for key, group in groups:
        disturbance_id = str(key[2])
        if disturbance_id != "D0":
            continue
        idx = group.index.to_numpy(dtype=int)
        n = len(idx)
        if n >= 5:
            chosen = chooseN5TrainIndices(idx, rng)
            train_idx.update(chosen)
            validation_idx.update(int(i) for i in idx if int(i) not in chosen)
        elif n in {3, 4}:
            if spec.d0_n3_to_validation:
                validation_idx.update(int(i) for i in idx)
            else:
                train_idx.update(int(i) for i in idx)
        elif n == 2:
            chosen = int(rng.choice(idx))
            train_idx.add(chosen)
            validation_idx.update(int(i) for i in idx if int(i) != chosen)
        else:
            if rng.random() < 0.7:
                validation_idx.update(int(i) for i in idx)
            else:
                train_idx.update(int(i) for i in idx)

    n34_candidates: list[tuple[float, np.ndarray]] = []
    small_candidates: list[int] = []
    for key, group in groups:
        if str(key[2]) == "D0":
            continue
        idx = group.index.to_numpy(dtype=int)
        n = len(idx)
        if n >= 5:
            chosen = chooseN5TrainIndices(idx, rng)
            train_idx.update(chosen)
            validation_idx.update(int(i) for i in idx if int(i) not in chosen)
        elif n in {3, 4}:
            flow = int(key[1])
            official_flow = 1.0 if 40 <= flow <= 100 else 0.0
            score = spec.n34_validation_bias * official_flow + rng.random()
            n34_candidates.append((score, idx))
        else:
            small_candidates.extend(int(i) for i in idx)

    n34_candidates = sorted(n34_candidates, key=lambda item: item[0], reverse=True)
    selected_whole_groups = 0
    max_whole_groups = 3 + (spec.seed % 2)
    for _, idx in n34_candidates:
        idx_set = set(int(i) for i in idx)
        if len(validation_idx) < target_validation and selected_whole_groups < max_whole_groups:
            validation_idx.update(idx_set)
            selected_whole_groups += 1
        else:
            train_idx.update(idx_set)

    rng.shuffle(small_candidates)
    for index in small_candidates:
        if len(validation_idx) < target_validation and rng.random() < spec.small_validation_prob:
```

```
            validation_idx.add(int(index))
        else:
            train_idx.add(int(index))

    unassigned = set(range(len(data))) - train_idx - validation_idx
    for index in sorted(unassigned):
        if len(validation_idx) < target_validation:
            validation_idx.add(index)
        else:
            train_idx.add(index)

    overlap = train_idx & validation_idx
    if overlap:
        raise RuntimeError(f"split overlap: {sorted(overlap)[:5]}")

    split = data[["window_id", "date", "flow_point", "condition_note", "disturbance_id"]].copy()
    split["split"] = ["validation" if i in validation_idx else "train" for i in range(len(data))]
    split["split_mode"] = "label_blind_stratified_group_internal"
    split["split_seed"] = spec.seed

    train = data.loc[split["split"].eq("train")].copy().reset_index(drop=True)
    validation = data.loc[split["split"].eq("validation")].copy().reset_index(drop=True)
    return train, validation, split


def validateSplitRules(data: pd.DataFrame, split: pd.DataFrame) -> tuple[bool, str]:
    merged = data[SPLIT_GROUP_KEY].copy()
    merged["split"] = split["split"].to_numpy()
    for key, group in merged.groupby(SPLIT_GROUP_KEY, sort=True):
        n = len(group)
        train_n = int(group["split"].eq("train").sum())
        validation_n = int(group["split"].eq("validation").sum())
        if n >= 5 and not (train_n == 2 and validation_n >= 3):
            return False, f"n>=5 rule failed for {key}: train={train_n}, validation={validation_n}"
        if n in {3, 4} and not (
            (train_n == n and validation_n == 0)
            or (train_n == 0 and validation_n == n)
        ):
            return False, f"n=3/4 whole-group rule failed for {key}: train={train_n}, validation={validation_n}"
    return True, "ok"


def pressureSplit(
    data: pd.DataFrame,
    seed: int,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    rng = np.random.default_rng(seed)
    target_validation = int(round(len(data) * TARGET_VALIDATION_FRACTION))
    validation_idx: set[int] = set()
    candidates: list[tuple[int, int, float, np.ndarray]] = []

    for key, group in data.groupby(SPLIT_GROUP_KEY, sort=True):
        flow = int(key[1])
        priority = 2 if str(key[2]) == "D0" else int(40 <= flow <= 100)
        candidates.append((priority, len(group), rng.random(), group.index.to_numpy(dtype=int)))

    for _, _, _, idx in sorted(candidates, key=lambda item: (-item[0], -item[1], item[2])):
        if len(validation_idx) >= target_validation:
            break
        validation_idx.update(int(i) for i in idx)

    train_idx = set(range(len(data))) - validation_idx
    split = data[["window_id", "date", "flow_point", "condition_note", "disturbance_id"]].copy()
    split["split"] = ["validation" if i in validation_idx else "train" for i in range(len(data))]
    split["split_mode"] = "whole_stratified_group_holdout_pressure"
    split["split_seed"] = seed

    train = data.loc[split["split"].eq("train")].copy().reset_index(drop=True)
    validation = data.loc[split["split"].eq("validation")].copy().reset_index(drop=True)
    return train, validation, split


# ---------------------------------------------------------------------
# 候选模型
# ---------------------------------------------------------------------


def buildSpecs() -> list[CandidateSpec]:
    """
    物理--稳定性双核 KRR 预注册小网格。

    主搜索:
    - raw_gamma: 0.002 / 0.005 / 0.01 / 0.02
    - raw kernel weight beta: 0.35 / 0.50 / 0.65 / 0.80
    - 固定 physical gamma=0.1, lambda=0.3, 线性 D0

    消融:
    - 全部10个附件6特征;
    - 删除Hampel两特征;
    - 仅保留5个波动特征。

    局部稳健性:
    - 对最有希望的平滑原始核提高lambda;
```

```
    - 对少量候选比较 physical gamma;
    - 保留无D0对照，避免后校准造成虚假改善。
    """
    specs: list[CandidateSpec] = [
        CandidateSpec(
            model_id="baseline_phys6",
            family="baseline",
            model_type="baseline",
            base_mode="phys6",
            blend_d0_curve=False,
        ),
        CandidateSpec(
            model_id="krr_phys6_compact_g0.1_lam0.1_d0quad",
            family="kernel_ridge_rbf",
            model_type="krr_rbf",
            base_mode="phys6",
            feature_set="compact_physical",
            gamma=0.1,
            lam=0.1,
            blend_d0_curve=True,
            d0_curve_degree=2,
            d0_curve_lam=0.1,
        ),
        CandidateSpec(
            model_id="krr_phys6_raw_single_g0.03_lam0.3_d0lin",
            family="kernel_ridge_rbf",
            model_type="krr_rbf",
            base_mode="phys6",
            feature_set="compact_plus_raw",
            gamma=0.03,
            lam=0.3,
            blend_d0_curve=True,
            d0_curve_degree=1,
            d0_curve_lam=1.0,
        ),
        CandidateSpec(
            model_id="krr_mk_v51_pg0.1_rg0.01_b0.4_lam0.3_d0lin",
            family="kernel_ridge_rbf",
            model_type="multikernel_krr",
            base_mode="phys6",
            feature_set="compact_physical",
            raw_feature_set="raw_stability_only",
            gamma=0.1,
            raw_gamma=0.01,
            raw_kernel_weight=0.40,
            lam=0.3,
            blend_d0_curve=True,
            d0_curve_degree=1,
            d0_curve_lam=1.0,
        ),
    ]

    # A. 当前边界向外扩展：全部10个附件6特征。
    for raw_gamma in [0.002, 0.005, 0.01, 0.02]:
        for beta in [0.35, 0.50, 0.65, 0.80]:
            specs.append(
                CandidateSpec(
                    model_id=(
                        f"krr_mk_all_pg0.1_rg{raw_gamma:g}_b{beta:g}"
                        "_lam0.3_d0lin"
                    ),
                    family="kernel_ridge_rbf",
                    model_type="multikernel_krr",
                    base_mode="phys6",
                    feature_set="compact_physical",
                    raw_feature_set="raw_stability_only",
                    gamma=0.1,
                    raw_gamma=raw_gamma,
                    raw_kernel_weight=beta,
                    lam=0.3,
                    blend_d0_curve=True,
                    d0_curve_degree=1,
                    d0_curve_lam=1.0,
                )
            )

    # B. 特征组消融：删除Hampel特征，或仅保留纯波动特征。
    for raw_feature_set, tag in [
        ("raw_no_hampel", "nohp"),
        ("raw_variability_only", "var"),
    ]:
        for raw_gamma in [0.005, 0.01]:
            for beta in [0.50, 0.65]:
                specs.append(
                    CandidateSpec(
                        model_id=(
                            f"krr_mk_{tag}_pg0.1_rg{raw_gamma:g}_b{beta:g}"
                            "_lam0.3_d0lin"
                        ),
                        family="kernel_ridge_rbf",
                        model_type="multikernel_krr",
                        base_mode="phys6",
                        feature_set="compact_physical",
```

```
                        raw_feature_set=raw_feature_set,
                        gamma=0.1,
                        raw_gamma=raw_gamma,
                        raw_kernel_weight=beta,
                        lam=0.3,
                        blend_d0_curve=True,
                        d0_curve_degree=1,
                        d0_curve_lam=1.0,
                    )
                )

    # C. 更强正则：若高beta降低MAE但放大尾部，lambda=0.6可抑制。
    for raw_gamma in [0.005, 0.01]:
        for beta in [0.50, 0.65]:
            specs.append(
                CandidateSpec(
                    model_id=(
                        f"krr_mk_all_pg0.1_rg{raw_gamma:g}_b{beta:g}"
                        "_lam0.6_d01in"
                    ),
                    family="kernel_ridge_rbf",
                    model_type="multikernel_krr",
                    base_mode="phys6",
                    feature_set="compact_physical",
                    raw_feature_set="raw_stability_only",
                    gamma=0.1,
                    raw_gamma=raw_gamma,
                    raw_kernel_weight=beta,
                    lam=0.6,
                    blend_d0_curve=True,
                    d0_curve_degree=1,
                    d0_curve_lam=1.0,
                )
            )

    # D. 物理核尺度局部消融，只围绕最平滑原始核区域。
    for physical_gamma in [0.05, 0.20]:
        for beta in [0.50, 0.65]:
            specs.append(
                CandidateSpec(
                    model_id=(
                        f"krr_mk_all_pg{physical_gamma:g}_rg0.005_b{beta:g}"
                        "_lam0.3_d01in"
                    ),
                    family="kernel_ridge_rbf",
                    model_type="multikernel_krr",
                    base_mode="phys6",
                    feature_set="compact_physical",
                    raw_feature_set="raw_stability_only",
                    gamma=physical_gamma,
                    raw_gamma=0.005,
                    raw_kernel_weight=beta,
                    lam=0.3,
                    blend_d0_curve=True,
                    d0_curve_degree=1,
                    d0_curve_lam=1.0,
                )
            )

    # E. 少量无D0对照，判断线性D0是否真正改善跨划分稳健性。
    for raw_gamma, beta in [(0.005, 0.50), (0.01, 0.50)]:
        specs.append(
            CandidateSpec(
                model_id=(
                    f"krr_mk_all_pg0.1_rg{raw_gamma:g}_b{beta:g}"
                    "_lam0.3_nod0"
                ),
                family="kernel_ridge_rbf",
                model_type="multikernel_krr",
                base_mode="phys6",
                feature_set="compact_physical",
                raw_feature_set="raw_stability_only",
                gamma=0.1,
                raw_gamma=raw_gamma,
                raw_kernel_weight=beta,
                lam=0.3,
                blend_d0_curve=False,
            )
        )

    # 去重保护，避免同一规格被重复加入而无意增加选择权重。
    unique: dict[str, CandidateSpec] = {}
    for spec in specs:
        if spec.model_id in unique:
            raise RuntimeError(f"duplicate model_id: {spec.model_id}")
        unique[spec.model_id] = spec
    return list(unique.values())

# ---------------------------------------------------------------------
# 评价、诊断与报告
# ---------------------------------------------------------------------
```

```
def evaluateModel(
    model: FittedCandidate,
    frame: pd.DataFrame,
) -> tuple[pd.Series, pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    features = modelFeatureFrame(frame)
    yuceJieguo, diagnostics = model.predictWithDiagnostics(frame, features)
    summary, groups, windows = evaluatePredictions(frame, yuceJieguo)
    diagnostics = diagnostics.merge(
        windows[["window_id", "model_error_pct"]],
        on="window_id",
        how="left",
    )
    return summary, groups, windows, diagnostics


def evaluateNamedVolume(
    data: pd.DataFrame,
    name: str,
    volume: np.ndarray,
) -> dict[str, object]:
    summary, groups, _ = evaluatePredictions(data, np.asarray(volume, dtype=float))
    return {
        "method": name,
        "mae_pct": float(summary["mae_pct"]),
        "group_pass": summary["group_pass"],
        "max_abs_group_mean_pct": float(summary["max_abs_group_mean_pct"]),
        "max_group_sd_pct": float(summary["max_group_sd_pct"]),
        "u_nor_L_pct": float(summary["u_nor_L_pct"]),
        "u_nor_r_pct": float(summary["u_nor_r_pct"]),
        "u_nor_d_pct": float(summary["u_nor_d_pct"]),
        "pass_all_standards": bool(summary["pass_all_standards"]),
        "worst_group": (
            ""
            if groups.empty
            else f"{groups.sort_values('repeatability_sd_pct', ascending=False).iloc[0]['date']}-"
                 f"{groups.sort_values('repeatability_sd_pct', ascending=False).iloc[0]['flow_point']}"
        ),
    }


def physicalBaseDiagnostics(
    model: FittedCandidate,
    data: pd.DataFrame,
    model_prediction: np.ndarray,
) -> pd.DataFrame:
    matrix = baseMatrix(data)
    rows = [
        evaluateNamedVolume(data, "Phys6", matrix[:, 0]),
        evaluateNamedVolume(data, "RawHampelPhys6_s0.25", rawHampelPhys6Volume(data, 0.25, 0.5)),
        evaluateNamedVolume(data, "RawHampelPhys6_s0.5", rawHampelPhys6Volume(data, 0.5, 0.5)),
        evaluateNamedVolume(data, "RawHampelPhys6_s1.0", rawHampelPhys6Volume(data, 1.0, 0.5)),
        evaluateNamedVolume(data, "Lagrange", matrix[:, 1]),
        evaluateNamedVolume(data, "OWICS", matrix[:, 2]),
        evaluateNamedVolume(data, "EqualWeight", matrix[:, 3]),
        evaluateNamedVolume(data, "SelectedPhysicalBase", model.baseVolume(data)),
        evaluateNamedVolume(data, "FinalKRR", model_prediction),
    ]
    return pd.DataFrame(rows)


def parseGroupPassRatio(value: str) -> float:
    try:
        passed, total = str(value).split("/")
        total_i = int(total)
        return float(int(passed) / total_i) if total_i > 0 else float("nan")
    except Exception:
        return float("nan")


def aggregateMultisplit(ranking: pd.DataFrame) -> pd.DataFrame:
    frame = ranking.copy()
    frame["validation_group_pass_ratio"] = frame["validation_group_pass"].map(parseGroupPassRatio)

    rows: list[dict[str, object]] = []
    for model_id, group in frame.groupby("model_id", sort=False):
        first = group.iloc[0]
        rows.append(
            {
                "model_id": model_id,
                "family": first["family"],
                "split_count": int(len(group)),
                "all_pass_count": int(group["validation_pass_all"].astype(bool).sum()),
                "all_pass_rate": float(group["validation_pass_all"].astype(bool).mean()),
                "median_fail_count": float(group["validation_fail_count"].median()),
                "median_gap_score": float(group["validation_gap_score"].median()),
                "q75_gap_score": float(group["validation_gap_score"].quantile(0.75)),
                "median_mae_pct": float(group["validation_mae_pct"].median()),
                "median_group_pass_ratio": float(group["validation_group_pass_ratio"].median()),
                "median_u_nor_L_pct": float(group["validation_u_nor_L_pct"].median()),
                "median_u_nor_r_pct": float(group["validation_u_nor_r_pct"].median()),
                "q75_u_nor_r_pct": float(group["validation_u_nor_r_pct"].quantile(0.75)),
                "q90_u_nor_r_pct": float(group["validation_u_nor_r_pct"].quantile(0.90)),
                "max_u_nor_r_pct": float(group["validation_u_nor_r_pct"].max()),
```

```
                "u_nor_r_pass_count": int(
                    (group["validation_u_nor_r_pct"] < STANDARD_LIMITS["u_nor_r_pct"]).sum()
                ),
                "median_u_nor_d_pct": float(group["validation_u_nor_d_pct"].median()),
            }
        )

    result = pd.DataFrame(rows)
    return result.sort_values(
        [
            "all_pass_count",
            "median_fail_count",
            "median_gap_score",
            "q75_gap_score",
            "q90_u_nor_r_pct",
            "median_u_nor_d_pct",
            "median_mae_pct",
        ],
        ascending=[False, True, True, True, True, True, True],
    ).reset_index(drop=True)


def chooseKrrChampion(multisplit: pd.DataFrame) -> pd.Series:
    candidates = multisplit[multisplit["family"].eq("kernel_ridge_rbf")].copy()
    if candidates.empty:
        raise RuntimeError("no KRR candidates in multisplit summary")
    return candidates.iloc[0]


def chooseRepresentativeRow(ranking: pd.DataFrame, champion_id: str) -> pd.Series:
    rows = ranking[ranking["model_id"].eq(champion_id)].copy()
    if rows.empty:
        raise RuntimeError(f"champion not found: {champion_id}")
    medians = {
        "mae": float(rows["validation_mae_pct"].median()),
        "ur": float(rows["validation_u_nor_r_pct"].median()),
        "ul": float(rows["validation_u_nor_L_pct"].median()),
        "ud": float(rows["validation_u_nor_d_pct"].median()),
    }
    rows["representative_distance"] = (
        np.abs(rows["validation_mae_pct"] - medians["mae"])
        + np.abs(rows["validation_u_nor_r_pct"] - medians["ur"])
        + np.abs(rows["validation_u_nor_L_pct"] - medians["ul"])
        + np.abs(rows["validation_u_nor_d_pct"] - medians["ud"])
    )
    return rows.sort_values(
        ["validation_fail_count", "representative_distance", "validation_gap_score"],
        ascending=[True, True, True],
    ).iloc[0]


def toJsonable(value):
    if isinstance(value, dict):
        return {str(key): toJsonable(item) for key, item in value.items()}
    if isinstance(value, (list, tuple)):
        return [toJsonable(item) for item in value]
    if isinstance(value, np.ndarray):
        return [toJsonable(item) for item in value.tolist()]
    if isinstance(value, np.integer):
        return int(value)
    if isinstance(value, np.floating):
        number = float(value)
        return number if math.isfinite(number) else None
    if isinstance(value, np.bool_):
        return bool(value)
    if isinstance(value, float) and not math.isfinite(value):
        return None
    if pd.isna(value) and not isinstance(value, (str, bytes)):
        return None
    return value


def saveFittedModel(model: FittedCandidate, output_dir: Path) -> None:
    meta = {
        "spec": asdict(model.spec),
        "base_columns": BASE_COLUMNS,
        "base_weights": None if model.base_weights is None else model.base_weights.tolist(),
        "feature_columns": model.standardizer.columns if model.standardizer is not None else [],
        "system_feature_columns": (
            model.system_standardizer.columns if model.system_standardizer is not None else []
        ),
        "raw_kernel_feature_columns": (
            model.raw_standardizer.columns if model.raw_standardizer is not None else []
        ),
        "dynamic_feature_columns": (
            model.dynamic_standardizer.columns if model.dynamic_standardizer is not None else []
        ),
        "d0_curve_coef": None if model.d0_curve_coef is None else model.d0_curve_coef.tolist(),
        "note": (
            "Full-data fitted parameters. For multikernel_krr, the physical and raw-stability "
            "feature spaces use separate RBF kernels and one shared alpha vector. "
            "Generalisation evidence is reported separately from the full-data fit."
        ),
```

```
    }
    (output_dir / "krr_final_model_meta.json").write_text(
        json.dumps(toJsonable(meta), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

    arrays: dict[str, np.ndarray] = {}
    if model.base_weights is not None:
        arrays["base_weights"] = model.base_weights
    arrays["base_columns"] = np.asarray(BASE_COLUMNS, dtype="U")

    if model.standardizer is not None:
        arrays["feature_columns"] = np.asarray(model.standardizer.columns, dtype="U")
        arrays["median"] = model.standardizer.median.reindex(model.standardizer.columns).to_numpy(dtype=float)
        arrays["mean"] = model.standardizer.mean
        arrays["scale"] = model.standardizer.scale
    if model.train_x is not None:
        arrays["train_x"] = model.train_x
    if model.alpha is not None:
        arrays["alpha"] = model.alpha
    if model.raw_standardizer is not None:
        arrays["raw_kernel_feature_columns"] = np.asarray(
            model.raw_standardizer.columns,
            dtype="U",
        )
        arrays["raw_kernel_median"] = model.raw_standardizer.median.reindex(
            model.raw_standardizer.columns
        ).to_numpy(dtype=float)
        arrays["raw_kernel_mean"] = model.raw_standardizer.mean
        arrays["raw_kernel_scale"] = model.raw_standardizer.scale
    if model.raw_train_x is not None:
        arrays["raw_train_x"] = model.raw_train_x
    if model.coef is not None:
        arrays["coef"] = model.coef

    if model.system_standardizer is not None:
        arrays["system_feature_columns"] = np.asarray(model.system_standardizer.columns, dtype="U")
        arrays["system_median"] = model.system_standardizer.median.reindex(
            model.system_standardizer.columns
        ).to_numpy(dtype=float)
        arrays["system_mean"] = model.system_standardizer.mean
        arrays["system_scale"] = model.system_standardizer.scale
    if model.system_train_x is not None:
        arrays["system_train_x"] = model.system_train_x
    if model.system_alpha is not None:
        arrays["system_alpha"] = model.system_alpha

    if model.dynamic_standardizer is not None:
        arrays["dynamic_feature_columns"] = np.asarray(model.dynamic_standardizer.columns, dtype="U")
        arrays["dynamic_median"] = model.dynamic_standardizer.median.reindex(
            model.dynamic_standardizer.columns
        ).to_numpy(dtype=float)
        arrays["dynamic_mean"] = model.dynamic_standardizer.mean
        arrays["dynamic_scale"] = model.dynamic_standardizer.scale
    if model.dynamic_train_x is not None:
        arrays["dynamic_train_x"] = model.dynamic_train_x
    if model.dynamic_alpha is not None:
        arrays["dynamic_alpha"] = model.dynamic_alpha

    if model.d0_curve_coef is not None:
        arrays["d0_curve_coef"] = model.d0_curve_coef
    np.savez_compressed(output_dir / "krr_final_model.npz", **arrays)

def markdownTable(frame: pd.DataFrame, max_rows: int = 20) -> str:
    if frame.empty:
        return ""
    use = frame.head(max_rows).copy()
    for column in use.columns:
        if pd.api.types.is_float_dtype(use[column]):
            use[column] = use[column].map(lambda value: f"{value:.6g}")
    use = use.astype(str)
    headers = list(use.columns)
    rows = use.values.tolist()
    widths = [max(len(header), *(len(row[i]) for row in rows)) for i, header in enumerate(headers)]
    header_line = "| " + " | ".join(header.ljust(widths[i]) for i, header in enumerate(headers)) + " |"
    separator = "| " + " | ".join("-" * widths[i] for i in range(len(headers))) + " |"
    body = ["| " + " | ".join(row[i].ljust(widths[i]) for i in range(len(headers))) + " |" for row in rows]
    return "\n".join([header_line, separator] + body)


def writeReport(
    ranking: pd.DataFrame,
    multisplit: pd.DataFrame,
    champion: pd.Series,
    champion_spec: CandidateSpec,
    representative: pd.Series,
    representative_model: FittedCandidate,
    representative_groups: pd.DataFrame,
    representative_base_diag: pd.DataFrame,
    pressure_summary: pd.Series,
    pressure_groups: pd.DataFrame,
    pressure_base_diag: pd.DataFrame,
    fullfit_summary: pd.Series,
```

```
    fullfit_base_diag: pd.DataFrame,
    full_model: FittedCandidate,
) -> None:
    multi_columns = [
        "model_id",
        "all_pass_count",
        "median_fail_count",
        "median_gap_score",
        "median_mae_pct",
        "median_group_pass_ratio",
        "median_u_nor_L_pct",
        "median_u_nor_r_pct",
        "q75_u_nor_r_pct",
        "u_nor_r_pass_count",
        "median_u_nor_d_pct",
    ]
    ranking_columns = [
        "split_no",
        "model_id",
        "validation_fail_count",
        "validation_pass_all",
        "validation_mae_pct",
        "validation_group_pass",
        "validation_max_abs_group_mean_pct",
        "validation_u_nor_L_pct",
        "validation_u_nor_r_pct",
        "validation_u_nor_d_pct",
    ]

    rep_weights = (
        "median4（无固定线性权重）"
        if representative_model.base_weights is None
        else json.dumps(dict(zip(BASE_COLUMNS, representative_model.base_weights.tolist())), ensure_ascii=False)
    )
    full_weights = (
        "median4（无固定线性权重）"
        if full_model.base_weights is None
        else json.dumps(dict(zip(BASE_COLUMNS, full_model.base_weights.tolist())), ensure_ascii=False)
    )

    text = f"""# 分层划分多核 KRR 边界扩展与附件6特征消融报告

## 本轮修改

- 恢复按 `(date, flow_point, disturbance_id)` 进行标签盲分层划分。
- 官方指标仍按题目脚本计算，其中重复性组为 `(date, flow_point)`。
- 第一阶段按 `(date, flow_point, disturbance_id)` 组均值学习系统偏差；第二阶段只学习组内窗口偏差并用 `rho` 强收缩。
- 从附件6逐秒 `diff_ns_0 - 9` 与 `mean_us_0 - 9` 提取MAD、漂移、同步性、Hampel尖峰率和A/B不一致特征。
- Hampel滤波仅替换局部尖峰，并以滤波前后Phys6整窗积分信号之比微调附件1 Phys6，不使用平台均值替代整窗积分。
- 附件6逐秒序列先按window_id聚合为窗口级稳定性特征；本版比较全部特征、删除Hampel特征和仅波动特征，并扩展原始核尺度与核权重边界。
- 日期、标准体积和扰流编号只用于离线划分或评价，不进入最终运行时特征。

## 40 次标签盲划分聚合排名

{markdownTable(multisplit[multi_columns], 20)}

## 最终 KRR 配置

```text
{champion['model_id']}
```

- 规格：`{asdict(champion_spec)}`
- 全标准通过：`{int(champion['all_pass_count'])}/{int(champion['split_count'])}`
- 中位 MAE：`{champion['median_mae_pct']:.6g}%`
- 中位 $u_{{nor,r}}$：`{champion['median_u_nor_r_pct']:.6g}%`
- 75% 分位 $u_{{nor,r}}$：`{champion['q75_u_nor_r_pct']:.6g}%`
- 90% 分位 $u_{{nor,r}}$：`{champion['q90_u_nor_r_pct']:.6g}%`
- $u_{{nor,r}}$ 通过次数：`{int(champion['u_nor_r_pass_count'])}/{int(champion['split_count'])}`

代表性验证划分

- split：`{int(representative['split_no'])}`，seed：`{int(representative['split_seed'])}`
- MAE：`{representative['validation_mae_pct']:.6g}%`
- 分组通过：`{representative['validation_group_pass']}`
- $u_{{nor,L}}$：`{representative['validation_u_nor_L_pct']:.6g}%`
- $u_{{nor,r}}$：`{representative['validation_u_nor_r_pct']:.6g}%`
- $u_{{nor,d}}$：`{representative['validation_u_nor_d_pct']:.6g}%`
- 本折物理基线权重：`{rep_weights}`

代表性验证的物理基线方差分解

{markdownTable(representative_base_diag, 20)}

代表性验证中重复性最差的组

{markdownTable(representative_groups.sort_values('repeatability_sd_pct', ascending=False), 12)}

单次结果前 20（仅诊断）

{markdownTable(ranking[ranking_columns], 20)}

整组留出压力测试
```

```
{markdownTable(pd.DataFrame([pressure_summary.to_dict()]), 1)}

### 压力测试物理基线方差分解

{markdownTable(pressure_base_diag, 20)}

### 压力测试中重复性最差的组

{markdownTable(pressure_groups.sort_values('repeatability_sd_pct', ascending=False), 12)}

## 全量重拟合 KRR（样本内结果）

- MAE: `{fullfit_summary['mae_pct']:.6g}%`
- 分组通过: `{fullfit_summary['group_pass']}`
- $u_{{nor,L}}$: `{fullfit_summary['u_nor_L_pct']:.6g}%`
- $u_{{nor,r}}$: `{fullfit_summary['u_nor_r_pct']:.6g}%`
- $u_{{nor,d}}$: `{fullfit_summary['u_nor_d_pct']:.6g}%`
- 全量物理基线权重: `{full_weights}`

{markdownTable(fullfit_base_diag, 20)}

## 主要输出

- `code/outputs/stratified_model_search_ranking.csv`
- `code/outputs/stratified_model_search_multisplit_summary.csv`
- `code/outputs/stratified_model_search_champion_splits.csv`
- `code/outputs/stratified_model_search_representative_*.csv`
- `code/outputs/stratified_model_search_pressure_*.csv`
- `code/outputs/physical_base_diagnostics_*.csv`
- `code/outputs/submission_krr_final.csv`
- `code/outputs/krr_final_model.npz`
- `code/outputs/krr_final_model_meta.json`
"""
    REPORT.parent.mkdir(parents=True, exist_ok=True)
    REPORT.write_text(text, encoding="utf-8")


# ---------------------------------------------------------------------------
# 主流程
# ---------------------------------------------------------------------------


def runSearch() -> None:
    print(f"=== Running {SCRIPT_VERSION} ===", flush=True)
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    REPORT.parent.mkdir(parents=True, exist_ok=True)

    # 必须读取附件1，并合并附件6构造的窗口级稳定性特征。
    # v5 中误用了 load_data()，导致所有多核候选的 raw_stability_only
    # 特征列表为空，同时 compact_plus_raw 候选悄悄退化为仅 compact 特征。
    data = loadDataWithRawFeatures().reset_index(drop=True)

    missing_raw = [
        column for column in RAW_MODEL_FEATURE_COLUMNS
        if column not in data.columns
    ]
    if missing_raw:
        raise RuntimeError(
            "raw stability features were not merged: "
            + ", ".join(missing_raw)
        )

    raw_numeric = (
        data[RAW_MODEL_FEATURE_COLUMNS]
        .apply(pd.to_numeric, errors="coerce")
        .replace([np.inf, -np.inf], np.nan)
    )
    nonconstant_raw = [
        column for column in RAW_MODEL_FEATURE_COLUMNS
        if raw_numeric[column].notna().any()
        and float(raw_numeric[column].std(ddof=0)) > 1e-12
    ]
    if not nonconstant_raw:
        raise RuntimeError(
            "all raw stability features are missing or constant; "
            "check attachment6/attachment7 paths and chord mapping"
        )

    print(
        "Raw stability features loaded: "
        f"{len(RAW_MODEL_FEATURE_COLUMNS)} columns, "
        f"{len(nonconstant_raw)} non-constant, "
        f"{len(data)} windows",
        flush=True,
    )

    specs = buildSpecs()
    spec_lookup = {spec.model_id: spec for spec in specs}

    probe_features = modelFeatureFrame(data.head(min(len(data), 8)))
    raw_sets = sorted({
        spec.raw_feature_set
```

```
        for spec in specs
        if spec.model_type == "multikernel_krr"
    })
    for raw_set in raw_sets:
        raw_columns = columnsForFeatureSet(probe_features, raw_set)
        print(
            f"Raw kernel set {raw_set}: {len(raw_columns)} columns",
            flush=True,
        )

    for spec in specs:
        if spec.model_type == "multikernel_krr":
            physical_columns = columnsForFeatureSet(
                probe_features,
                spec.feature_set,
            )
            raw_columns = columnsForFeatureSet(
                probe_features,
                spec.raw_feature_set,
            )
            if not physical_columns:
                raise RuntimeError(
                    f"{spec.model_id}: physical kernel feature list is empty"
                )
            if not raw_columns:
                raise RuntimeError(
                    f"{spec.model_id}: raw kernel feature list is empty"
                )

    split_specs = [
        SplitSpec(
            seed=RANDOM_SEED + 1000 + i * 17,
            d0_n3_to_validation=bool(i % 3 == 0),
            n34_validation_bias=[0.0, 0.25, 0.5, 1.0][i % 4],
            small_validation_prob=[0.25, 0.40, 0.55, 0.70][(i // 4) % 4],
        )
        for i in range(N_SPLITS)
    ]

    rows: list[dict[str, object]] = []
    split_cache: dict[int, tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]] = {}

    for split_no, split_spec in enumerate(split_specs):
        print(f"[split {split_no + 1}/{N_SPLITS}] seed={split_spec.seed}", flush=True)
        train, validation, split = splitAssignments(data, split_spec)
        valid, message = validateSplitRules(data, split)
        if not valid:
            raise RuntimeError(message)
        split_cache[split_no] = (train, validation, split)

        xunlianTezheng = modelFeatureFrame(train)
        for spec in specs:
            try:
                model = fitCandidate(spec, train, xunlianTezheng)
                train_summary, _, _, _ = evaluateModel(model, train)
                validation_summary, _, _, _ = evaluateModel(model, validation)

                validation_fails, validation_gap = standardsGap(validation_summary)
                train_fails, _ = standardsGap(train_summary)
                score = (
                    10000.0 * int(validation_fails)
                    + 500.0 * float(validation_gap)
                    + 30.0 * max(
                        0.0,
                        float(validation_summary["mae_pct"]) - float(train_summary["mae_pct"]),
                    )
                    + float(validation_summary["mae_pct"])
                )

                if model.base_weights is None:
                    base_weight_values = [float("nan")] * len(BASE_COLUMNS)
                else:
                    base_weight_values = model.base_weights.tolist()

                row = {
                    "split_no": split_no,
                    "split_seed": split_spec.seed,
                    "train_count": len(train),
                    "validation_count": len(validation),
                    "model_id": spec.model_id,
                    "family": spec.family,
                    "rank_score": score,
                    "validation_fail_count": int(validation_fails),
                    "validation_gap_score": float(validation_gap),
                    "validation_pass_all": bool(validation_summary["pass_all_standards"]),
                    "validation_mae_pct": float(validation_summary["mae_pct"]),
                    "validation_group_pass": validation_summary["group_pass"],
                    "validation_max_abs_group_mean_pct": float(validation_summary["max_abs_group_mean_pct"]),
                    "validation_max_group_sd_pct": float(validation_summary["max_group_sd_pct"]),
                    "validation_u_nor_L_pct": float(validation_summary["u_nor_L_pct"]),
                    "validation_u_nor_r_pct": float(validation_summary["u_nor_r_pct"]),
                    "validation_u_nor_d_pct": float(validation_summary["u_nor_d_pct"]),
                    "train_fail_count": int(train_fails),
```

```
                    "train_mae_pct": float(train_summary["mae_pct"]),
                    **{f"base_weight_{BASE_COLUMNS[i]}": base_weight_values[i] for i in range(len(BASE_COLUMNS))},
                    **{f"spec_{key}": value for key, value in asdict(spec).items()},
                }
                rows.append(row)
            except (np.linalg.LinAlgError, FloatingPointError, ValueError) as exc:
                print(f"  [skip] {spec.model_id}: {exc}", flush=True)

    ranking = pd.DataFrame(rows)
    if ranking.empty:
        raise RuntimeError("no candidate model completed")
    ranking = ranking.sort_values(
        ["validation_fail_count", "rank_score", "validation_mae_pct"],
        ascending=[True, True, True],
    ).reset_index(drop=True)
    ranking.to_csv(
        OUT_DIR / "stratified_model_search_ranking.csv",
        index=False,
        encoding="utf-8-sig",
    )

    multisplit = aggregateMultisplit(ranking)
    multisplit.to_csv(
        OUT_DIR / "stratified_model_search_multisplit_summary.csv",
        index=False,
        encoding="utf-8-sig",
    )

    champion = chooseKrrChampion(multisplit)
    champion_id = str(champion["model_id"])
    champion_spec = spec_lookup[champion_id]
    champion_rows = ranking[ranking["model_id"].eq(champion_id)].copy()
    champion_rows.to_csv(
        OUT_DIR / "stratified_model_search_champion_splits.csv",
        index=False,
        encoding="utf-8-sig",
    )

    representative = chooseRepresentativeRow(ranking, champion_id)
    representative_split_no = int(representative["split_no"])
    rep_train, rep_validation, rep_split = split_cache[representative_split_no]
    rep_model = fitCandidate(champion_spec, rep_train, modelFeatureFrame(rep_train))
    rep_summary, rep_groups, rep_windows, rep_diagnostics = evaluateModel(rep_model, rep_validation)
    rep_prediction = rep_diagnostics["model_volume_m3"].to_numpy(dtype=float)
    rep_base_diag = physicalBaseDiagnostics(rep_model, rep_validation, rep_prediction)

    rep_split.to_csv(OUT_DIR / "stratified_model_search_representative_split.csv", index=False, encoding="utf-8-sig")
    rep_groups.to_csv(OUT_DIR / "stratified_model_search_representative_validation_groups.csv", index=False, encoding="utf-8-sig")
    rep_windows.to_csv(OUT_DIR / "stratified_model_search_representative_validation_windows.csv", index=False, encoding="utf-8-sig")
    rep_diagnostics.to_csv(OUT_DIR / "stratified_model_search_representative_diagnostics.csv", index=False, encoding="utf-8-sig")
    rep_base_diag.to_csv(OUT_DIR / "physical_base_diagnostics_representative.csv", index=False, encoding="utf-8-sig")

    pressure_seed = RANDOM_SEED + 424242
    pressure_train, pressure_validation, pressure_assignment = pressureSplit(data, pressure_seed)
    pressure_model = fitCandidate(champion_spec, pressure_train, modelFeatureFrame(pressure_train))
    pressure_summary, pressure_groups, pressure_windows, pressure_diagnostics = evaluateModel(
        pressure_model,
        pressure_validation,
    )
    pressure_prediction = pressure_diagnostics["model_volume_m3"].to_numpy(dtype=float)
    pressure_base_diag = physicalBaseDiagnostics(
        pressure_model,
        pressure_validation,
        pressure_prediction,
    )
    pressure_fails, pressure_gap = standardsGap(pressure_summary)

    pressure_assignment.to_csv(OUT_DIR / "stratified_model_search_pressure_split.csv", index=False, encoding="utf-8-sig")
    pressure_groups.to_csv(OUT_DIR / "stratified_model_search_pressure_groups.csv", index=False, encoding="utf-8-sig")
    pressure_windows.to_csv(OUT_DIR / "stratified_model_search_pressure_windows.csv", index=False, encoding="utf-8-sig")
    pressure_diagnostics.to_csv(OUT_DIR / "stratified_model_search_pressure_diagnostics.csv", index=False, encoding="utf-8-sig")
    pressure_base_diag.to_csv(OUT_DIR / "physical_base_diagnostics_pressure.csv", index=False, encoding="utf-8-sig")

    # 全量重拟合仅用于生成题目给定 159 个窗口的 KRR 结果文件，不替代泛化验证。
    full_features = modelFeatureFrame(data)
    full_model = fitCandidate(champion_spec, data, full_features)
    full_prediction, full_diagnostics = full_model.predictWithDiagnostics(data, full_features)
    full_summary, full_groups, full_windows = evaluatePredictions(data, full_prediction)
    full_base_diag = physicalBaseDiagnostics(full_model, data, full_prediction)

    pd.DataFrame(
        {
            "window_id": data["window_id"],
            "model_volume_m3": full_prediction,
        }
    ).to_csv(
        OUT_DIR / "submission_krr_final.csv",
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
    )
    pd.DataFrame([full_summary.to_dict()]).to_csv(
        OUT_DIR / "krr_fullfit_evaluation_summary.csv",
```

```
        index=False,
        encoding="utf-8-sig",
    )
    full_groups.to_csv(OUT_DIR / "krr_fullfit_evaluation_groups.csv", index=False, encoding="utf-8-sig")
    full_windows.to_csv(OUT_DIR / "krr_fullfit_evaluation_windows.csv", index=False, encoding="utf-8-sig")
    full_diagnostics.to_csv(OUT_DIR / "krr_fullfit_diagnostics.csv", index=False, encoding="utf-8-sig")
    full_base_diag.to_csv(OUT_DIR / "physical_base_diagnostics_fullfit.csv", index=False, encoding="utf-8-sig")
    saveFittedModel(full_model, OUT_DIR)

    meta = {
        "selection_rule": "best aggregate KRR across 40 label-blind stratified splits",
        "split_group_key": SPLIT_GROUP_KEY,
        "official_repeatability_group_key": OFFICIAL_GROUP_KEY,
        "champion_multisplit": champion.to_dict(),
        "champion_spec": asdict(champion_spec),
        "representative_validation": representative.to_dict(),
        "representative_base_weights": (
            None if rep_model.base_weights is None else rep_model.base_weights.tolist()
        ),
        "pressure_summary": {
            **pressure_summary.to_dict(),
            "standard_fail_count": int(pressure_fails),
            "standard_gap_score": float(pressure_gap),
        },
        "pressure_base_weights": (
            None if pressure_model.base_weights is None else pressure_model.base_weights.tolist()
        ),
        "fullfit_sample_fit_summary": full_summary.to_dict(),
        "fullfit_base_weights": (
            None if full_model.base_weights is None else full_model.base_weights.tolist()
        ),
        "standard_limits": STANDARD_LIMITS,
        "n_splits": N_SPLITS,
        "n_specs": len(specs),
    }
    (OUT_DIR / "stratified_model_search_best_meta.json").write_text(
        json.dumps(toJsonable(meta), ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

    writeReport(
        ranking=ranking,
        multisplit=multisplit,
        champion=champion,
        champion_spec=champion_spec,
        representative=representative,
        representative_model=rep_model,
        representative_groups=rep_groups,
        representative_base_diag=rep_base_diag,
        pressure_summary=pressure_summary,
        pressure_groups=pressure_groups,
        pressure_base_diag=pressure_base_diag,
        fullfit_summary=full_summary,
        fullfit_base_diag=full_base_diag,
        full_model=full_model,
    )

    print("\n=== KRR multisplit champion ===")
    print(champion.to_string())
    print("\n=== Representative validation ===")
    print(rep_summary.to_string())
    print("\nRepresentative physical-base weights:")
    if rep_model.base_weights is None:
        print("median4")
    else:
        print(dict(zip(BASE_COLUMNS, rep_model.base_weights.tolist())))
    print("\n=== Whole-group pressure test ===")
    print(pressure_summary.to_string())
    print("\n=== Full-fit KRR sample-fit result ===")
    print(full_summary.to_string())
    print("\nFull-fit physical-base weights:")
    if full_model.base_weights is None:
        print("median4")
    else:
        print(dict(zip(BASE_COLUMNS, full_model.base_weights.tolist())))
    print(f"\nSubmission: {OUT_DIR / 'submission_krr_final.csv'}")
    print(f"Report: {REPORT}")


if __name__ == "__main__":
    runSearch()
```

B.10 最终后校准模型：final_postcalibrated_dual_kernel_krr.py

```
from __future__ import annotations

# 论文附录复现代码。
# 问题四最终模型：双核 KRR 加受约束 DO-RBF 后校准。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

```

```
"""
d0_rbf_postcalibrated_dual_kernel_krr.py

用途
----
以题目给出的 evaluate_submission.py 为最终判据，对完整 159 个窗口执行
“已知样本全量标定型”迭代：

1. 冻结物理--稳定性双核 KRR 作为物理主模型；
2. 只针对未通过的 u_nor,L、u_nor,r、u_nor,d 增加一个受约束校准层；
3. 候选选择严格按官方脚本的 pass_all_3 和三个指标超限量排序；
4. 输出可直接交给 evaluate_submission.py 的 submission CSV；
5. 同时保留模型参数和评价明细。

重要说明
--------
这是“完整已知数据标定/提交优化”，训练和最终评价使用同一批 159 个窗口。
它可以作为官方提交文件的优化手段，但不能作为未知工况泛化能力的证据。
泛化结论仍应引用 40 次开发划分和 20 次独立确认划分结果。

依赖
----
同目录需存在以下任一核心模块：
    stratified_model_search_dual_kernel_krr.py

核心模块继续负责：
    - 读取附件1、附件6、附件7；
    - 构造物理特征和附件6稳定性特征；
    - 拟合冻结的物理--稳定性双核 KRR。
"""

import importlib
import json
import math
import shutil
import subprocess
import sys
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Iterable

import numpy as np
import pandas as pd


SCRIPT_VERSION = "constrained-d0-rbf-postcalibrated-dual-kernel-krr-20260712"

HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
OUT_DIR = HERE / "verification_outputs"
REPORT_DIR = Path(__file__).resolve().parent / "verification_reports"

OFFICIAL_LIMITS = {
    "u_nor_L_pct": 0.036,
    "u_nor_r_pct": 0.040,
    "u_nor_d_pct": 0.115,
    "group_mean_abs_pct": 0.200,
    "group_sd_pct": 0.040,
}

FORBIDDEN_RUNTIME_COLUMNS = {
    "standard_volume_m3",
    "window_id",
    "date",
    "disturbance_id",
    "condition_note",
    "model_error_pct",
}

# 搜索规模控制：所有候选都在完整159窗口上按官方脚本直接评价。
GAMMA_GRID = [0.003, 0.01, 0.03, 0.10, 0.30]
LAMBDA_GRID = [1e-10, 1e-8, 1e-6, 1e-4, 1e-3, 1e-2]
RHO_GRID = [0.50, 0.75, 1.00]

# DO流量线性度模块。scale=0表示关闭。
LINEARITY_CONFIGS = [
    ("none", 0, 1.0, 0.0),
    ("linear_s0.5", 1, 1.0, 0.5),
    ("linear_s1", 1, 1.0, 1.0),
    ("quadratic_s0.5", 2, 1.0, 0.5),
    ("quadratic_s1", 2, 1.0, 1.0),
]

# 允许校准层对冻结主模型做的最大相对修正。
CORRECTION_CLIP_PCT = 0.80

# 只有在官方三个指标全部通过时，才自动提升为 submission_final.csv。
PROMOTE_TO_SUBMISSION_FINAL_IF_PASS = True


def importCoreModule():
    errors: list[str] = []
    for name in ["physical_stability_dual_kernel_krr"]:
```

```
        try:
            module = importlib.import_module(name)
            required = [
                "CandidateSpec",
                "fitCandidate",
                "loadDataWithRawFeatures",
                "modelFeatureFrame",
                "columnsForFeatureSet",
            ]
            missing = [attr for attr in required if not hasattr(module, attr)]
            if missing:
                errors.append(f"{name}: missing {missing}")
                continue
            return module
        except Exception as exc:  # 启动阶段要给出完整、可读的依赖错误。
            errors.append(f"{name}: {type(exc).__name__}: {exc}")
    raise ImportError(
        "无法导入物理--稳定性双核 KRR 核心模块。请把双核脚本放在本脚本同目录。\n"
        + "\n".join(errors)
    )


CORE = importCoreModule()


@dataclass(frozen=True)
class LinearityConfig:
    name: str
    degree: int
    ridge_lambda: float
    scale: float


@dataclass
class RobustStandardizer:
    columns: list[str]
    median: np.ndarray
    mean: np.ndarray
    scale: np.ndarray

    @classmethod
    def fit(cls, frame: pd.DataFrame, columns: list[str]) -> "RobustStandardizer":
        if not columns:
            raise ValueError("校准层没有可用特征")
        clean = (
            frame[columns]
            .apply(pd.to_numeric, errors="coerce")
            .replace([np.inf, -np.inf], np.nan)
        )
        median_series = clean.median(numeric_only=True).fillna(0.0)
        arr = clean.fillna(median_series).to_numpy(dtype=float)
        mean = arr.mean(axis=0)
        scale = arr.std(axis=0)
        scale[scale < 1e-12] = 1.0
        return cls(
            columns=list(columns),
            median=median_series.to_numpy(dtype=float),
            mean=mean,
            scale=scale,
        )

    def transform(self, frame: pd.DataFrame) -> np.ndarray:
        clean = (
            frame[self.columns]
            .apply(pd.to_numeric, errors="coerce")
            .replace([np.inf, -np.inf], np.nan)
        )
        arr = clean.to_numpy(dtype=float)
        for j in range(arr.shape[1]):
            mask = ~np.isfinite(arr[:, j])
            if np.any(mask):
                arr[mask, j] = self.median[j]
        return (arr - self.mean) / self.scale


def relativeErrorPct(moxingTiji: np.ndarray, biaozhunTiji: np.ndarray) -> np.ndarray:
    model = np.asarray(moxingTiji, dtype=float)
    standard = np.asarray(biaozhunTiji, dtype=float)
    return (model - standard) / standard * 100.0


def officialGroupTable(data: pd.DataFrame, errorPct: np.ndarray) -> pd.DataFrame:
    work = data[["date", "flow_point"]].reset_index(drop=True).copy()
    work["model_error_pct"] = np.asarray(errorPct, dtype=float)
    rows: list[dict[str, object]] = []
    for (date, flow_point), group in work.groupby(
        ["date", "flow_point"],
        sort=True,
    ):
        if len(group) < 3:
            continue
        values = group["model_error_pct"].astype(float)
        mean_err = float(values.mean())
```

```
        sd = float(values.std(ddof=1))
        rows.append(
            {
                "date": str(date),
                "flow_point": int(flow_point),
                "n": int(len(group)),
                "mean_error_pct": mean_err,
                "abs_mean_error_pct": abs(mean_err),
                "repeatability_sd_pct": sd,
                "range_pct": float(values.max() - values.min()),
                "pass_group": bool(
                    abs(mean_err) <= OFFICIAL_LIMITS["group_mean_abs_pct"]
                    and sd <= OFFICIAL_LIMITS["group_sd_pct"]
                ),
            }
        )
    return pd.DataFrame(rows)


def officialLinearityUL(data: pd.DataFrame, errorPct: np.ndarray) -> float:
    work = data.reset_index(drop=True).copy()
    work["model_error_pct"] = np.asarray(errorPct, dtype=float)
    use = work[
        work["condition_note"].eq("no_disturbance_reference")
        & work["flow_point"].between(40, 100)
    ]
    by_flow = use.groupby("flow_point")["model_error_pct"].mean()
    if len(by_flow) <= 1:
        return float("nan")
    return math.sqrt(float((by_flow**2).sum()) / (len(by_flow) - 1))


def officialDisturbanceUD(data: pd.DataFrame, errorPct: np.ndarray) -> float:
    work = data.reset_index(drop=True).copy()
    work["model_error_pct"] = np.asarray(errorPct, dtype=float)
    use = work[work["flow_point"].between(40, 100)].copy()

    base = (
        use[use["condition_note"].eq("no_disturbance_reference")]
        .groupby("flow_point")["model_error_pct"]
        .agg(["mean", "count", "std"])
        .rename(
            columns={
                "mean": "baseline_mean",
                "count": "baseline_n",
                "std": "baseline_sd",
            }
        )
        .reset_index()
    )
    disturb = (
        use[use["condition_note"].eq("disturbed_test")]
        .groupby(["disturbance_id", "flow_point"])["model_error_pct"]
        .agg(["mean", "count", "std"])
        .rename(
            columns={
                "mean": "disturb_mean",
                "count": "disturb_n",
                "std": "disturb_sd",
            }
        )
        .reset_index()
    )
    detail = disturb.merge(base, on="flow_point", how="left")
    detail["disturb_sd"] = detail["disturb_sd"].fillna(0.0)
    detail["abs_drift"] = (
        detail["baseline_mean"] - detail["disturb_mean"]
    ).abs()
    available = detail[
        (detail["baseline_n"] >= 1)
        & (detail["disturb_n"] >= 1)
    ].copy()
    if available.empty:
        return float("nan")
    u_d_c = float(available["abs_drift"].max()) / math.sqrt(3.0)
    u_d_r = float(available["disturb_sd"].max())
    return math.sqrt(u_d_c * u_d_c + u_d_r * u_d_r)


def officialEvaluate(
    data: pd.DataFrame,
    yuceJieguo: np.ndarray,
) -> tuple[dict[str, object], pd.DataFrame, pd.DataFrame]:
    standard = data["standard_volume_m3"].to_numpy(dtype=float)
    error = relativeErrorPct(yuceJieguo, standard)
    groups = officialGroupTable(data, error)
    u_l = officialLinearityUL(data, error)
    u_r = (
        float(groups["repeatability_sd_pct"].max())
        if not groups.empty
        else float("nan")
    )
    u_d = officialDisturbanceUD(data, error)
```

```

    pass_l = bool(np.isfinite(u_l) and u_l < OFFICIAL_LIMITS["u_nor_L_pct"])
    pass_r = bool(np.isfinite(u_r) and u_r < OFFICIAL_LIMITS["u_nor_r_pct"])
    pass_d = bool(np.isfinite(u_d) and u_d < OFFICIAL_LIMITS["u_nor_d_pct"])

    summary: dict[str, object] = {
        "window_count": int(len(data)),
        "mae_pct": float(np.mean(np.abs(error))),
        "mean_error_pct": float(np.mean(error)),
        "max_abs_error_pct": float(np.max(np.abs(error))),
        "group_count": int(len(groups)),
        "group_pass_count": (
            int(groups["pass_group"].sum()) if not groups.empty else 0
        ),
        "group_pass": (
            f"{int(groups['pass_group'].sum())}/{len(groups)}"
            if not groups.empty
            else "0/0"
        ),
        "max_abs_group_mean_pct": (
            float(groups["abs_mean_error_pct"].max())
            if not groups.empty
            else float("nan")
        ),
        "u_nor_L_pct": float(u_l),
        "u_nor_r_pct": float(u_r),
        "u_nor_d_pct": float(u_d),
        "pass_u_nor_L": pass_l,
        "pass_u_nor_r": pass_r,
        "pass_u_nor_d": pass_d,
        "pass_all_3": bool(pass_l and pass_r and pass_d),
    }

    windows = data[
        [
            "window_id",
            "date",
            "flow_point",
            "condition_note",
            "disturbance_id",
            "standard_volume_m3",
        ]
    ].reset_index(drop=True).copy()
    windows["model_volume_m3"] = np.asarray(yuceJieguo, dtype=float)
    windows["model_error_pct"] = error
    return summary, groups, windows


def metricGaps(summary: dict[str, object]) -> tuple[int, float, float, float, float]:
    values = {
        "L": float(summary["u_nor_L_pct"]),
        "R": float(summary["u_nor_r_pct"]),
        "D": float(summary["u_nor_d_pct"]),
    }
    gap_l = max(0.0, values["L"] / OFFICIAL_LIMITS["u_nor_L_pct"] - 1.0)
    gap_r = max(0.0, values["R"] / OFFICIAL_LIMITS["u_nor_r_pct"] - 1.0)
    gap_d = max(0.0, values["D"] / OFFICIAL_LIMITS["u_nor_d_pct"] - 1.0)
    fail_count = int(gap_l > 0.0) + int(gap_r > 0.0) + int(gap_d > 0.0)

    # 只针对未通过指标；重复性权重最大。
    weighted_gap = 0.25 * gap_l + 0.55 * gap_r + 0.20 * gap_d
    return fail_count, weighted_gap, gap_l, gap_r, gap_d


def selectionKey(row: dict[str, object]) -> tuple:
    """
    排序完全围绕官方脚本：
    1. pass_all_3；
    2. 失败指标个数；
    3. 三个失败指标归一化超限；
    4. 保护组均值与组通过数；
    5. 若已经通过，优先更强正则和更小校准；
    6. 最后比较 MAE。
    """
    return (
        0 if bool(row["pass_all_3"]) else 1,
        int(row["official_fail_count"]),
        float(row["official_weighted_gap"]),
        int(row["group_fail_count"]),
        max(
            0.0,
            float(row["max_abs_group_mean_pct"])
            / OFFICIAL_LIMITS["group_mean_abs_pct"]
            - 1.0,
        ),
        float(row["complexity_penalty"]),
        float(row["max_abs_total_correction_pct"]),
        float(row["mae_pct"]),
    )


def buildFrozenDualKernelModel(data: pd.DataFrame, features: pd.DataFrame):
    spec = CORE.CandidateSpec(
```

```
        model_id="krr_dual_ps_pg0.05_rg0.005_b0.5_lam0.3_d0lin",
        family="kernel_ridge_rbf",
        model_type="multikernel_krr",
        base_mode="phys6",
        feature_set="compact_physical",
        raw_feature_set="raw_stability_only",
        gamma=0.05,
        raw_gamma=0.005,
        raw_kernel_weight=0.50,
        lam=0.30,
        blend_d0_curve=True,
        d0_curve_degree=1,
        d0_curve_lam=1.0,
    )
    model = CORE.fitCandidate(spec, data, features)
    yuceJieguo, diagnostics = model.predictWithDiagnostics(data, features)
    return spec, model, np.asarray(yuceJieguo, dtype=float), diagnostics


def buildCalibrationFeatureSets(features: pd.DataFrame) -> dict[str, list[str]]:
    compact_raw = CORE.columnsForFeatureSet(
        features,
        "compact_plus_raw",
    )

    runtime_full = []
    try:
        runtime_full = CORE.columnsForFeatureSet(features, "runtime_full")
    except Exception:
        runtime_full = []

    all_numeric = [
        column
        for column in features.columns
        if column not in FORBIDDEN_RUNTIME_COLUMNS
        and pd.api.types.is_numeric_dtype(features[column])
    ]

    feature_sets = {
        "compact_raw": compact_raw,
        "runtime_raw": list(
            dict.fromkeys(runtime_full + list(CORE.RAW_MODEL_FEATURE_COLUMNS))
        ),
        "all_runtime_numeric": all_numeric,
    }

    cleaned: dict[str, list[str]] = {}
    for name, columns in feature_sets.items():
        valid: list[str] = []
        for column in columns:
            if column in FORBIDDEN_RUNTIME_COLUMNS:
                continue
            if column not in features.columns:
                continue
            values = pd.to_numeric(features[column], errors="coerce")
            if not values.notna().any():
                continue
            if float(values.std(ddof=0)) <= 1e-12:
                continue
            valid.append(column)
        if valid:
            cleaned[name] = list(dict.fromkeys(valid))
    if not cleaned:
        raise RuntimeError("没有可用的运行时校准特征")
    return cleaned


def fitLinearityCurve(
    data: pd.DataFrame,
    base_prediction: np.ndarray,
    config: LinearityConfig,
) -> tuple[np.ndarray, np.ndarray]:
    if config.scale <= 0.0 or config.degree <= 0:
        return np.zeros(len(data), dtype=float), np.zeros(0, dtype=float)

    standard = data["standard_volume_m3"].to_numpy(dtype=float)
    base_error = relativeErrorPct(base_prediction, standard)
    mask = (
        data["condition_note"].eq("no_disturbance_reference")
        & data["flow_point"].between(40, 100)
    ).to_numpy()

    if int(mask.sum()) < config.degree + 1:
        return np.zeros(len(data), dtype=float), np.zeros(0, dtype=float)

    q = data["flow_point"].to_numpy(dtype=float)
    z = (q - 70.0) / 30.0
    design = np.column_stack(
        [z**degree for degree in range(config.degree + 1)]
    )
    x = design[mask]
    target = -base_error[mask]

```

```
    penalty = config.ridge_lambda * np.eye(x.shape[1], dtype=float)
    # 常数项也正则，防止D0稀疏样本整体平移过强。
    try:
        coef = np.linalg.solve(x.T @ x + penalty, x.T @ target)
    except np.linalg.LinAlgError:
        coef = np.linalg.lstsq(x.T @ x + penalty, x.T @ target, rcond=1e-12)[0]

    correction = config.scale * (design @ coef)
    correction = np.clip(correction, -0.25, 0.25)
    return correction, coef


def pairwiseSquaredDistance(x: np.ndarray) -> np.ndarray:
    x2 = np.sum(x * x, axis=1)[:, None]
    return np.maximum(x2 + x2.T - 2.0 * (x @ x.T), 0.0)


def eigKrrTrainingCorrection(
    kernel: np.ndarray,
    eigvalues: np.ndarray,
    eigvectors: np.ndarray,
    target: np.ndarray,
    lam: float,
) -> tuple[np.ndarray, np.ndarray]:
    projected = eigvectors.T @ target
    denom = np.maximum(eigvalues + float(lam), 1e-15)
    alpha = eigvectors @ (projected / denom)
    correction = kernel @ alpha
    return correction, alpha


def markdownTable(frame: pd.DataFrame, max_rows: int | None = None) -> str:
    if frame.empty:
        return ""
    use = frame.copy() if max_rows is None else frame.head(max_rows).copy()
    for column in use.columns:
        if pd.api.types.is_float_dtype(use[column]):
            use[column] = use[column].map(
                lambda value: "" if pd.isna(value) else f"{value:.8g}"
            )
    use = use.fillna("").astype(str)
    headers = list(use.columns)
    rows = use.values.tolist()
    widths = [
        max(len(headers[i]), *(len(row[i]) for row in rows))
        for i in range(len(headers))
    ]
    header = "| " + " | ".join(
        headers[i].ljust(widths[i]) for i in range(len(headers))
    ) + " |"
    separator = "| " + " | ".join(
        "-" * widths[i] for i in range(len(headers))
    ) + " |"
    body = [
        "| " + " | ".join(
            row[i].ljust(widths[i]) for i in range(len(headers))
        ) + " |"
        for row in rows
    ]
    return "\n".join([header, separator] + body)


def saveOfficialOutputs(
    data: pd.DataFrame,
    yuceJieguo: np.ndarray,
    summary: dict[str, object],
    groups: pd.DataFrame,
    windows: pd.DataFrame,
    ranking: pd.DataFrame,
    best: dict[str, object],
    model_arrays: dict[str, np.ndarray],
    model_meta: dict[str, object],
    base_summary: dict[str, object],
) -> None:
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    REPORT_DIR.mkdir(parents=True, exist_ok=True)

    submission = data[["window_id"]].reset_index(drop=True).copy()
    submission["model_volume_m3"] = np.asarray(yuceJieguo, dtype=float)

    submission_path = OUT_DIR / "submission_d0_rbf_postcalibrated_dual_kernel_krr.csv"
    submission.to_csv(
        submission_path,
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
    )

    pd.DataFrame([summary]).to_csv(
        OUT_DIR / "d0_rbf_postcalibrated_dual_kernel_krr_evaluation_summary.csv",
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
```

```
    )
    groups.to_csv(
        OUT_DIR / "d0_rbf_postcalibrated_dual_kernel_krr_evaluation_groups.csv",
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
    )
    windows.to_csv(
        OUT_DIR / "d0_rbf_postcalibrated_dual_kernel_krr_evaluation_windows.csv",
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
    )
    ranking.to_csv(
        OUT_DIR / "d0_rbf_postcalibrated_dual_kernel_krr_candidate_ranking.csv",
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
    )

    np.savez_compressed(
        OUT_DIR / "d0_rbf_postcalibrated_dual_kernel_krr_model.npz",
        **model_arrays,
    )
    (OUT_DIR / "d0_rbf_postcalibrated_dual_kernel_krr_model_meta.json").write_text(
        json.dumps(model_meta, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

    promoted_path = ""
    if bool(summary["pass_all_3"]) and PROMOTE_TO_SUBMISSION_FINAL_IF_PASS:
        final_path = OUT_DIR / "submission_final.csv"
        if final_path.exists():
            backup = OUT_DIR / "submission_final_before_postcalibration.csv"
            shutil.copy2(final_path, backup)
        shutil.copy2(submission_path, final_path)
        promoted_path = str(final_path)

    report_cols = [
        "rank",
        "candidate_id",
        "feature_set",
        "linearity_config",
        "gamma",
        "lambda",
        "rho",
        "pass_all_3",
        "official_fail_count",
        "official_weighted_gap",
        "mae_pct",
        "group_pass",
        "u_nor_L_pct",
        "u_nor_r_pct",
        "u_nor_d_pct",
        "max_abs_total_correction_pct",
    ]
    top = ranking[report_cols].head(30)

    report = f"""# 受约束 D0-RBF 后校准双核 KRR

## 模型定位

本脚本以冻结的物理--稳定性双核 KRR 为主模型，仅增加一个完整数据校准层，
并直接使用 `evaluate_submission.py` 的三个指标公式选择候选。

这是已知 159 窗口上的全量标定结果，不代表未知工况泛化性能。

## 冻结主模型

```text
krr_dual_ps_pg0.05_rg0.005_b0.5_lam0.3_d0lin
```

主模型全量结果

{markdownTable(pd.DataFrame([base_summary]), 1)}

后校准双核 KRR 最终结果

{markdownTable(pd.DataFrame([summary]), 1)}

最佳校准候选

```json
{json.dumps(best, ensure_ascii=False, indent=2)}
```

候选排名前 30

{markdownTable(top, 30)}

最坏重复组

```

```
{markdownTable(groups.sort_values(
    ["pass_group", "repeatability_sd_pct", "abs_mean_error_pct"],
    ascending=[True, False, False],
).head(15), 15)}

输出文件

- `code/outputs/submission_d0_rbf_postcalibrated_dual_kernel_krr.csv`
- `code/outputs/d0_rbf_postcalibrated_dual_kernel_krr_evaluation_summary.csv`
- `code/outputs/d0_rbf_postcalibrated_dual_kernel_krr_evaluation_groups.csv`
- `code/outputs/d0_rbf_postcalibrated_dual_kernel_krr_evaluation_windows.csv`
- `code/outputs/d0_rbf_postcalibrated_dual_kernel_krr_candidate_ranking.csv`
- `code/outputs/d0_rbf_postcalibrated_dual_kernel_krr_model.npz`
- `code/outputs/d0_rbf_postcalibrated_dual_kernel_krr_model_meta.json`

自动提升到 `submission_final.csv`: `{promoted_path or "否"}`
"""
    (REPORT_DIR / "D0_RBF_POSTCALIBRATED_DUAL_KERNEL_KRR_REPORT.md").write_text(
        report,
        encoding="utf-8",
)

def optionallyRunExternalEvaluator(submission_path: Path) -> None:
    candidates = [
        ROOT / "A题 (1)" / "evaluate_submission.py",
        HERE / "evaluate_submission.py",
    ]
    evaluator = next((path for path in candidates if path.exists()), None)
    if evaluator is None:
        return
    output_dir = OUT_DIR / "d0_rbf_postcalibrated_dual_kernel_krr_external_evaluation"
    try:
        subprocess.run(
            [
                sys.executable,
                str(evaluator),
                str(submission_path),
                "--output-dir",
                str(output_dir),
            ],
            check=True,
)
    except Exception as exc:
        print(
            f"[warning] 外部 evaluate_submission.py 调用失败: {exc}",
            flush=True,
)

def run() -> None:
    print(f"=== Running {SCRIPT_VERSION} ===", flush=True)
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    REPORT_DIR.mkdir(parents=True, exist_ok=True)

    data = CORE.loadDataWithRawFeatures().reset_index(drop=True)
    features = CORE.modelFeatureFrame(data).reset_index(drop=True)

    if len(data) != 159:
        print(
            f"[warning] 当前窗口数为 {len(data)}, 不是预期的159; "
            "仍按实际数据运行。",
            flush=True,
)

    frozen_spec, frozen_model, frozen_prediction, frozen_diag = (
        buildFrozenDualKernelModel(data, features)
)
    base_summary, base_groups, base_windows = officialEvaluate(
        data,
        frozen_prediction,
)
    print("\n=== Frozen physical-stability dual-kernel KRR evaluation ===")
    print(pd.Series(base_summary).to_string())

    feature_sets = buildCalibrationFeatureSets(features)
    print("\nCalibration feature sets:")
    for name, columns in feature_sets.items():
        print(f" {name}: {len(columns)} columns")

    standard = data["standard_volume_m3"].to_numpy(dtype=float)
    line_configs = [
        LinearityConfig(*values)
        for values in LINEARITY_CONFIGS
    ]

    rows: list[dict[str, object]] = []
    candidate_payloads: dict[str, dict[str, object]] = {}

    # 基准候选。
    fail_count, weighted_gap, gap_l, gap_r, gap_d = metricGaps(base_summary)
    base_row = {
            "candidate_id": "frozen_dual_kernel_no_extra_calibration",
```

```
        "feature_set": "none",
        "linearity_config": "none",
        "gamma": 0.0,
        "lambda": 0.0,
        "rho": 0.0,
        **base_summary,
        "official_fail_count": fail_count,
        "official_weighted_gap": weighted_gap,
        "gap_L": gap_l,
        "gap_R": gap_r,
        "gap_D": gap_d,
        "group_fail_count": int(
            len(base_groups) - base_groups["pass_group"].sum()
        ),
        "complexity_penalty": 1e9,
        "max_abs_total_correction_pct": 0.0,
        "rms_total_correction_pct": 0.0,
    }
    rows.append(base_row)
    candidate_payloads[base_row["candidate_id"]] = {
        "prediction": frozen_prediction,
        "linearity_coef": np.zeros(0),
        "linearity_correction": np.zeros(len(data)),
        "krr_correction": np.zeros(len(data)),
        "total_correction": np.zeros(len(data)),
        "feature_columns": [],
        "standardizer": None,
        "train_x": np.zeros((len(data), 0)),
        "alpha": np.zeros(len(data)),
    }

    # 预计算不同特征集合和gamma的核特征分解。
    kernel_cache: dict[
        tuple[str, float],
        tuple[
            RobustStandardizer,
            np.ndarray,
            np.ndarray,
            np.ndarray,
            np.ndarray,
        ],
    ] = {}
    for feature_set_name, columns in feature_sets.items():
        standardizer = RobustStandardizer.fit(features, columns)
        x = standardizer.transform(features)
        dist2 = pairwiseSquaredDistance(x)
        for gamma in GAMMA_GRID:
            kernel = np.exp(-float(gamma) * dist2)
            kernel = 0.5 * (kernel + kernel.T)
            eigvalues, eigvectors = np.linalg.eigh(kernel)
            eigvalues = np.maximum(eigvalues, 0.0)
            kernel_cache[(feature_set_name, gamma)] = (
                standardizer,
                x,
                kernel,
                eigvalues,
                eigvectors,
            )

    candidate_count = (
        len(line_configs)
        * len(feature_sets)
        * len(GAMMA_GRID)
        * len(LAMBDA_GRID)
        * len(RHO_GRID)
    )
    print(f"\nSearching {candidate_count} targeted calibration candidates...")

    evaluated = 0
    for line_config in line_configs:
        line_correction, line_coef = fitLinearityCurve(
            data,
            frozen_prediction,
            line_config,
        )
        prediction_after_line = frozen_prediction * (
            1.0 + line_correction / 100.0
        )
        residual_target = (
            standard / np.maximum(prediction_after_line, 1e-12) - 1.0
        ) * 100.0

        for feature_set_name in feature_sets:
            for gamma in GAMMA_GRID:
                (
                    standardizer,
                    x,
                    kernel,
                    eigvalues,
                    eigvectors,
                ) = kernel_cache[(feature_set_name, gamma)]

                projected = eigvectors.T @ residual_target
```

```
            for lam in LAMBDA_GRID:
                denom = np.maximum(eigvalues + float(lam), 1e-15)
                alpha = eigvectors @ (projected / denom)
                raw_krr_correction = kernel @ alpha

                for rho in RHO_GRID:
                    krr_correction = float(rho) * raw_krr_correction
                    total_correction = line_correction + krr_correction
                    total_correction = np.clip(
                        total_correction,
                        -CORRECTION_CLIP_PCT,
                        CORRECTION_CLIP_PCT,
                    )
                    yuceJieguo = frozen_prediction * (
                        1.0 + total_correction / 100.0
                    )

                    summary, groups, windows = officialEvaluate(
                        data,
                        yuceJieguo,
                    )
                    (
                        fail_count,
                        weighted_gap,
                        gap_l,
                        gap_r,
                        gap_d,
                    ) = metricGaps(summary)

                    candidate_id = (
                        f"{feature_set_name}"
                        f"__{line_config.name}"
                        f"__g{gamma:g}"
                        f"__lam{lam:g}"
                        f"__rho{rho:g}"
                    )
                    row = {
                        "candidate_id": candidate_id,
                        "feature_set": feature_set_name,
                        "linearity_config": line_config.name,
                        "linearity_degree": line_config.degree,
                        "linearity_lambda": line_config.ridge_lambda,
                        "linearity_scale": line_config.scale,
                        "gamma": gamma,
                        "lambda": lam,
                        "rho": rho,
                        **summary,
                        "official_fail_count": fail_count,
                        "official_weighted_gap": weighted_gap,
                        "gap_L": gap_l,
                        "gap_R": gap_r,
                        "gap_D": gap_d,
                        "group_fail_count": int(
                            len(groups) - groups["pass_group"].sum()
                        ),
                        # 已经通过时优先更大的lambda、更小的特征集合。
                        "complexity_penalty": (
                            -math.log10(max(float(lam), 1e-15))
                            + 0.002 * len(feature_sets[feature_set_name])
                        ),
                        "max_abs_total_correction_pct": float(
                            np.max(np.abs(total_correction))
                        ),
                        "rms_total_correction_pct": float(
                            np.sqrt(np.mean(total_correction**2))
                        ),
                    }
                    rows.append(row)
                    candidate_payloads[candidate_id] = {
                        "prediction": yuceJieguo,
                        "summary": summary,
                        "groups": groups,
                        "windows": windows,
                        "linearity_coef": line_coef,
                        "linearity_correction": line_correction,
                        "krr_correction": krr_correction,
                        "total_correction": total_correction,
                        "feature_columns": feature_sets[feature_set_name],
                        "standardizer": standardizer,
                        "train_x": x,
                        "alpha": alpha,
                    }
                    evaluated += 1

        print(
            f"  completed linearity config {line_config.name}; "
            f"evaluated={evaluated}",
            flush=True,
        )

    ranking = pd.DataFrame(rows)
    ranking["_selection_key"] = ranking.apply(
        lambda row: selectionKey(row.to_dict()),
```

```
        axis=1,
    )
    ranking = (
        ranking.sort_values("_selection_key", kind="stable")
        .drop(columns=["_selection_key"])
        .reset_index(drop=True)
    )
    ranking.insert(0, "rank", np.arange(1, len(ranking) + 1))

    best = ranking.iloc[0].to_dict()
    best_id = str(best["candidate_id"])
    payload = candidate_payloads[best_id]
    best_prediction = np.asarray(payload["prediction"], dtype=float)
    best_summary, best_groups, best_windows = officialEvaluate(
        data,
        best_prediction,
    )

    std: RobustStandardizer | None = payload["standardizer"]
    model_arrays: dict[str, np.ndarray] = {
        "frozen_prediction_m3": frozen_prediction,
        "linearity_coef": np.asarray(payload["linearity_coef"], dtype=float),
        "linearity_correction_pct": np.asarray(
            payload["linearity_correction"],
            dtype=float,
        ),
        "calibration_train_x": np.asarray(payload["train_x"], dtype=float),
        "calibration_alpha": np.asarray(payload["alpha"], dtype=float),
        "calibration_correction_pct": np.asarray(
            payload["krr_correction"],
            dtype=float,
        ),
        "total_correction_pct": np.asarray(
            payload["total_correction"],
            dtype=float,
        ),
    }
    if std is not None:
        model_arrays.update(
            {
                "calibration_feature_columns": np.asarray(
                    std.columns,
                    dtype="U",
                ),
                "calibration_median": std.median,
                "calibration_mean": std.mean,
                "calibration_scale": std.scale,
            }
        )

    model_meta: dict[str, object] = {
        "script_version": SCRIPT_VERSION,
        "core_module": CORE.__name__,
        "model_type": "dual_kernel_krr_plus_constrained_d0_rbf_postcalibration",
        "frozen_dual_kernel_spec": asdict(frozen_spec),
        "best_candidate": best,
        "official_limits": OFFICIAL_LIMITS,
        "correction_clip_pct": CORRECTION_CLIP_PCT,
        "feature_columns": payload["feature_columns"],
        "warning": (
            "This calibration layer was fitted and selected on the same 159 labeled "
            "windows used by the official evaluator. It is submission/full-data "
            "calibration, not independent generalisation evidence."
        ),
    }

    saveOfficialOutputs(
        data=data,
        yuceJieguo=best_prediction,
        summary=best_summary,
        groups=best_groups,
        windows=best_windows,
        ranking=ranking,
        best=best,
        model_arrays=model_arrays,
        model_meta=model_meta,
        base_summary=base_summary,
    )

    print("\n=== Constrained D0-RBF postcalibrated dual-kernel KRR ===")
    print(pd.Series(best).to_string())
    print("\n=== Official evaluation ===")
    print(pd.Series(best_summary).to_string())

    submission_path = OUT_DIR / "submission_d0_rbf_postcalibrated_dual_kernel_krr.csv"
    optionallyRunExternalEvaluator(submission_path)

    print(f"\nSubmission: {submission_path}")
    print(
        "Summary: "
        f"{OUT_DIR / 'd0_rbf_postcalibrated_dual_kernel_krr_evaluation_summary.csv'}"
    )
    print(
```

```
        "Ranking: "
        f"{OUT_DIR / 'd0_rbf_postcalibrated_dual_kernel_krr_candidate_ranking.csv'}"
    )
    print(
        "Report: "
        f"{REPORT_DIR / 'D0_RBF_POSTCALIBRATED_DUAL_KERNEL_KRR_REPORT.md'}"
    )


if __name__ == "__main__":
    run()
```

B.11 官方指标复验：official_evaluator.py

```
from __future__ import annotations

# 论文附录复现代码。
# 题设复验：按官方分组和阈值计算全部评价指标。
# 命名与注释已统一整理；计算公式、数据字段和模型参数保持不变。

import argparse
import math
from pathlib import Path

import numpy as np
import pandas as pd


HERE = Path(__file__).resolve().parent
ROOT = HERE.parents[1]
DATA = ROOT / "A题 (1)" / "attachment1_window_data.csv"


def relativeErrorPct(moxingTiji: pd.Series, biaozhunTiji: pd.Series) -> pd.Series:
    return (moxingTiji.astype(float) - biaozhunTiji.astype(float)) / biaozhunTiji.astype(float) * 100.0


def groupTable(df: pd.DataFrame) -> pd.DataFrame:
    rows = []
    for (date, flow_point), group in df.groupby(["date", "flow_point"], sort=True):
        if len(group) < 3:
            continue
        values = group["model_error_pct"].astype(float)
        mean_err = float(values.mean())
        sd = float(values.std(ddof=1))
        rows.append(
            {
                "date": str(date),
                "flow_point": int(flow_point),
                "n": int(len(group)),
                "mean_error_pct": mean_err,
                "repeatability_sd_pct": sd,
                "range_pct": float(values.max() - values.min()),
                "pass_group": abs(mean_err) <= 0.2 and sd <= 0.040,
            }
        )
    return pd.DataFrame(rows)


def linearityUL(df: pd.DataFrame) -> float:
    use = df[(df["condition_note"].eq("no_disturbance_reference")) & (df["flow_point"].between(40, 100))]
    by_flow = use.groupby("flow_point")["model_error_pct"].mean()
    if len(by_flow) <= 1:
        return float("nan")
    return math.sqrt(float((by_flow**2).sum()) / (len(by_flow) - 1))


def disturbanceUD(df: pd.DataFrame) -> float:
    use = df[df["flow_point"].between(40, 100)].copy()
    base = (
        use[use["condition_note"].eq("no_disturbance_reference")]
        .groupby("flow_point")["model_error_pct"]
        .agg(["mean", "count", "std"])
        .rename(columns={"mean": "baseline_mean", "count": "baseline_n", "std": "baseline_sd"})
        .reset_index()
    )
    disturb = (
        use[use["condition_note"].eq("disturbed_test")]
        .groupby(["disturbance_id", "flow_point"])["model_error_pct"]
        .agg(["mean", "count", "std"])
        .rename(columns={"mean": "disturb_mean", "count": "disturb_n", "std": "disturb_sd"})
        .reset_index()
    )
    detail = disturb.merge(base, on="flow_point", how="left")
    detail["disturb_sd"] = detail["disturb_sd"].fillna(0.0)
    detail["abs_drift"] = (detail["baseline_mean"] - detail["disturb_mean"]).abs()
    available = detail[(detail["baseline_n"] >= 1) & (detail["disturb_n"] >= 1)].copy()
    if available.empty:
        return float("nan")
    u_d_c = float(available["abs_drift"].max()) / math.sqrt(3.0)
```

```
    u_d_r = float(available["disturb_sd"].max())
    return math.sqrt(u_d_c * u_d_c + u_d_r * u_d_r)


def evaluate(submission: Path, output_dir: Path) -> None:
    data = pd.read_csv(DATA, encoding="utf-8-sig")
    sub = pd.read_csv(submission, encoding="utf-8-sig")
    if not {"window_id", "model_volume_m3"}.issubset(sub.columns):
        raise ValueError("submission must contain columns: window_id, model_volume_m3")

    df = data.merge(sub[["window_id", "model_volume_m3"]], on="window_id", how="left")
    if df["model_volume_m3"].isna().any():
        missing = int(df["model_volume_m3"].isna().sum())
        raise ValueError(f"submission misses {missing} window_id rows")

    df["model_error_pct"] = relativeErrorPct(df["model_volume_m3"], df["standard_volume_m3"])
    groups = groupTable(df)
    u_l = linearityUL(df)
    u_r = float(groups["repeatability_sd_pct"].max()) if not groups.empty else float("nan")
    u_d = disturbanceUD(df)

    summary = pd.DataFrame(
        [
            {
                "window_count": int(len(df)),
                "mae_pct": float(df["model_error_pct"].abs().mean()),
                "mean_error_pct": float(df["model_error_pct"].mean()),
                "group_pass": f"{int(groups['pass_group'].sum())}/{len(groups)}",
                "u_nor_L_pct": u_l,
                "u_nor_r_pct": u_r,
                "u_nor_d_pct": u_d,
                "pass_u_nor_L": bool(pd.notna(u_l) and u_l < 0.036),
                "pass_u_nor_r": bool(pd.notna(u_r) and u_r < 0.040),
                "pass_u_nor_d": bool(pd.notna(u_d) and u_d < 0.115),
                "pass_all_3": bool(pd.notna(u_l) and u_l < 0.036 and pd.notna(u_r) and u_r < 0.040 and pd.notna(u_d) and u_d < 0.115),
            }
        ]
    )
    output_dir.mkdir(parents=True, exist_ok=True)
    summary.to_csv(output_dir / "evaluation_summary.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    groups.to_csv(output_dir / "evaluation_groups.csv", index=False, encoding="utf-8-sig", float_format="%.12g")
    df[["window_id", "date", "flow_point", "standard_volume_m3", "model_volume_m3", "model_error_pct"]].to_csv(
        output_dir / "evaluation_windows.csv",
        index=False,
        encoding="utf-8-sig",
        float_format="%.12g",
    )
    print(summary.to_string(index=False))


def main() -> None:
    parser = argparse.ArgumentParser(description="Evaluate an ultrasonic-flowmeter modeling submission.")
    parser.add_argument("submission", type=Path, help="CSV with columns window_id, model_volume_m3")
    parser.add_argument("--output-dir", type=Path, default=HERE / "evaluation_output")
    args = parser.parse_args()
    evaluate(args.submission, args.output_dir)


if __name__ == "__main__":
    main()
```